Clarity

Phase 2

Team 5

Member Name	Member Roll #	Primary Responsibility
Mohammad Hamza Iqbal (team lead)	23L-0848	Use Cases 1-7 and design class diagram
Bilal Kashif	23L-0757	Use Cases 15-21 and design class diagram
Mawahid Abbas	23L-0613	Use Cases 8-14 and design class diagram

Project Proposal

Abstract

Clarity is an all-in-one productivity application designed for web and mobile that unifies task management, scheduling, collaboration, and communication into a single platform. Built using Angular, Ionic, and Capacitor with real-time synchronization through Socket.IO, it offers an intuitive drag-and-drop interface, collaborative editing, and seamless integration with third- party services such as Gmail, Google Calendar, and Google Contacts. By incorporating AI-powered automation and insights, Clarity enhances productivity through intelligent scheduling, reminders, and progress analytics, while its unified environment reduces the inefficiencies caused by switching between multiple applications.

1. Introduction

Apps that we use today have many tools in them. We use a notes app for our to-do lists, documents for our papers, and calendars for reminders. However, there is not a single app that incorporates everything in it. Productivity means juggling many things like multitasking, keeping track of progress, collaborating with peers, and much more. Switching between apps and organizing has become complex.

Clarity is an all-in-one mobile and web app that brings these features together to create ease for users, especially students. At its core, Clarity offers a drag-and-drop based interface for intuitive task and project organization, with support for interconnected task threads that adapt dynamically as work progresses. It also supports collaborative editing, direct task-level communication, and third-party integrations such as Gmail, Google Calendar and Google Contacts to make teamwork smoother and more efficient.

What sets Clarity apart is its intelligent automation and AI-powered insights. By unifying everything in one coherent app, it makes doing tasks simpler and more engaging while creating an environment that supports learning and productivity.

2.Goals and Objectives

Our project's objectives include:

- Developing an all-in-one productivity application for web and mobile that combines task management, scheduling, collaboration, and communication.
- Providing an intuitive drag-and-drop interface with support for interconnected task threads that adapt dynamically as work progresses.
- Enabling real-time collaborative editing to improve teamwork efficiency.

Clarity - Unified Workspace for Tasks and Projects 3

- Integrating third-party services such as Gmail, Google Calendar and Google Contacts to streamline workflows.
- Implementing AI-powered automation for reminders, intelligent scheduling, and productivity insights.
- Offering personalized yet coherent workspace layouts through a semi-grid interface to enhance usability.
- Providing analytics, graphs, and visualizations to track productivity and progress effectively.

3.Scope of the Project

Our project mainly focuses on developing an all-in-one productivity application for web and mobile that integrates task management, collaboration, scheduling, and automation into a single platform. The system will be built using Angular for both the backend and frontend, with Ionic and Capacitor to ensure cross-platform compatibility on web and mobile devices.

For real-time collaboration, Socket.IO will be used to synchronize updates between users,

enabling multiple participants to work on shared projects simultaneously. Since offline syncing is not required, the system will operate with live data only. Third-party APIs such as Google Calendar, Google Contacts, and Gmail will be supported, with integration remaining optional for each user depending on their needs. User activity and project data will initially be stored in structured JSON objects within the application, reducing the need for a dedicated database at this stage.

Artificial Intelligence will form a key part of the system. The AI module will analyze task history to identify productivity bottlenecks, highlight areas for improvement, and generate insights for better workflow management. An AI agent will also be available to handle routine actions such as sending invites and scheduling tasks or events directly into integrated calendars. For analytics and visualizations, lightweight solutions such as Chart.js or Recharts will be adopted to display task completion rates, productivity trends, and progress summaries in a clear and interactive manner.

The project will not include a dedicated database in its initial implementation, as all user activity and project data will be managed through structured JSON objects within the application. Furthermore, offline synchronization will not be supported, since the system is designed to operate exclusively with live data in real time.

4. Initial Study and Work Done So Far

As far as our research is concerned, many tools already exist to manage productivity, tasks, collaboration, and scheduling. Applications such as Trello, Notion, and Asana focus on tasks and project organization; Gmail and Google Contacts focus on communication; and Google Calendar manages scheduling. However, no single tool brings all of these together in one place, forcing users to switch between multiple platforms to accomplish different parts of their work.

Studies show that switching between apps wastes significant time and reduces productivity. A *Harvard Business Review* report found that employees lose hours each week regaining focus after toggling between tools, which contributes to mental fatigue and lower efficiency [1].

Other research highlights that fragmented application environments, where different apps do not

integrate or synchronize properly, cause delays, misunderstandings, and additional effort in teamwork [2]. Furthermore, real-time collaboration has been identified as a critical factor for improving productivity and reducing communication overhead in digital teamwork [3].

Our aim is to integrate these fragmented features like task management, scheduling, communication, collaboration, and analytics into one web and mobile application. So far, we have reviewed existing productivity platforms, explored real-time synchronization technologies such as Socket.IO, and identified relevant third-party APIs (Google Calendar, Gmail, Google Contacts). These findings will guide our system architecture and feature set for Clarity.

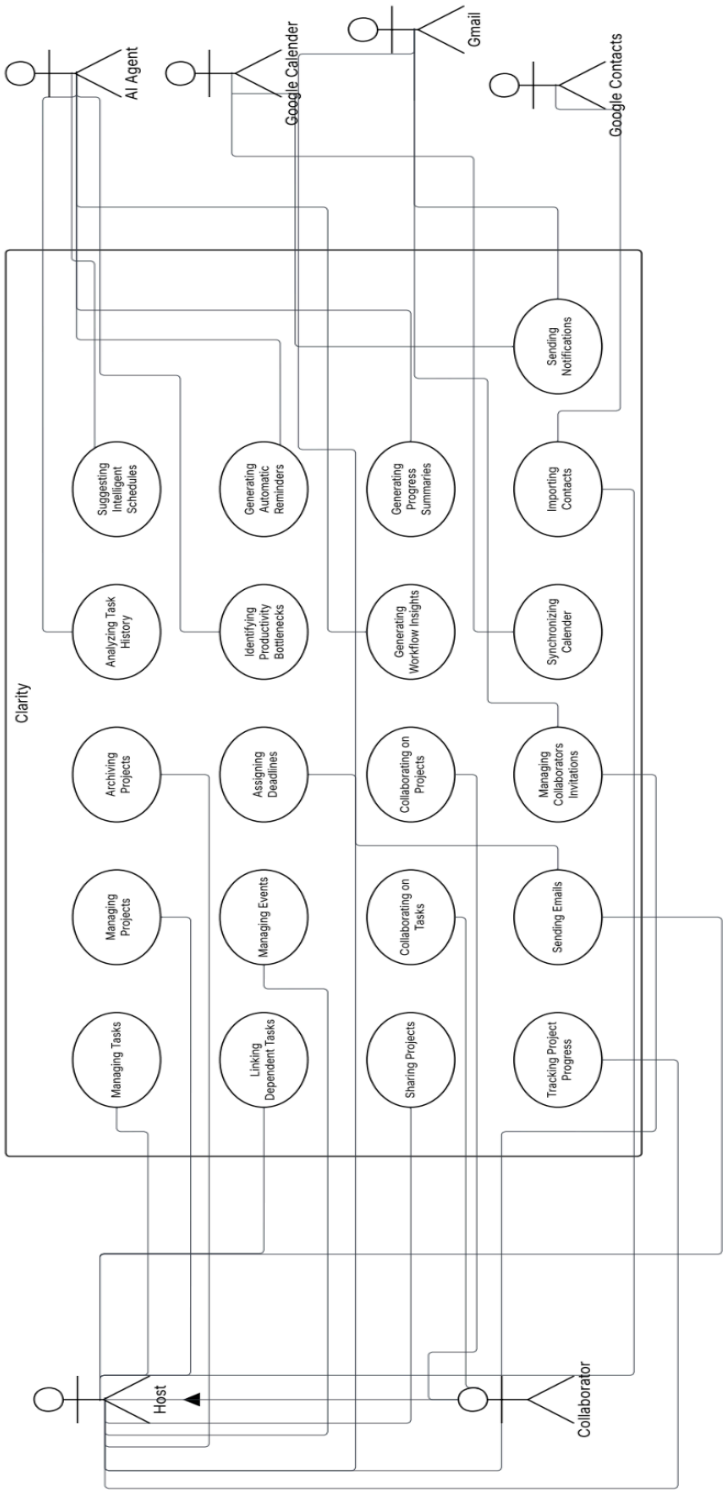
References

[1] A. Waber, "How much time and energy do we waste toggling between applications?," *Harvard Business Review*, Aug. 2022. [Online]. Available: https://hbr.org/2022/08/how-much-time-and-energy-do-we-waste-toggling-between applications

[2] M. S. González et al., "Tools for Online Collaboration: Do they contribute to Improve Teamwork?," *Revista Facultad de Ingeniería Universidad de Antioquia*, no. 77, pp. 23– 32, 2015. [Online]. Available: https://www.researchgate.net/publication/287157555_Tools_for_Online_Collaboration_Do_they_contribute_to_Improve_Teamwork

[3] "Real-time collaboration: What it is and why it matters," *ProofHub*, 2023. [Online]. Available: https://www.proofhub.com/articles/real-time-collaboration

Use Case Diagram



Use Case Description 1

Identifier	UC-01
Name	Managing Tasks
Summary	This use case describes how the host manages tasks within the system, including creating, editing, deleting, and organizing tasks.
Priority	High
Actors	Host
Pre-condition(s)	Host must be logged into the system. A project or workspace must already exist.
Post-condition(s)	Tasks are successfully updated in the system. All participants can view the updated task list in real time.
Dependencies	Depends on Managing Projects i.e, a project or workspace must exist.

Typical Course of Action

S#	Actor Action	System Response
1	Host selects “Manage Tasks” option.	System displays the task management interface.
2	Host creates a new task by entering details (title, description, deadline).	System verifies and saves the task.
3	Host edits an existing task.	System updates the task details.
4	Host deletes a task.	System removes the task and updates the list.
5	Host organizes tasks by drag-and-drop into categories or priority.	System updates the task order and synchronizes changes in real time.

Alternate Course of Action

S#	Actor Action	System Response
2a	Host enters incomplete or invalid task details.	System prompts host to correct the details.
3a	Host attempts to edit a non-existent or deleted task.	System displays an error message.

4a	Host tries to delete a locked or assigned task.	System displays a warning and prevents deletion.

Use Case Description 2

Exceptions:
System fails to save or update tasks due to server issues.

Identifier	UC-02
Name	Linking Dependent Tasks
Summary	The host links tasks together to establish dependencies (e.g., one task must be completed before another begins).
Priority	Medium
Actors	Host
Pre-condition(s)	Tasks must already exist in the project.
Post-condition(s)	Dependency relationships are created and visible in task flow.
Dependencies	Depends on Managing Tasks as tasks must exist first.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Host selects tasks to be linked.	System highlights selected tasks.
2	Host sets dependency relationship (e.g., Task B depends on Task A).	System records and displays dependency.

Alternate Course of Action		
S#	**Actor Action**	**System Response**
2a	Host attempts to link a task to itself.	System shows error.
2b	Host tries to link circular dependencies (A→B→A).	System blocks and alerts host.

Exceptions:
Dependency not saved due to server error.

Use Case Description 3

Identifier	UC-03
Name	Sharing Projects
Summary	The host shares a project with collaborators for joint work.
Priority	High
Actors	Host
Pre-condition(s)	Project must exist. Collaborators must have accounts.
Post-condition(s)	Project is shared, and collaborators gain access.
Dependencies	Depends on Managing Projects as a project must exist.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Host selects “Share Project.”	System displays sharing options.
2	Host enters collaborator’s email/ID.	System validates collaborator.
3	Host sets access level (view/edit).	System applies permissions.
4	Host confirms sharing.	System sends invitation to collaborator.

Alternate Course of Action		
S#	**Actor Action**	**System Response**
2a	Invalid email ID entered.	System displays error.
3a	Host tries to assign unsupported permission.	System shows warning.

Exceptions:
Email server or notification service unavailable.

Use Case Description 4

Identifier	UC-04
Name	Tracking Project Progress
Summary	The host tracks the project's progress through visual dashboards, analytics, and reports.
Priority	Medium
Actors	Host
Pre-condition(s)	Project and tasks must exist.
Post-condition(s)	Progress data is displayed.
Dependencies	Depends on Managing Tasks and Linking Dependent Tasks as tasks and dependencies provide progress data.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Host selects "View Progress."	System loads dashboard.
2	Host selects metrics (e.g., completion rate, deadlines).	System generates visual reports.

Alternate Course of Action		
S#	**Actor Action**	**System Response**
2a	No tasks exist.	System shows empty progress message.

Exceptions:
Analytics service fails.

Use Case Description 5

Identifier	UC-05	
Name	Managing Projects	
Summary	The host creates, edits, or deletes entire projects.	
Priority	High	
Actors	Host	
Pre-condition(s)	Host must be logged in.	
Post-condition(s)	Project list updated.	
Dependencies	No dependency	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Host selects “Manage Projects.”	System displays projects list.
2	Host creates/edits/deletes project.	System updates project records.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
2a	Host tries to delete project with active collaborators.	System asks for confirmation.

Exceptions:
Project not saved or deleted due to system error.

Use Case Description 6

Identifier	UC-06	
Name	Managing Events	
Summary	The host manages events within a project (e.g., deadlines, meetings).	
Priority	Medium	
Actors	Host	
Pre-condition(s)	Project must exist.	
Post-condition(s)	Events updated and synchronized.	
Dependencies	Depends on Managing Projects as events are tied to projects.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Host selects "Manage Events."	System shows calendar view.
2	Host creates, edits or deletes event.	System updates calendar.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
2a	User enters an invalid date.	System prompts correction.
2b	User enters an invalid time.	System prompts correction.

Exceptions:
Integration with calendar API fails.

Use Case Description 7

Identifier	UC-07
Name	Collaborating on Tasks
Summary	Collaborators work jointly on tasks, adding comments, making edits, and tracking updates.
Priority	High
Actors	Collaborator
Pre-condition(s)	Project must be shared with collaborator.
Post-condition(s)	Task collaboration history updated.
Dependencies	Depends on Sharing Projects and Managing Tasks as project must be shared and tasks must exist.

Typical Course of Action

S#	Actor Action	System Response
1	Collaborator opens shared task.	System displays task details.
2	Collaborator adds comments or edits task.	System updates and synchronizes changes.
3	Collaborator checks activity log.	System shows updates from all collaborators.

Alternate Course of Action

S#	Actor Action	System Response
2a	Collaborator tries to edit with read-only access.	System denies and shows warning.

Exceptions:
Synchronization failure due to connection issue.

Use Case Description 8

Identifier	UC-08
Name	Sending Emails
Summary	The host sends emails to collaborators through the integrated Gmail service.
Priority	Medium
Actors	**Primary**: Host **Supporting**: Gmail
Pre-condition(s)	Host must be logged in. Collaborator's email address must be available. Gmail integration must be active.
Post-condition(s)	Email is successfully sent and collaborator receives it.
Dependencies:	Depends on Sharing Projects (project context for communication) and requires Gmail integration.

Typical Course of Action

S#	Actor Action	System Response
1	Host selects "Send Email."	System displays email composition window.
2	Host enters recipient(s), subject, and message.	System validates email details.
3	Host clicks "Send."	System routes message to Gmail API.
4	Gmail sends the email.	System confirms successful delivery.

Alternate Course of Action

S#	Actor Action	System Response
2a	Invalid email format entered.	System shows error.
3a	Email exceeds attachment size.	System notifies host.

Exceptions:

Gmail API unavailable.

Internet connectivity lost.

Use Case Description 9

Identifier	UC-09	
Name	Archiving Projects	
Summary	The host archives completed projects to keep the workspace organized.	
Priority	Medium	
Actors	Host	
Pre-condition(s)	Project must exist.	
Post-condition(s)	Project is marked as archived and hidden from active workspace.	
Dependencies	Depends on Managing Projects (a project must already exist).	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Host selects "Archive Project."	System confirms action.
2	Host approves archive request.	System archives project and moves it to archive folder.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1a	Host cancels request.	Project remains active.

Exceptions:
Archive action fails due to server error.

Use Case Description 10

Identifier	UC-10
Name	Assigning Deadlines
Summary	The host assigns deadlines to tasks or projects.
Priority	High
Actors	Host
Pre-condition(s)	Task or project must exist.
Post-condition(s)	Deadline assigned and reflected in project calendar.
Dependencies	Depends on Managing Tasks (tasks must exist before deadlines can be assigned).

Typical Course of Action

S#	Actor Action	System Response
1	Host selects task/project.	System displays details.
2	Host sets deadline date or time.	System validates entry.
3	Host saves changes.	System updates and displays deadline in calendar.

Alternate Course of Action

S#	Actor Action	System Response
2a	Invalid date or time entered (past deadline).	System prompts correction.

Exceptions:

Calendar service unavailable.

Use Case Description 11

Identifier	UC-11
Name	Collaborating on Projects
Summary	Collaborators work jointly on shared projects, including editing details, assigning tasks, and tracking updates.
Priority	High
Actors	Collaborator
Pre-condition(s)	Project must be shared with collaborator.
Post-condition(s)	Collaboration changes are reflected in real time.
Dependencies	Depends on Sharing Projects (project must be shared first).

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Collaborator opens shared project.	System displays project details.
2	Collaborator edits or updates project details.	System synchronizes updates.
3	Collaborator views changes from others.	System refreshes project data.

Alternate Course of Action		
S#	**Actor Action**	**System Response**
2a	Collaborator has read-only access.	System blocks editing.

Exceptions:
Sync failure due to poor connectivity.

Use Case Description 12

Identifier	UC-12
Name	Managing Collaborator Invitations
Summary	The host invites collaborators to projects via Gmail integration.
Priority	High
Actors	**Primary**: Host **Supporting**: Gmail
Pre-condition(s)	Project must exist. Collaborator must have an email address.
Post-condition(s)	Invitation is sent and collaborator notified.
Dependencies	Depends on Managing Projects (project exists to invite into) and requires Gmail integration.

Typical Course of Action

S#	Actor Action	System Response
1	Host selects "Invite Collaborator."	System displays invitation form.
2	Host enters collaborator's email.	System validates email.
3	Host sends invitation.	System routes request to Gmail API.
4	Gmail sends invitation email.	System confirms delivery.

Alternate Course of Action

S#	Actor Action	System Response
2a	Invalid email entered.	System shows error.
3a	Host cancels request.	System does not send invitation.

Exceptions:
Gmail API fails to send invitation.

Use Case Description 13

Identifier	UC-13
Name	Analyzing Task History
Summary	The AI agent analyzes past task data to generate insights.
Priority	Medium
Actors	AI Agent
Pre-condition(s)	Task history must exist.
Post-condition(s)	AI-generated analysis provided to host.
Dependencies	Depends on Managing Tasks as tasks history must exist.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	AI agent scans completed task history.	System retrieves stored task data.
2	AI agent applies analysis algorithms.	System generates insights and displays them.

Alternate Course of Action		
S#	**Actor Action**	**System Response**
2a	Insufficient data provided.	AI informs host that no meaningful analysis can be generated.

Exceptions:
AI service fails.

Use Case Description 14

Identifier	UC-14
Name	Identifying Productivity Bottlenecks
Summary	The AI agent identifies tasks or workflows that cause delays and reduces productivity.
Priority	Medium
Actors	AI Agent
Pre-condition(s)	Task history and progress data must exist.
Post-condition(s)	Host receives suggestions for resolving bottlenecks.
Dependencies	Depends on Analyzing History of Tasks as analysis is prerequisite for identifying bottlenecks.

Typical Course of Action

S#	Actor Action	System Response
1	AI agent reviews task timelines and completion data.	System loads relevant project data.
2	AI agent identifies overdue tasks or frequent blockers.	System highlights bottlenecks.
3	AI agent suggests improvements.	System presents recommendations to host.

Alternate Course of Action

S#	Actor Action	System Response
2a	AI finds no bottleneck.	System displays “No issues detected.”

Exceptions:
AI analysis fails due to missing data.

Use Case Description 15

Identifier	UC-15	
Name	Generating Workflow Insights	
Summary	The AI agent analyzes ongoing tasks and projects to generate insights into workflow efficiency.	
Priority	Medium	
Actors	AI Agent	
Pre-condition(s)	Task and project data must exist.	
Post-condition(s)	Workflow insights are displayed to the host.	
Dependencies	Depends on Analyzing History of Tasks and Identifying Productivity Bottlenecks.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	AI agent scans active projects and tasks.	System retrieves current data.
2	AI agent applies workflow analysis algorithms.	System generates insights.
3	AI agent presents insights.	System displays them in dashboard.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
2a	Limited task data.	System shows “insufficient data for insights.”

Exceptions:
AI service error.

Use Case Description 16

Identifier	UC-16
Name	Synchronizing Calendar
Summary	The system synchronizes events and deadlines with Google Calendar.
Priority	High
Actors	Google Calender
Pre-condition(s)	Calendar integration enabled.
Post-condition(s)	Events synced across both systems.
Dependencies	Depends on Managing Events i.e, events must exist to sync.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	System prepares event data.	Google Calendar API receives data.
2	Google Calendar confirms sync.	System updates status to host.

Alternate Course of Action		
S#	**Actor Action**	**System Response**
1a	Duplicate event detected.	System prevents duplicate entry.

Exceptions:
Google Calendar API unavailable.

Use Case Description 17

Identifier	UC-17
Name	Suggesting Intelligent Schedules
Summary	The AI agent suggests optimized schedules for tasks and events based on workload and availability.
Priority	Medium
Actors	AI Agent
Pre-condition(s)	Host must have active tasks/events.
Post-condition(s)	Suggested schedule displayed.
Dependencies	Depends on Synchronizing Calendar and Assigning Deadlines.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	AI agent reviews deadlines and availability.	System retrieves relevant data.
2	AI agent generates intelligent schedule.	System displays recommendations.

Alternate Course of Action		
S#	**Actor Action**	**System Response**
2a	Conflicting deadlines detected.	System highlights conflicts for manual resolution.

Exceptions:
AI scheduling module fails.

Use Case Description 18

Identifier	UC-18
Name	Generating Automatic Reminders
Summary	The AI agent generates reminders for tasks, events, or deadlines.
Priority	High
Actors	AI Agent
Pre-condition(s)	Tasks or events must exist with deadlines.
Post-condition(s)	Reminder notifications sent to host.
Dependencies	Depends on Suggesting Intelligent Schedules and Managing Events.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	AI agent scans upcoming deadlines.	System identifies tasks/events.
2	AI agent generates reminder.	System schedules reminder notification.

Alternate Course of Action		
S#	**Actor Action**	**System Response**
2a	No upcoming deadlines.	System displays “no reminders.”

Exceptions:
Notification service fails.

Use Case Description 19

Identifier	UC-19
Name	Generating Progress Summaries
Summary	The AI agent generates summaries of project progress for host review.
Priority	Medium
Actors	AI Agent
Pre-condition(s)	Project or task data must exist.
Post-condition(s)	Progress summary displayed in dashboard.
Dependencies	Depends on Tracking Project Progress and Analyzing History of Tasks.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	AI agent reviews task completion data.	System aggregates data.
2	AI agent generates summary.	System displays progress summary.

Alternate Course of Action		
S#	**Actor Action**	**System Response**
1a	No completed tasks.	System shows "progress summary unavailable."

Exceptions:
AI service failure.

Use Case Description 20

Identifier	UC-20
Name	Importing Contacts
Summary	The system imports user contacts from Google Contacts for collaboration and communication.
Priority	Medium
Actors	Google Contacts
Pre-condition(s)	Google Contacts integration enabled.
Post-condition(s)	Contacts successfully imported.
Dependencies	No direct dependency except system login; may indirectly support Managing Collaborator Invitations.

Typical Course of Action

S#	Actor Action	System Response
1	System requests contacts.	Google Contacts API provides list.
2	System imports contacts.	System updates user's contact directory.

Alternate Course of Action

S#	Actor Action	System Response
2a	Duplicate contact found.	System merges or skips.

Exceptions:
Google Contacts API unavailable.

Use Case Description 21

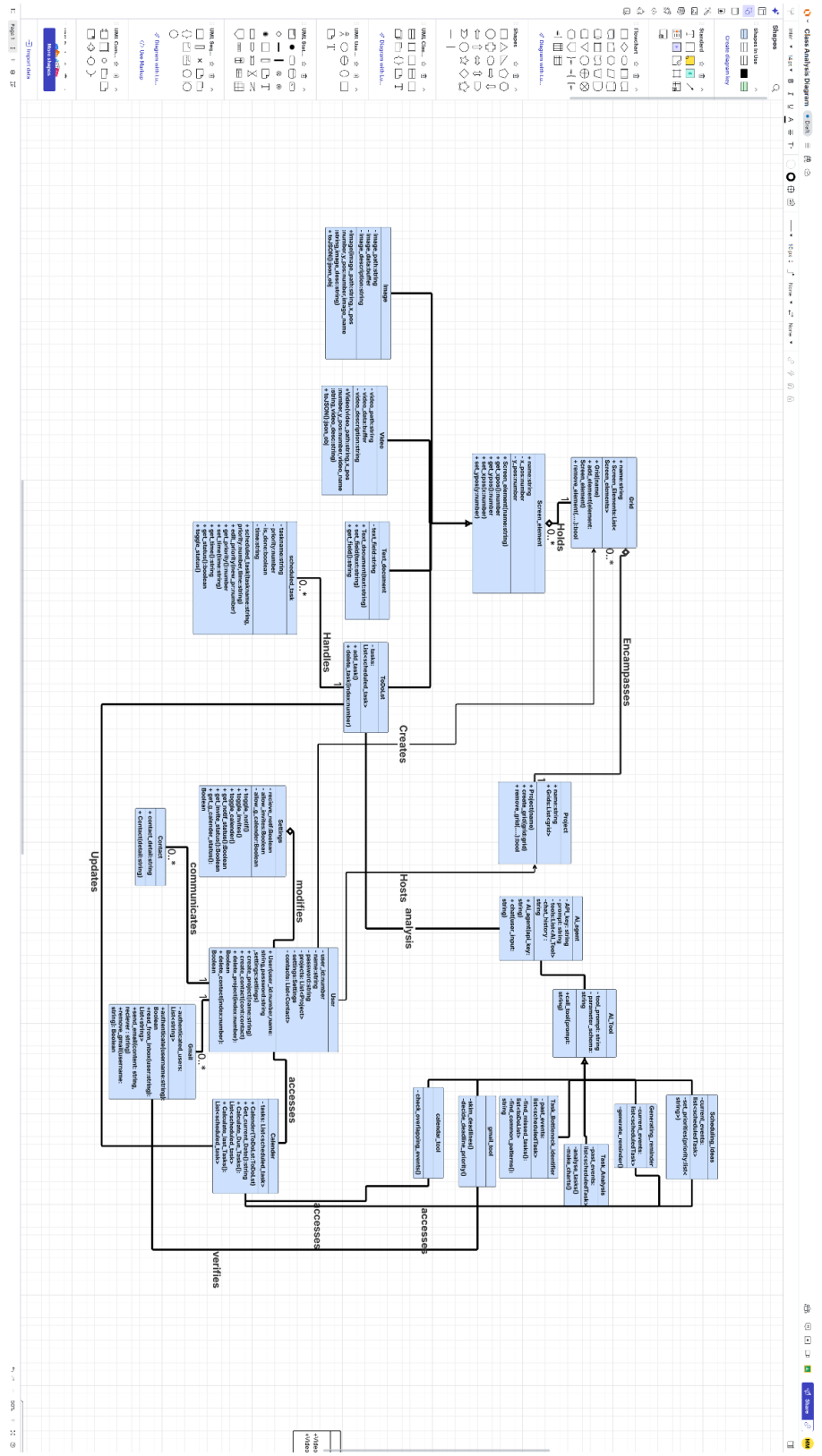
Identifier	UC-21
Name	Sending Notifications
Summary	The system sends notifications (e.g., reminders, updates) to users using Gmail and Google Calendar.
Priority	High
Actors	Google Calendar Gmail
Pre-condition(s)	Notifications must be scheduled.
Post-condition(s)	User receives notification successfully.
Dependencies	Depends on Managing Events, Sharing Projects, and external services Google Calendar and Gmail.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	System generates notification event.	Google Calendar/Gmail API processes request.
2	Notification sent to user.	System confirms delivery status.

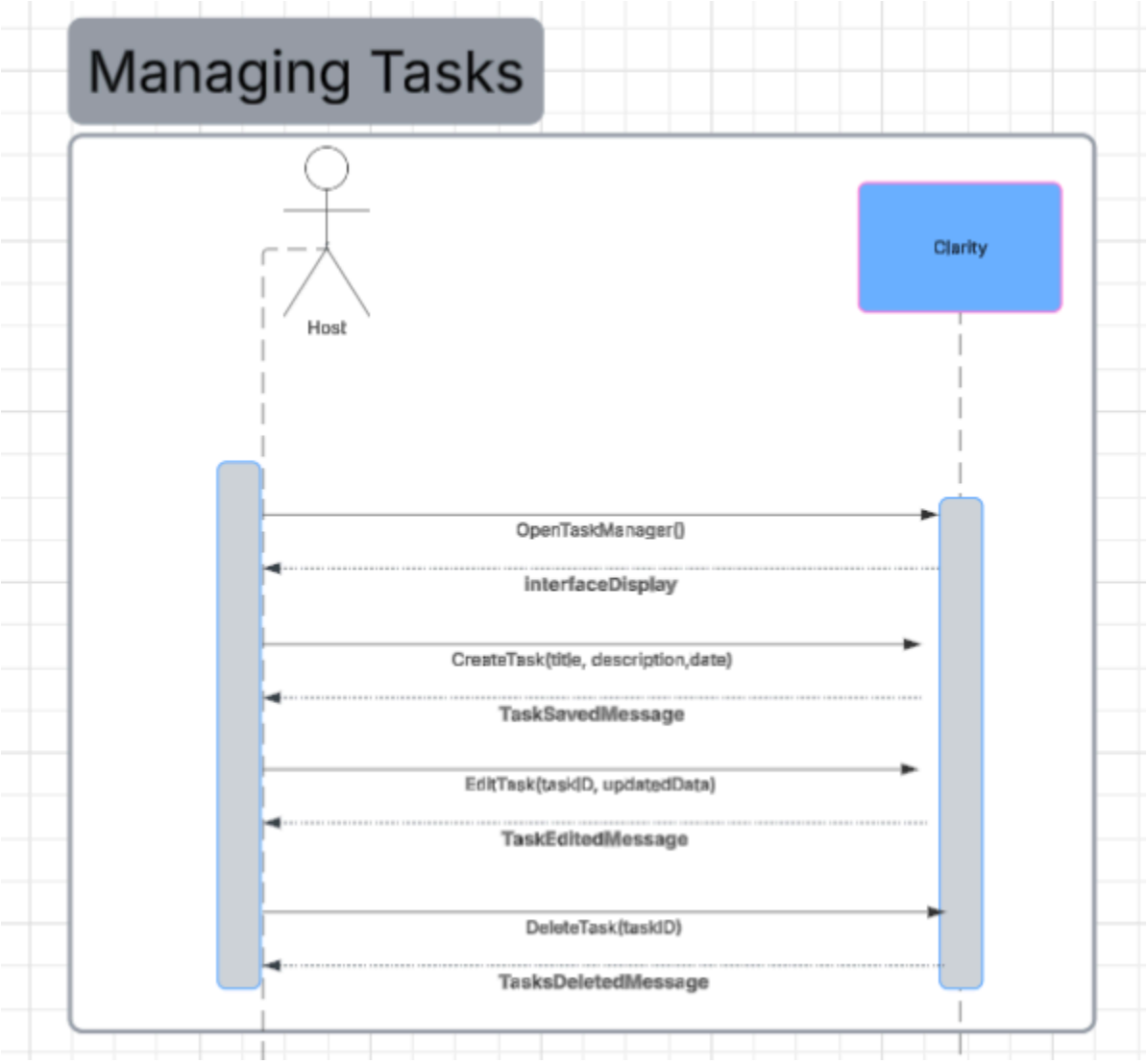
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1a	Invalid recipient.	System shows error.

Exceptions:
Gmail API failure.
Google Calendar API failure.

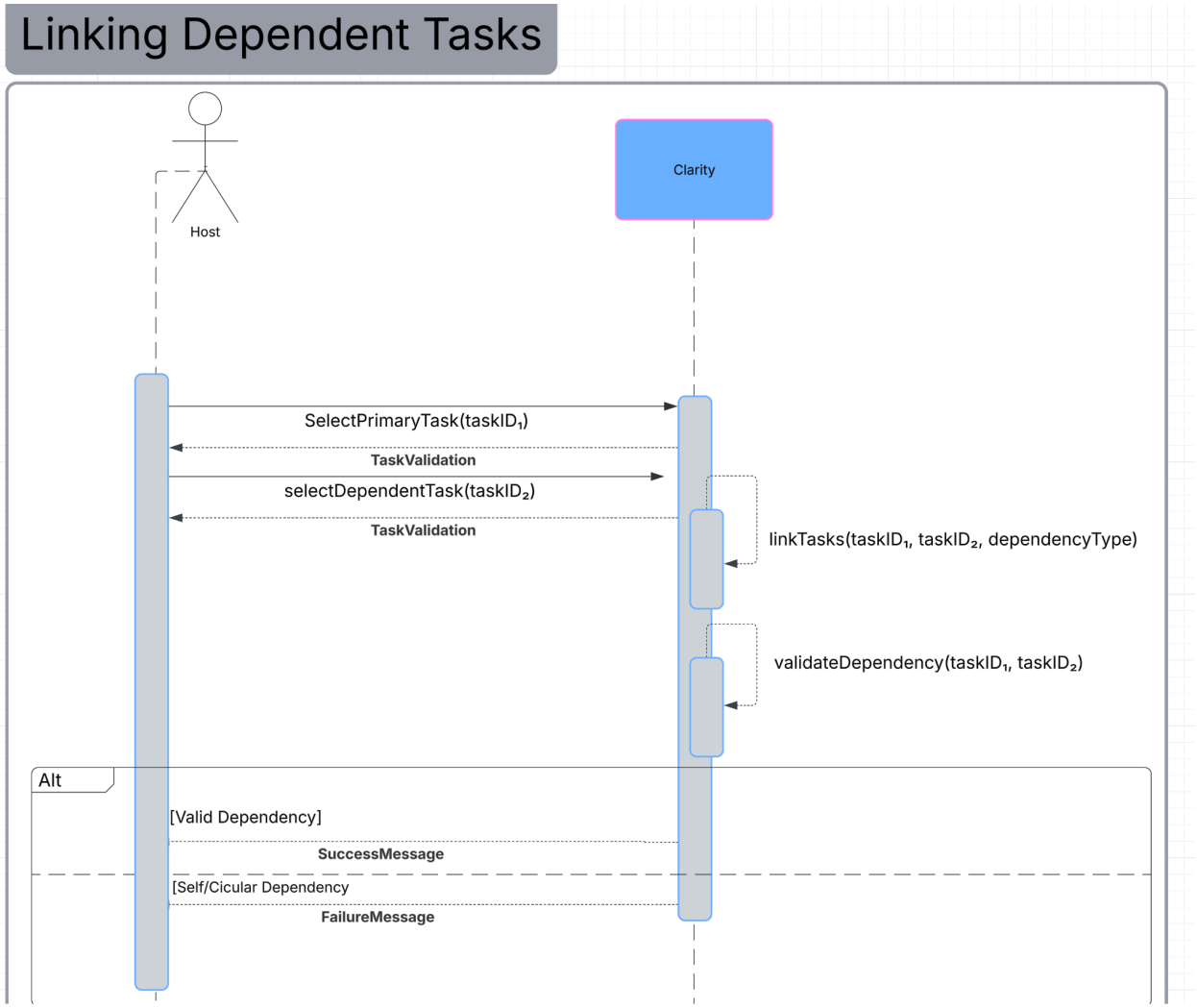
Design Class Diagram



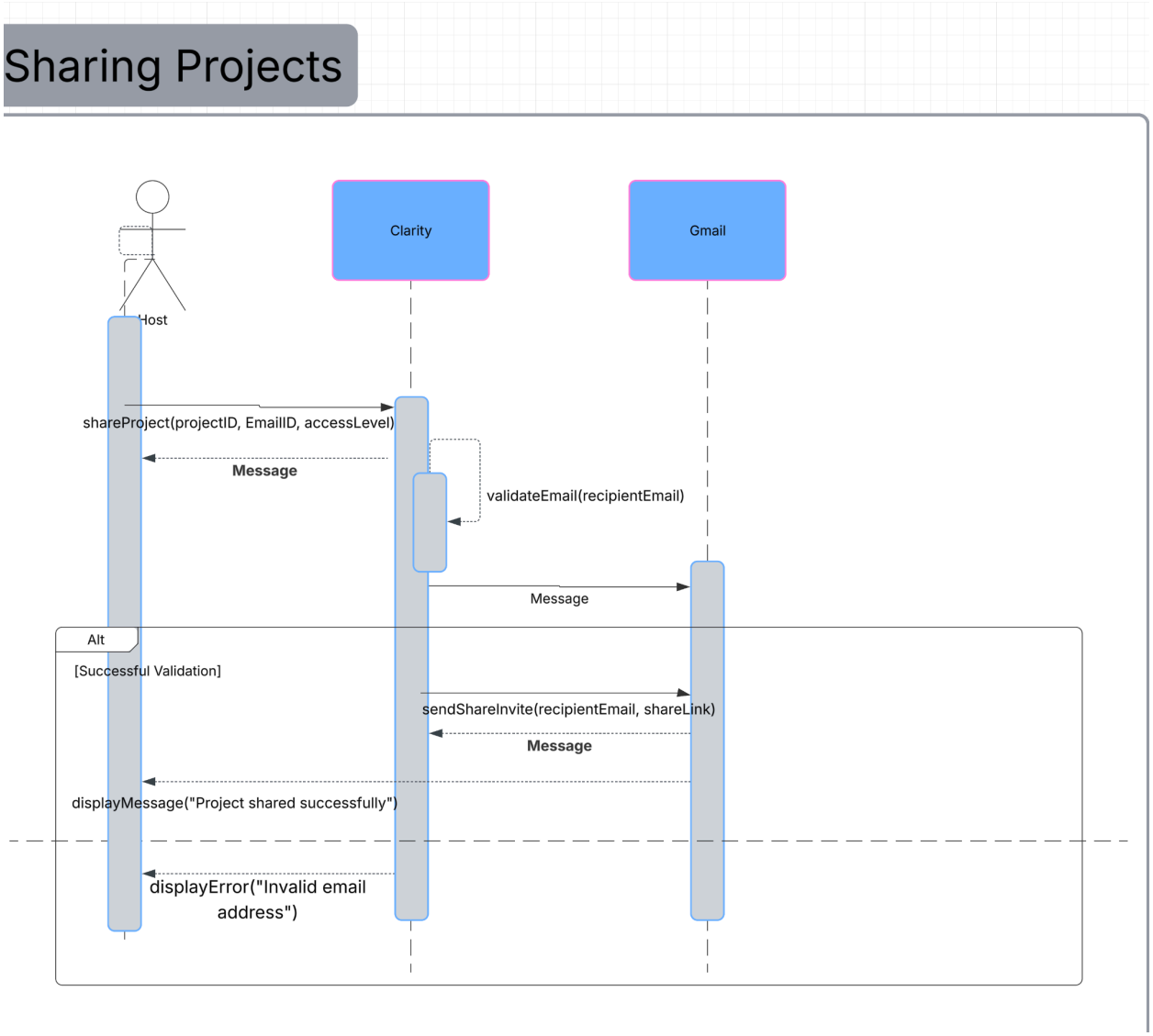
Sequence diagram 1



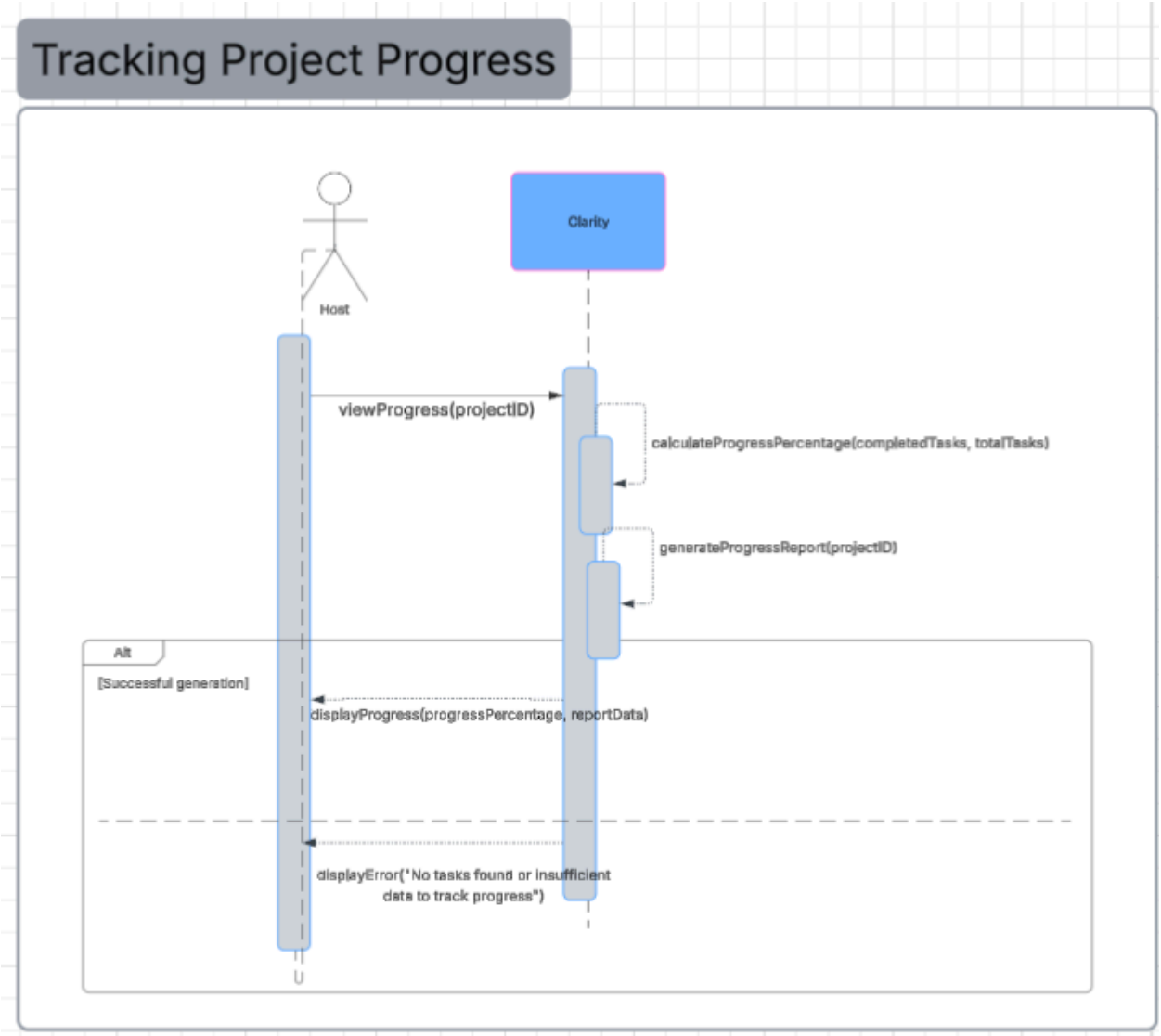
Sequence diagram 2



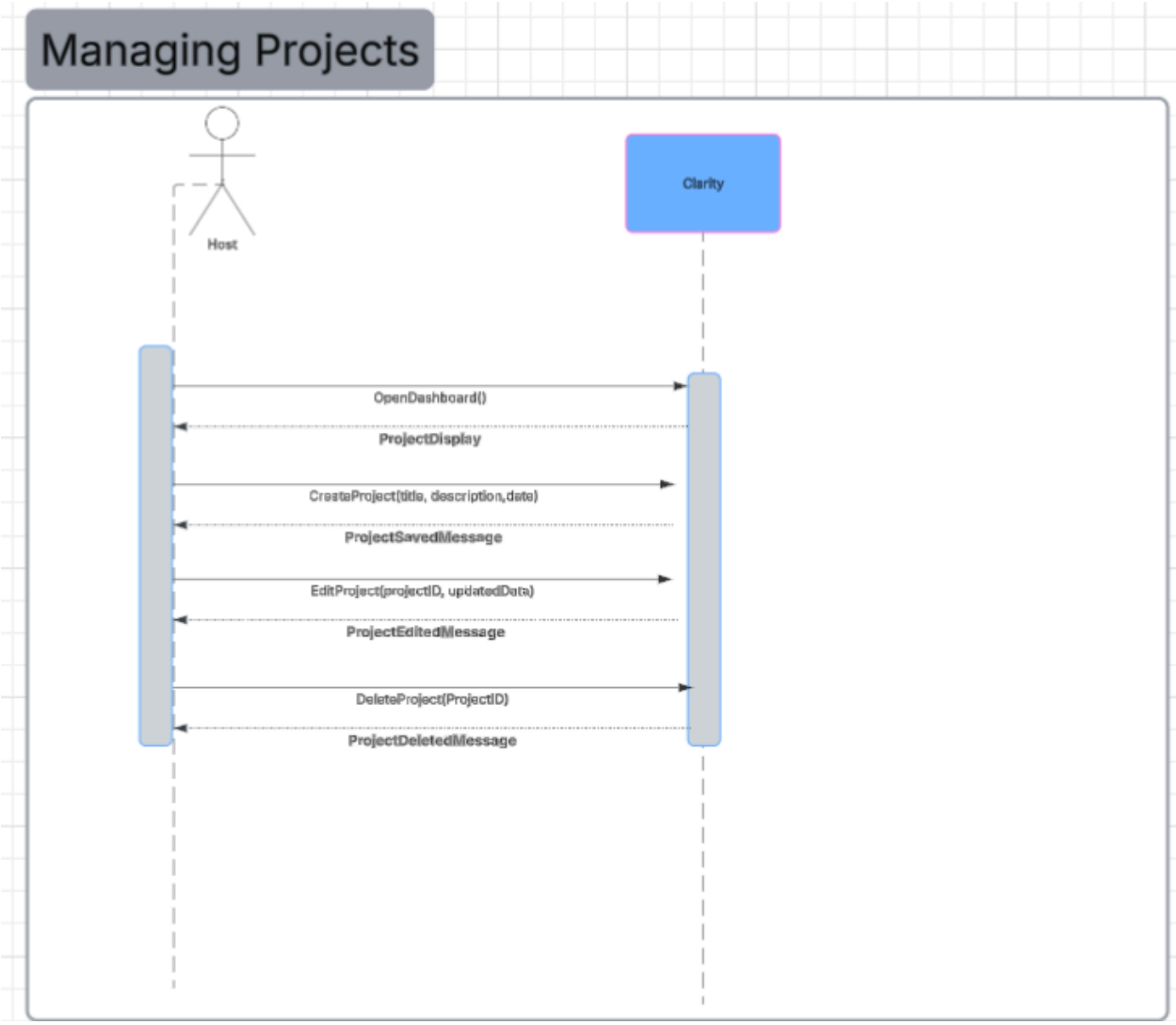
Sequence diagram 3



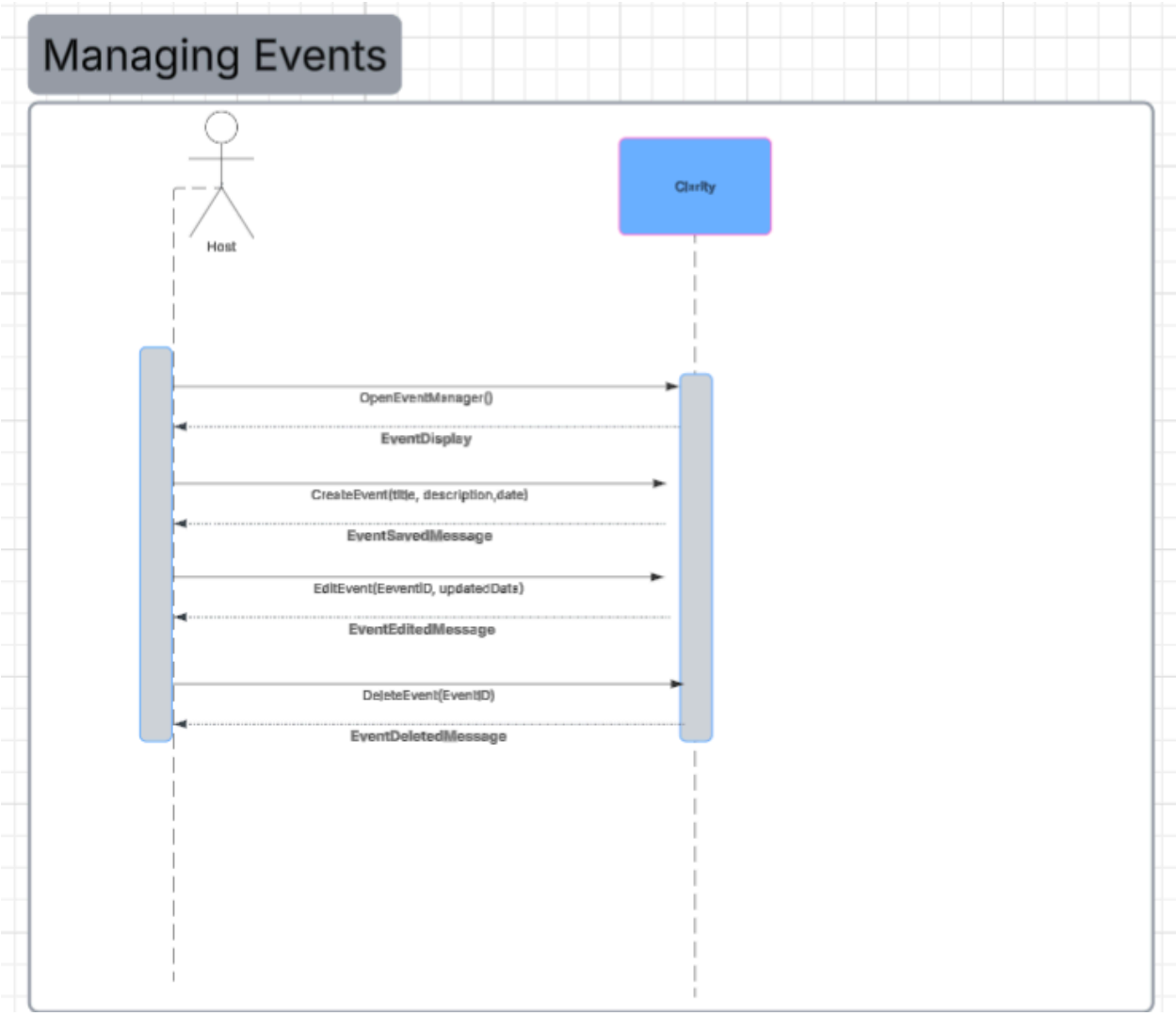
Sequence diagram 4



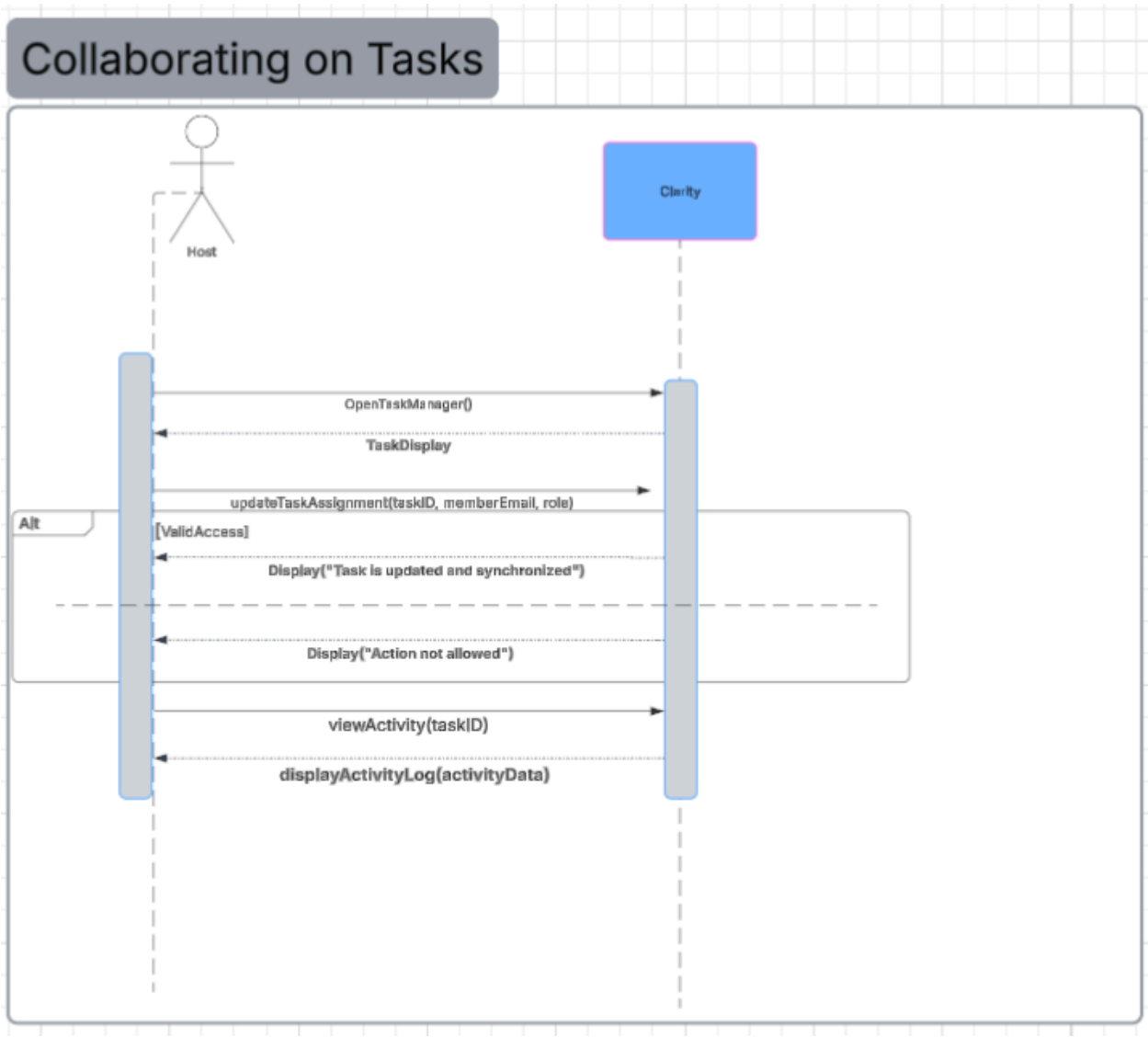
Sequence diagram 5



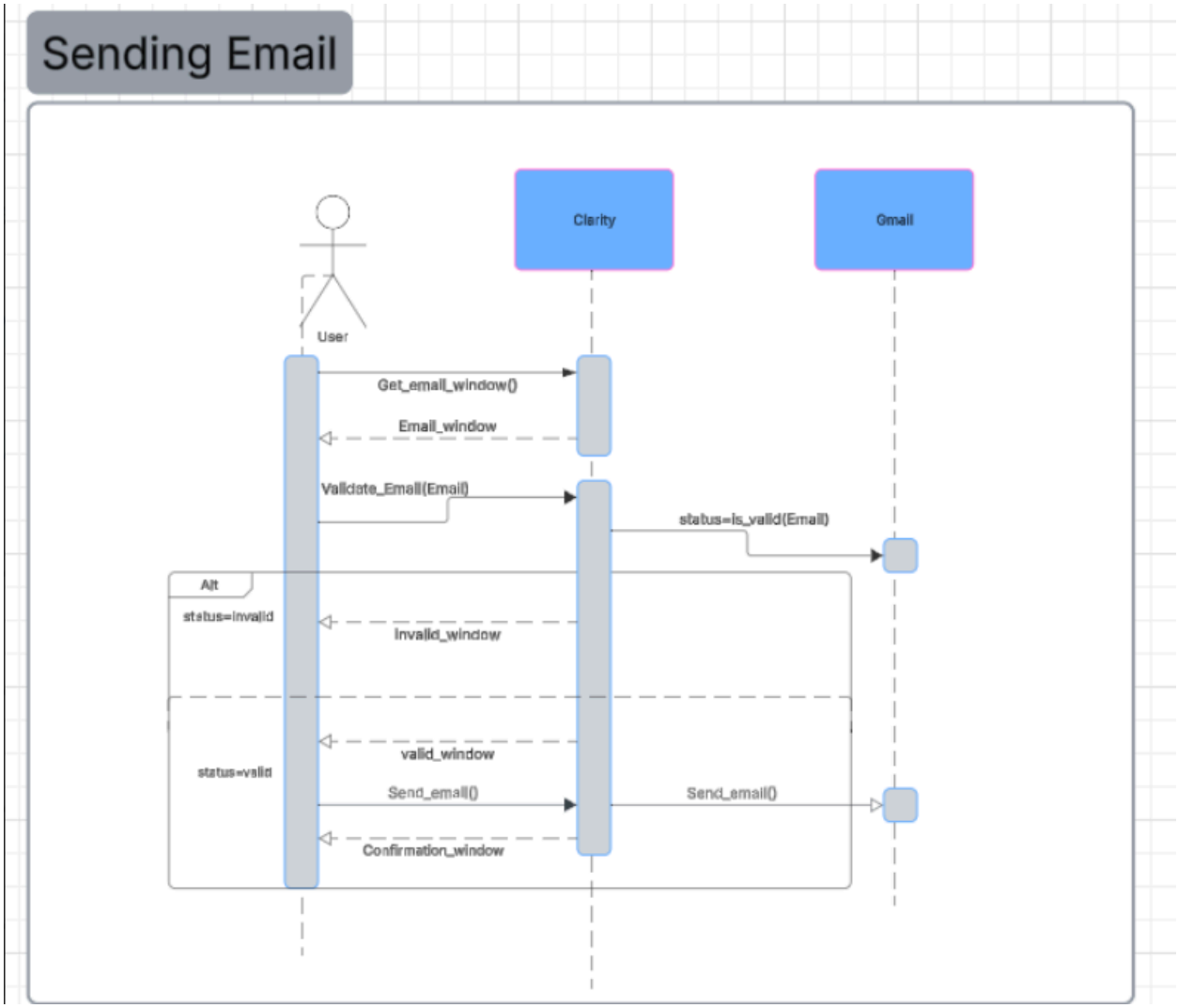
Sequence diagram 6



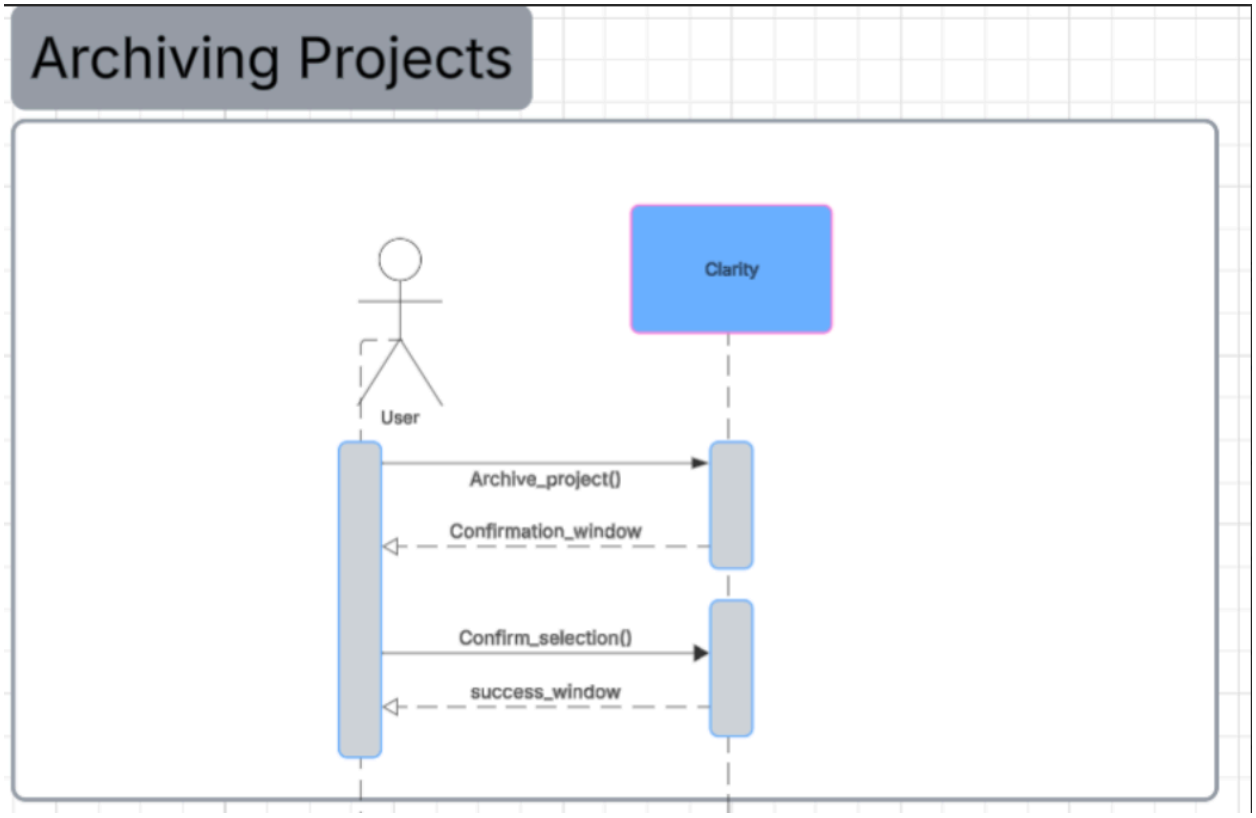
Sequence diagram 7



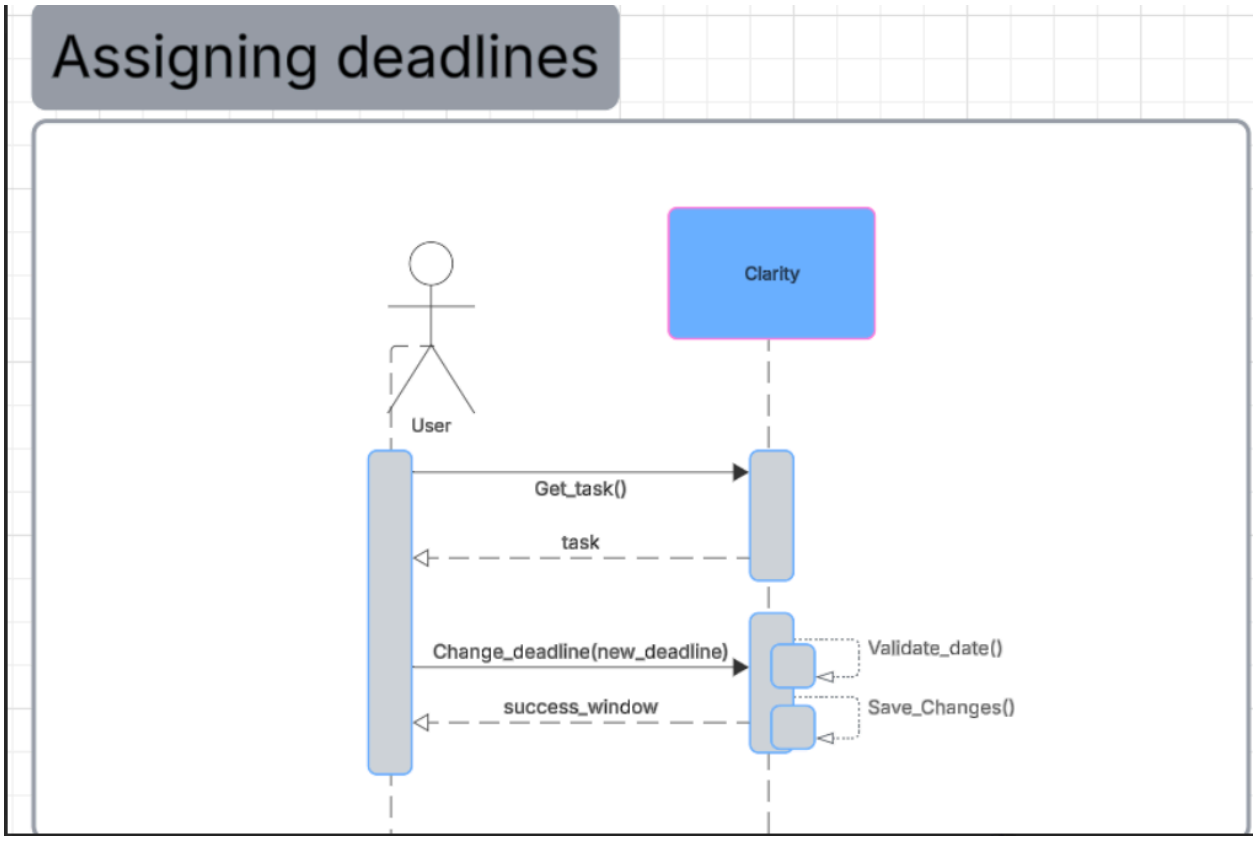
Sequence diagram 8



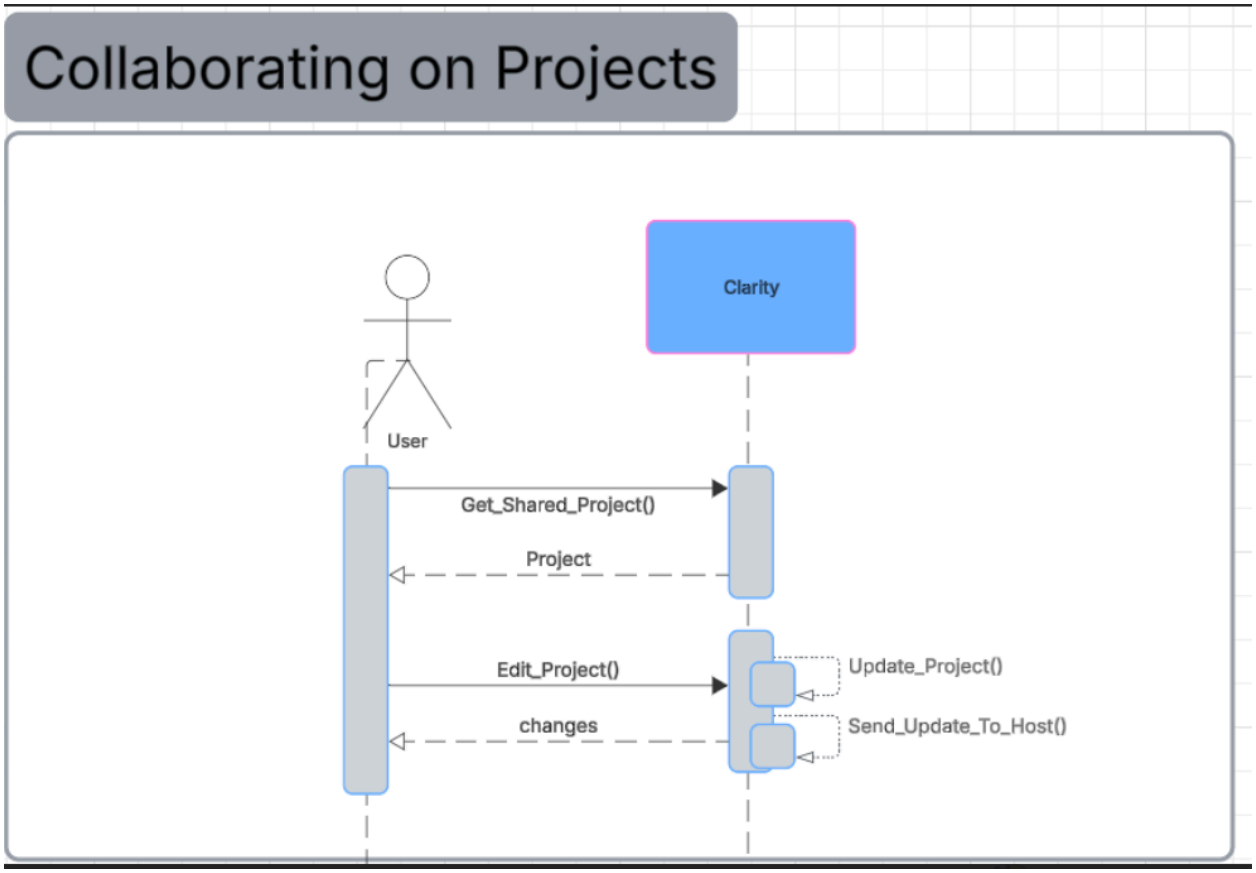
Sequence diagram 9



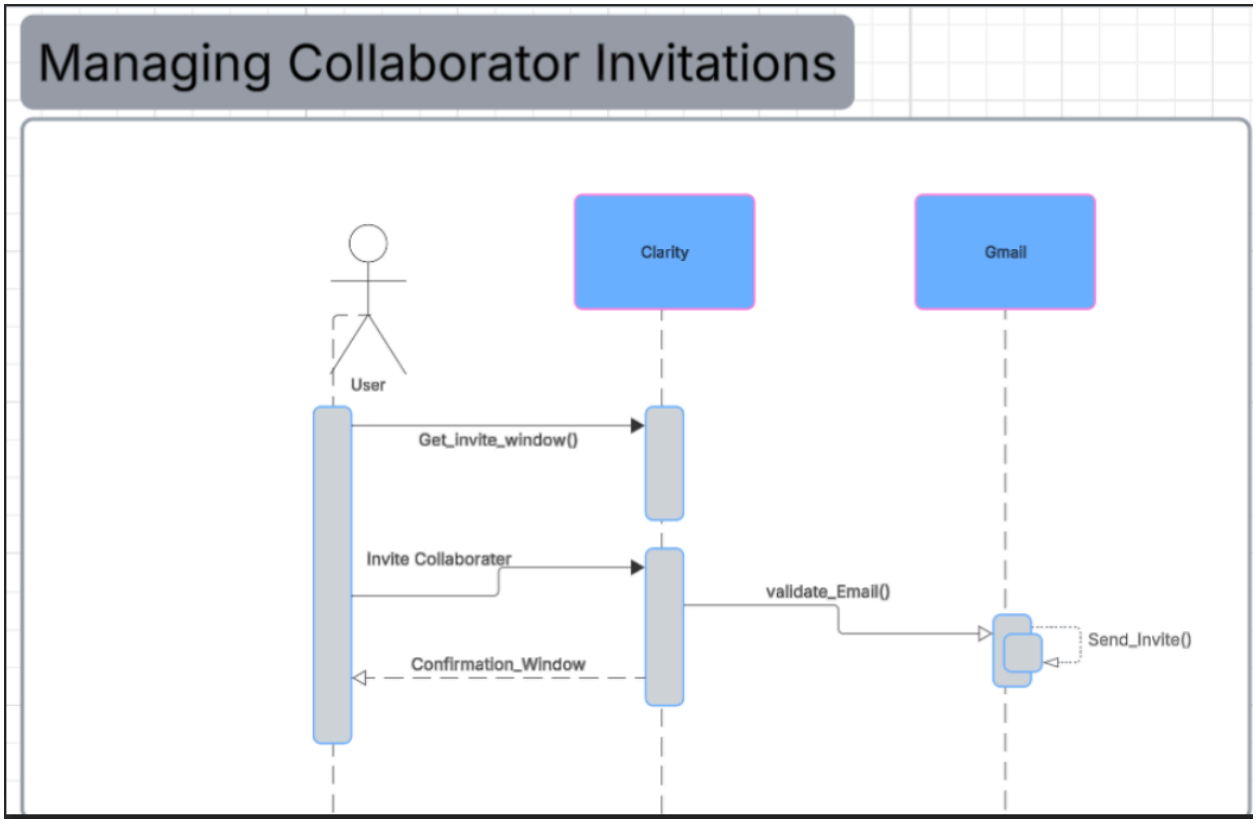
Sequence diagram 10



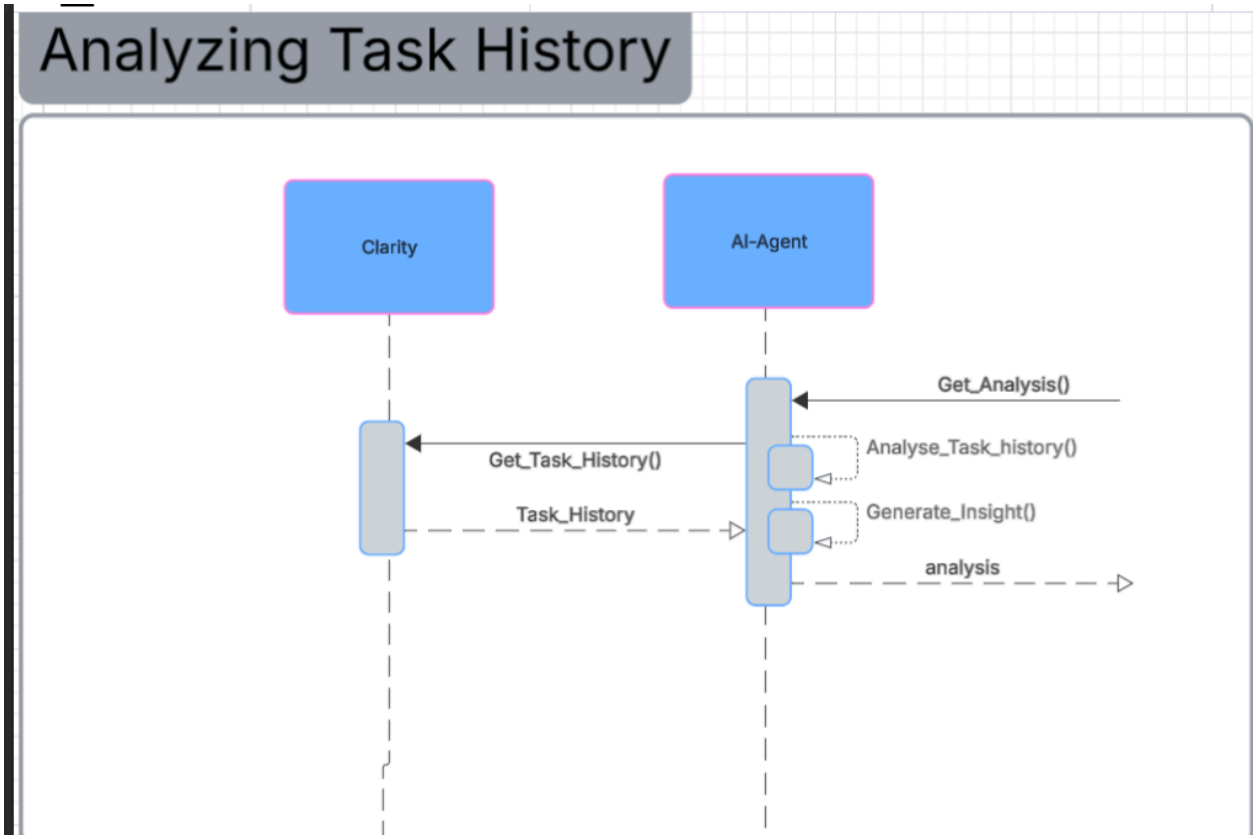
Sequence diagram 11



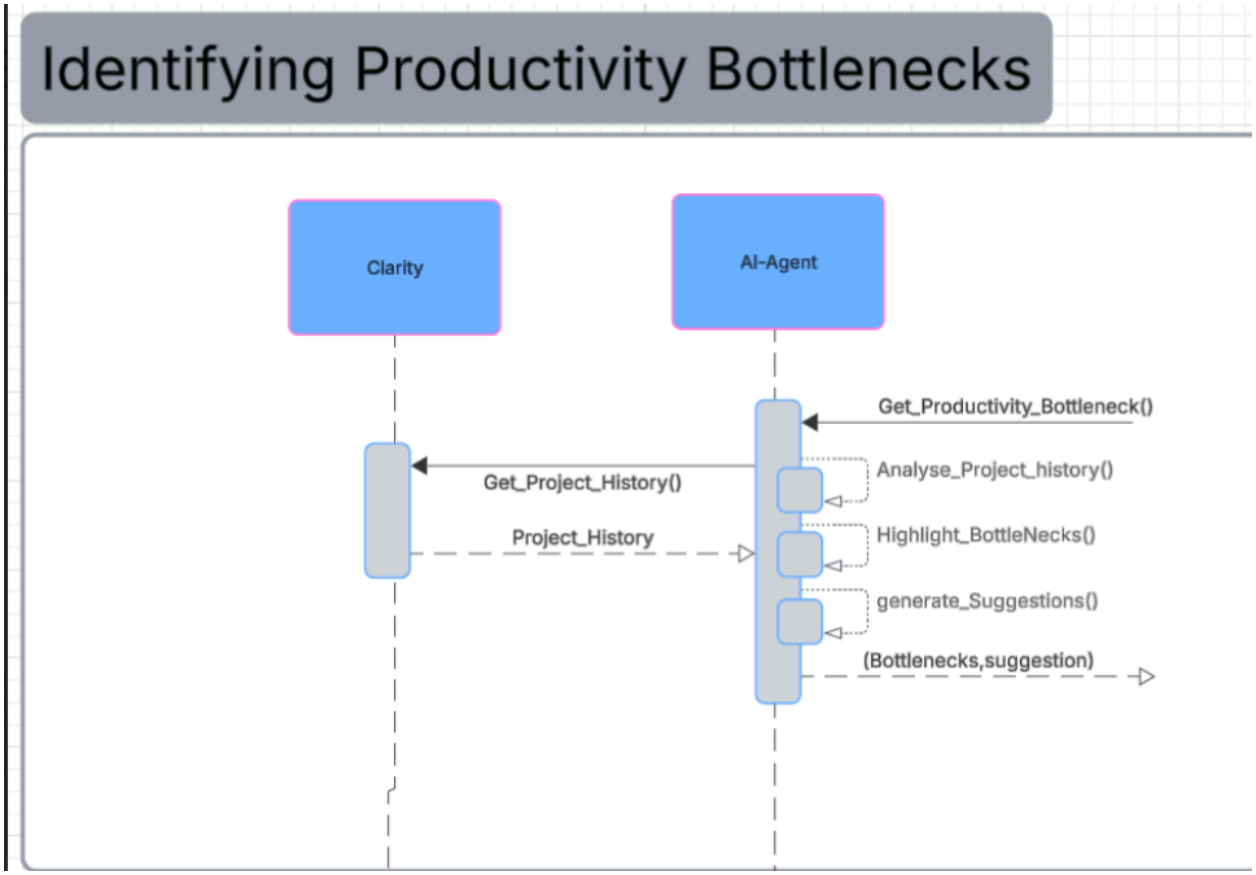
Sequence diagram 12



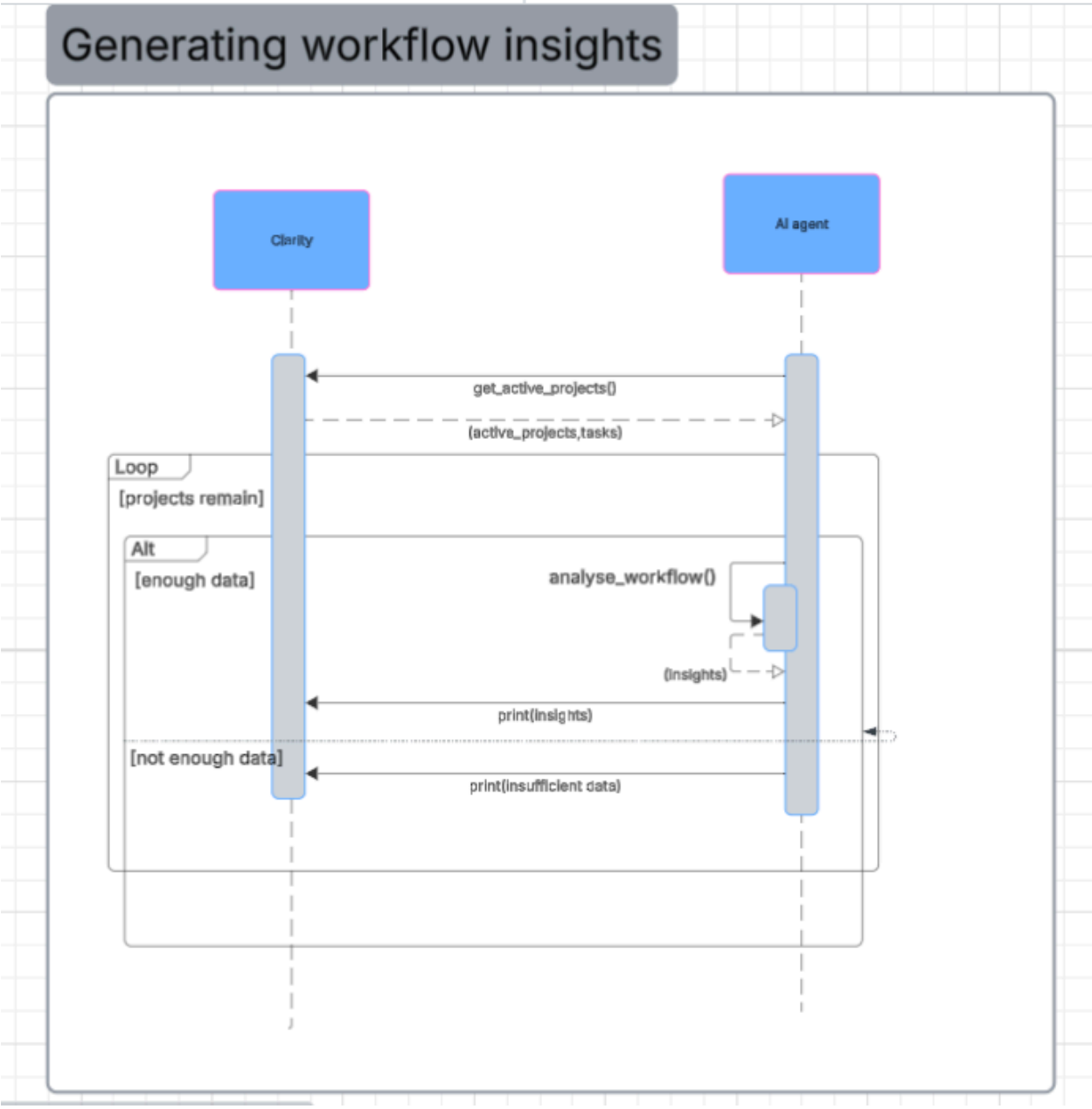
Sequence diagram 13



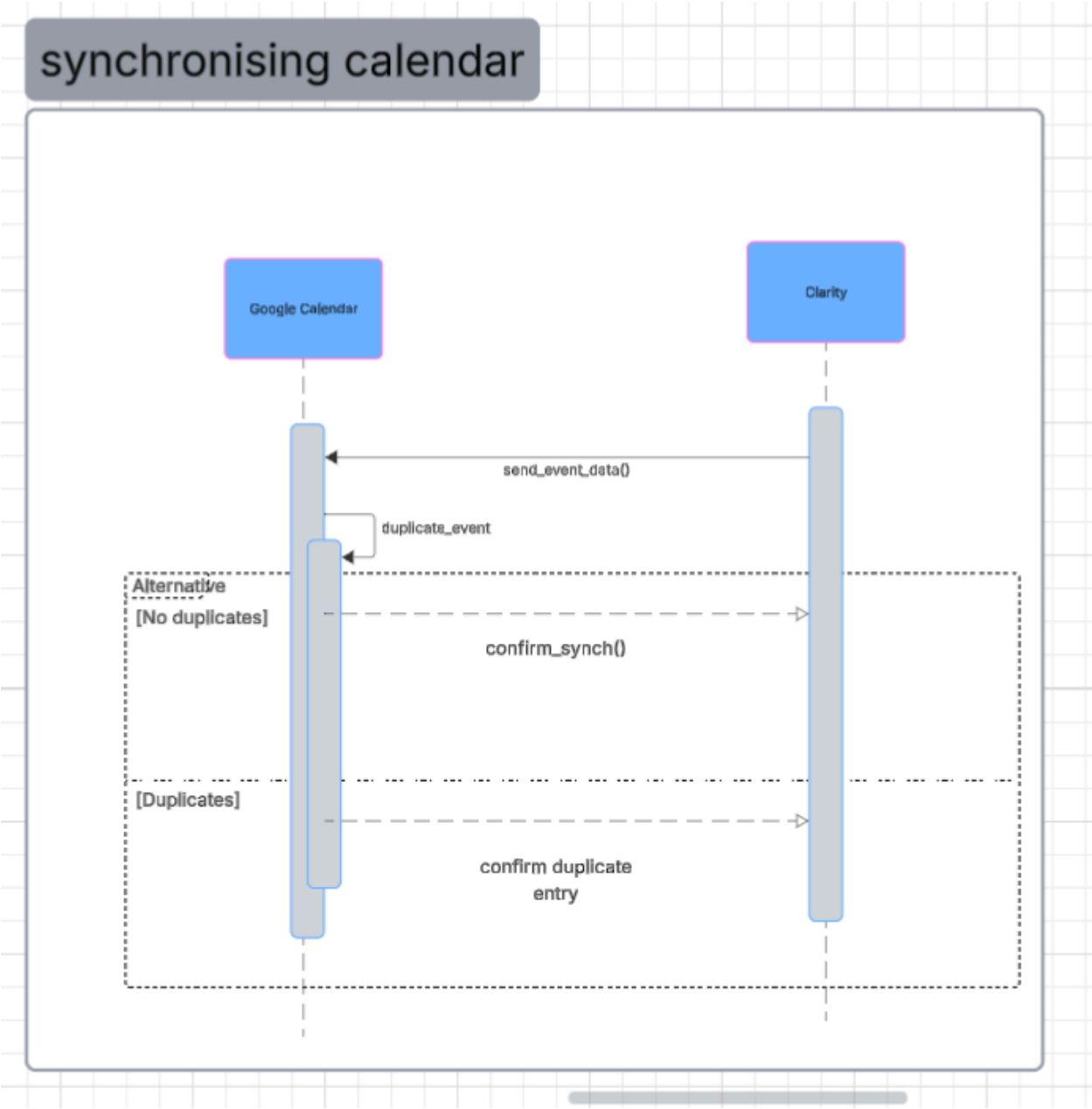
Sequence diagram 14



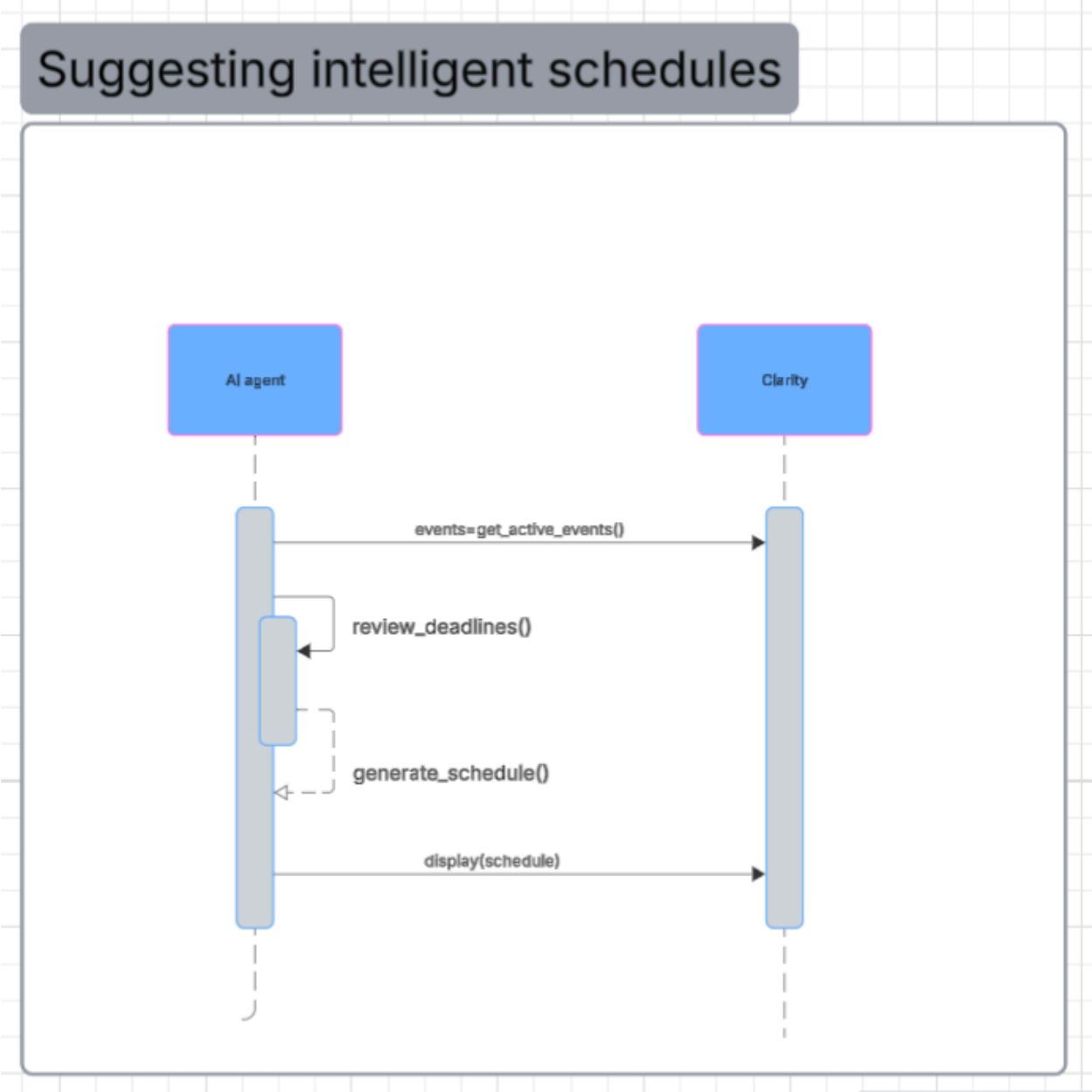
Sequence diagram 15



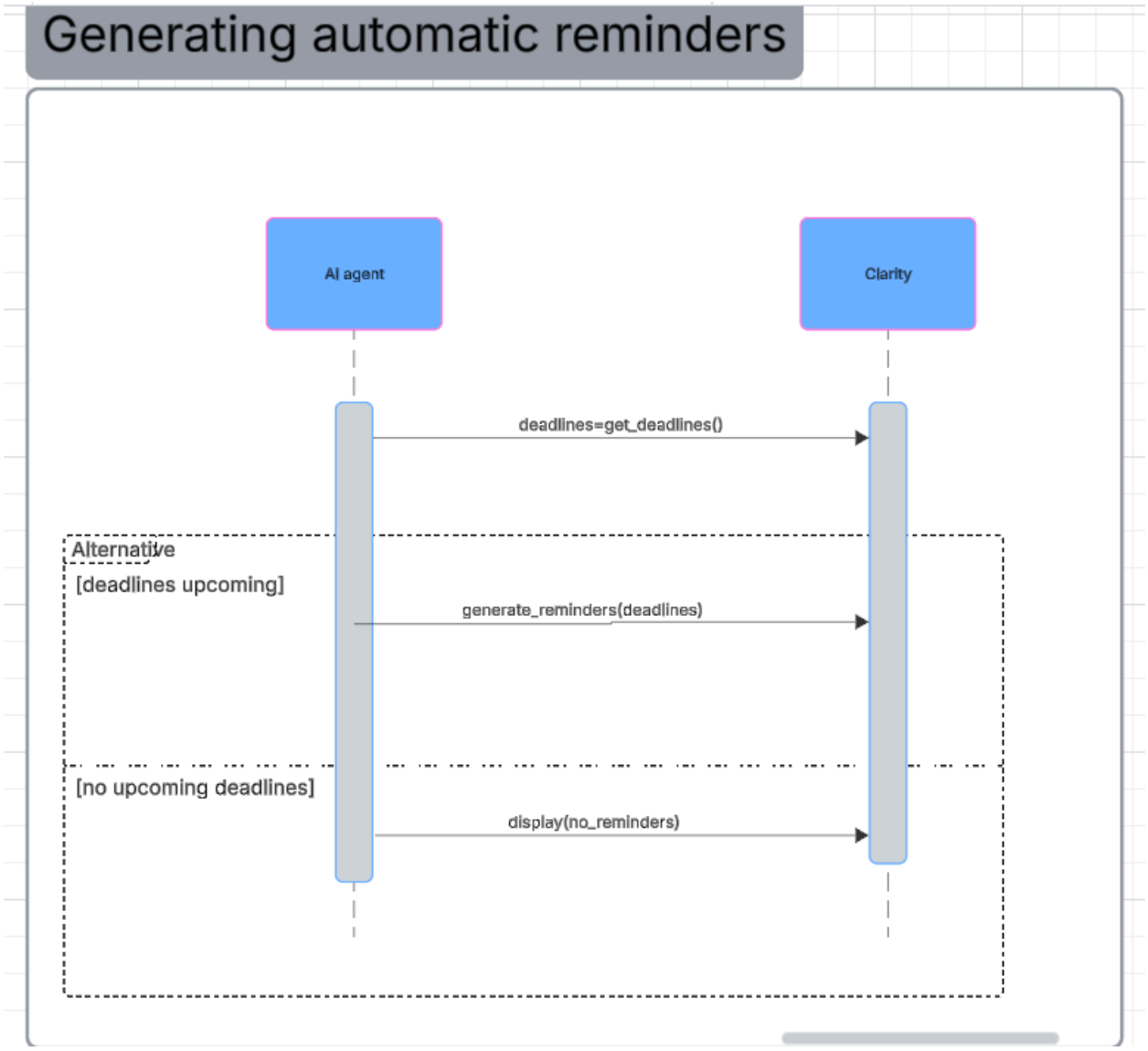
Sequence diagram 16



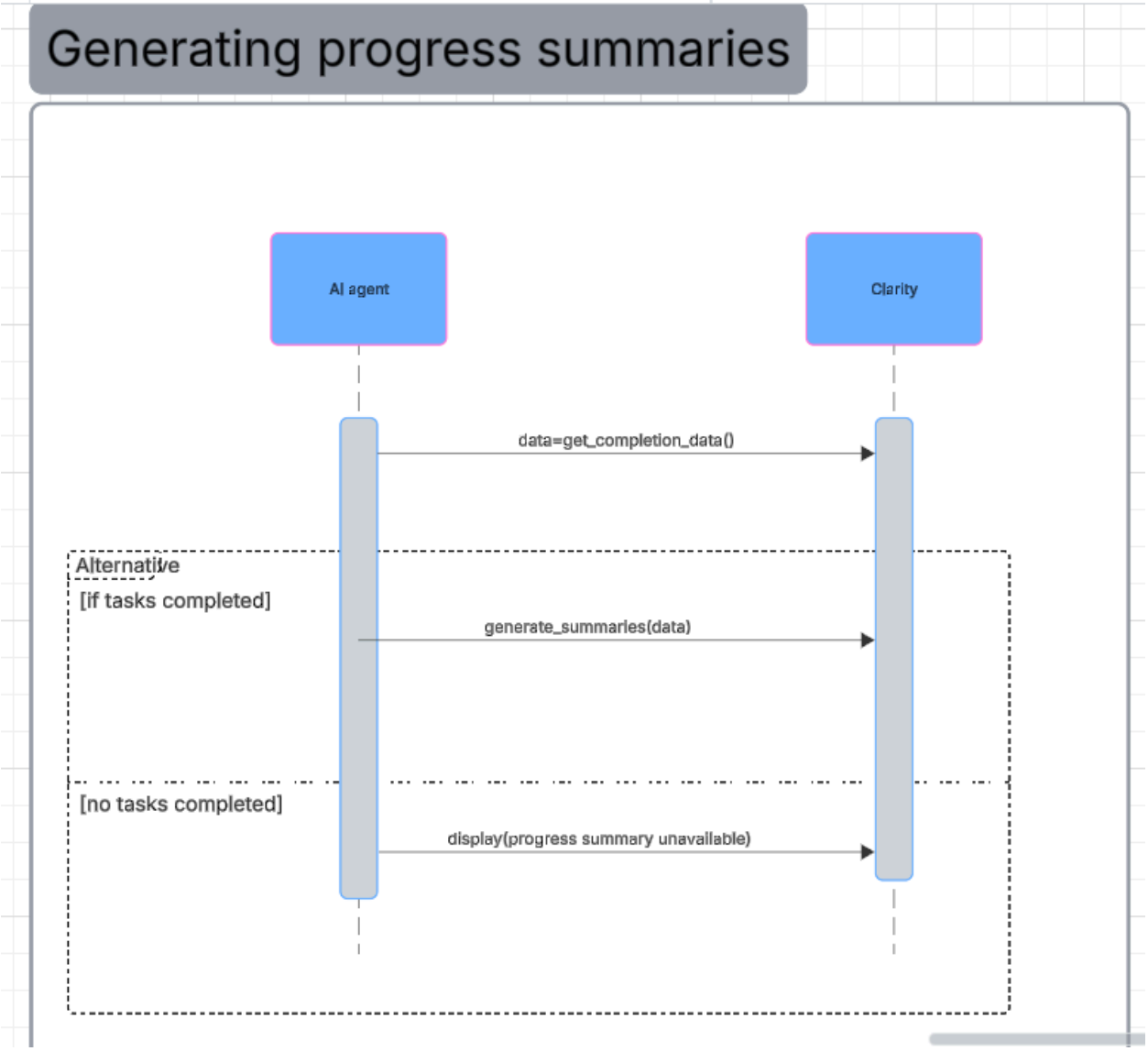
Sequence diagram 17



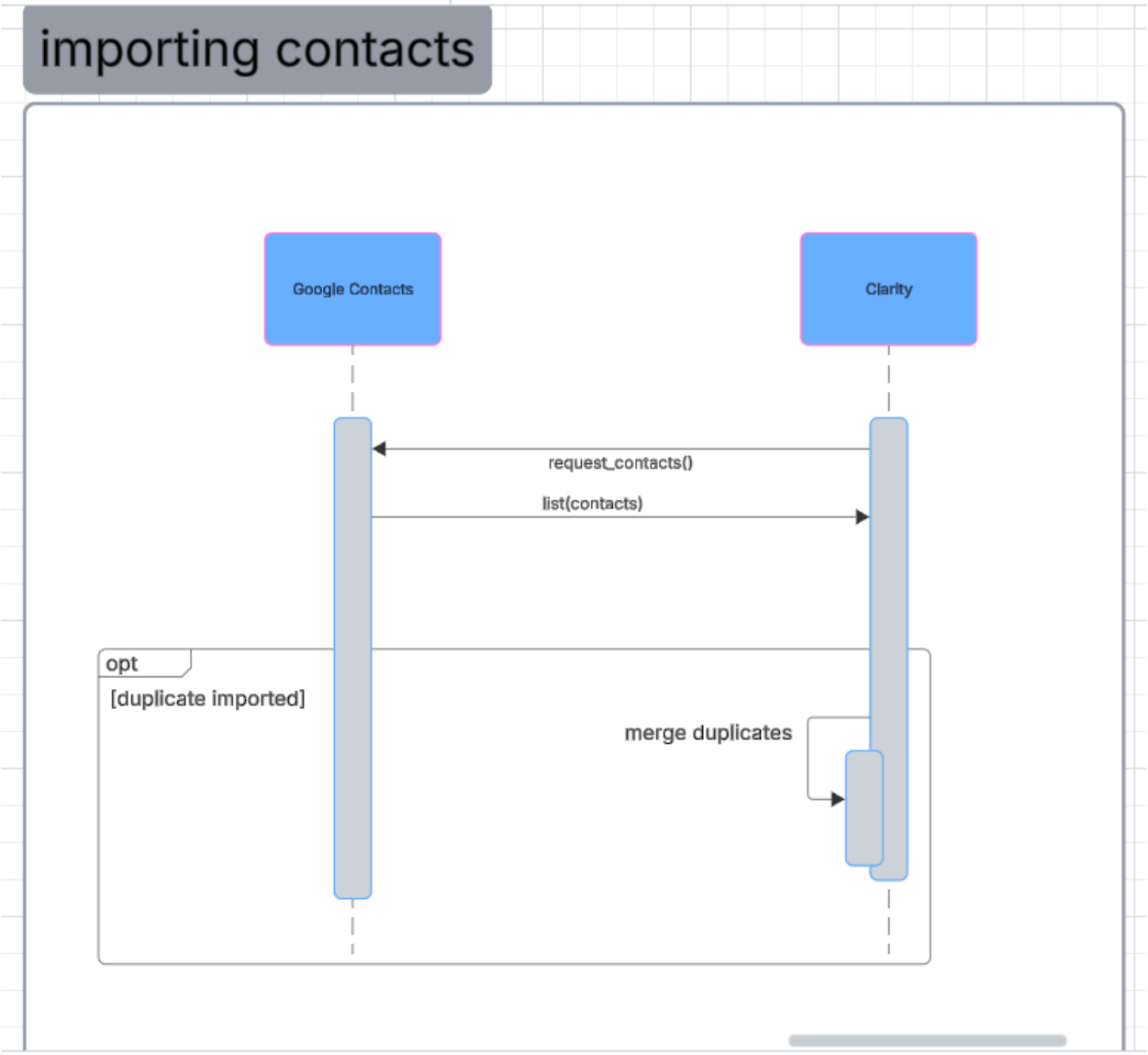
Sequence diagram 18



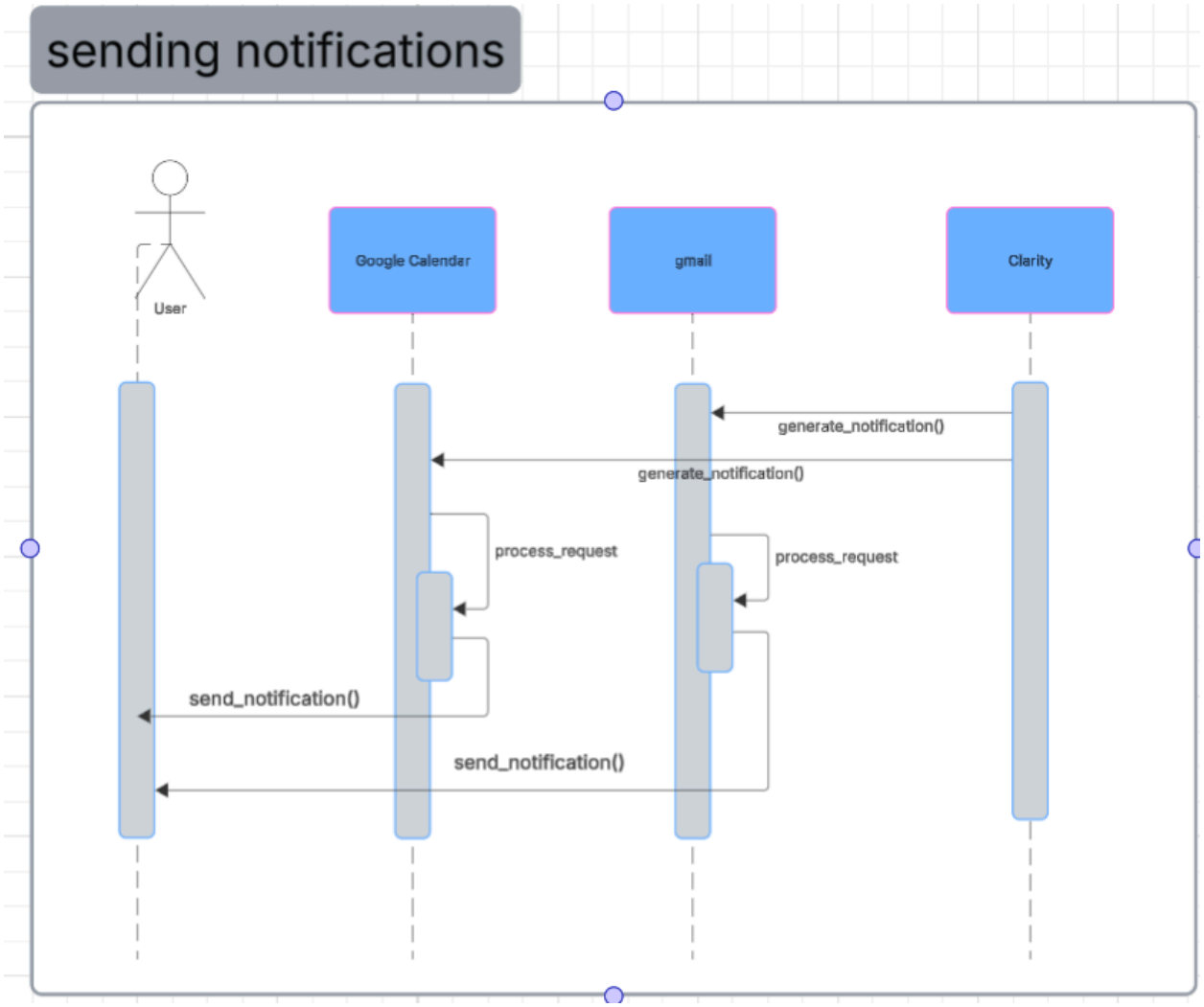
Sequence diagram 19



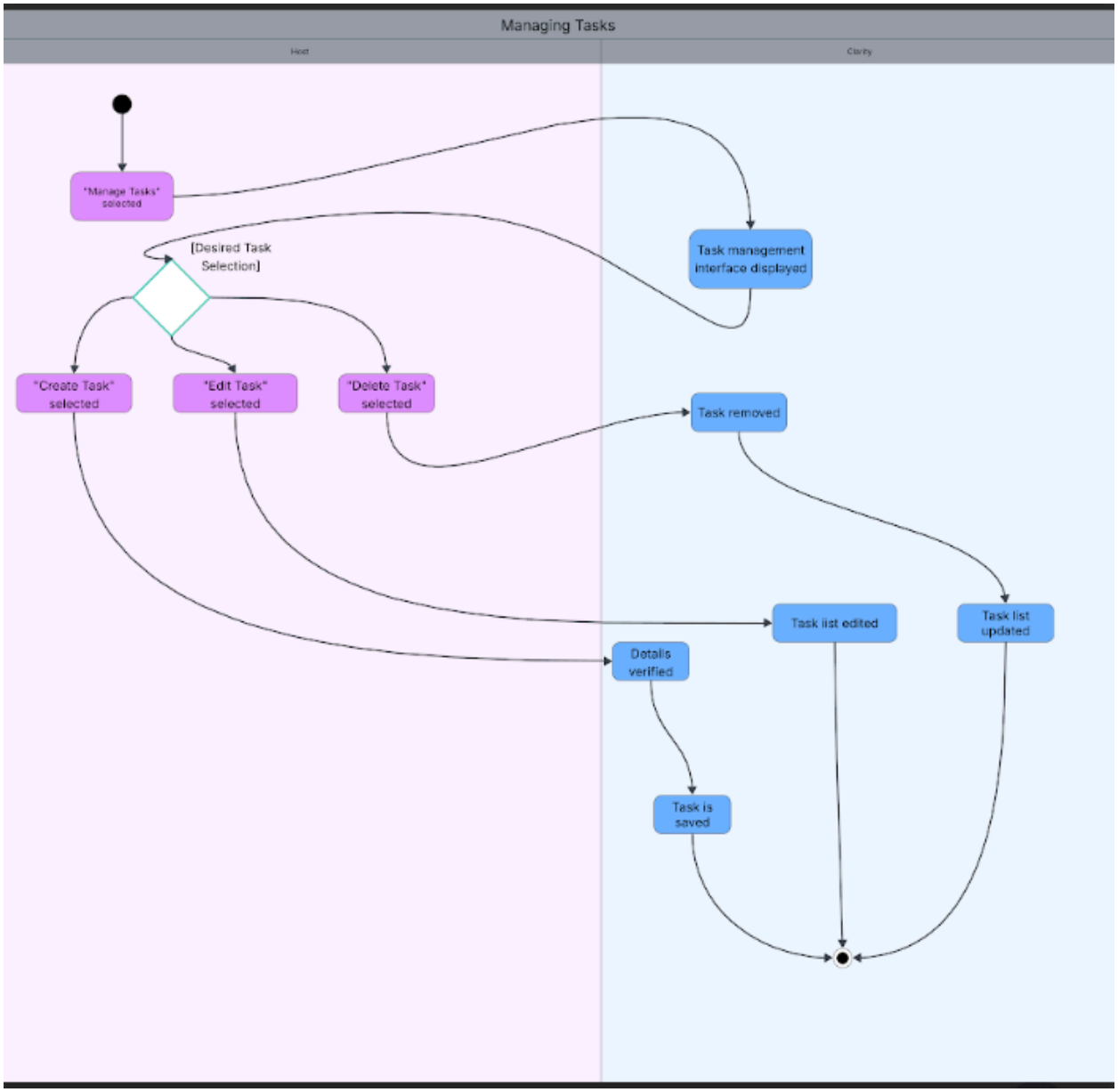
Sequence diagram 20



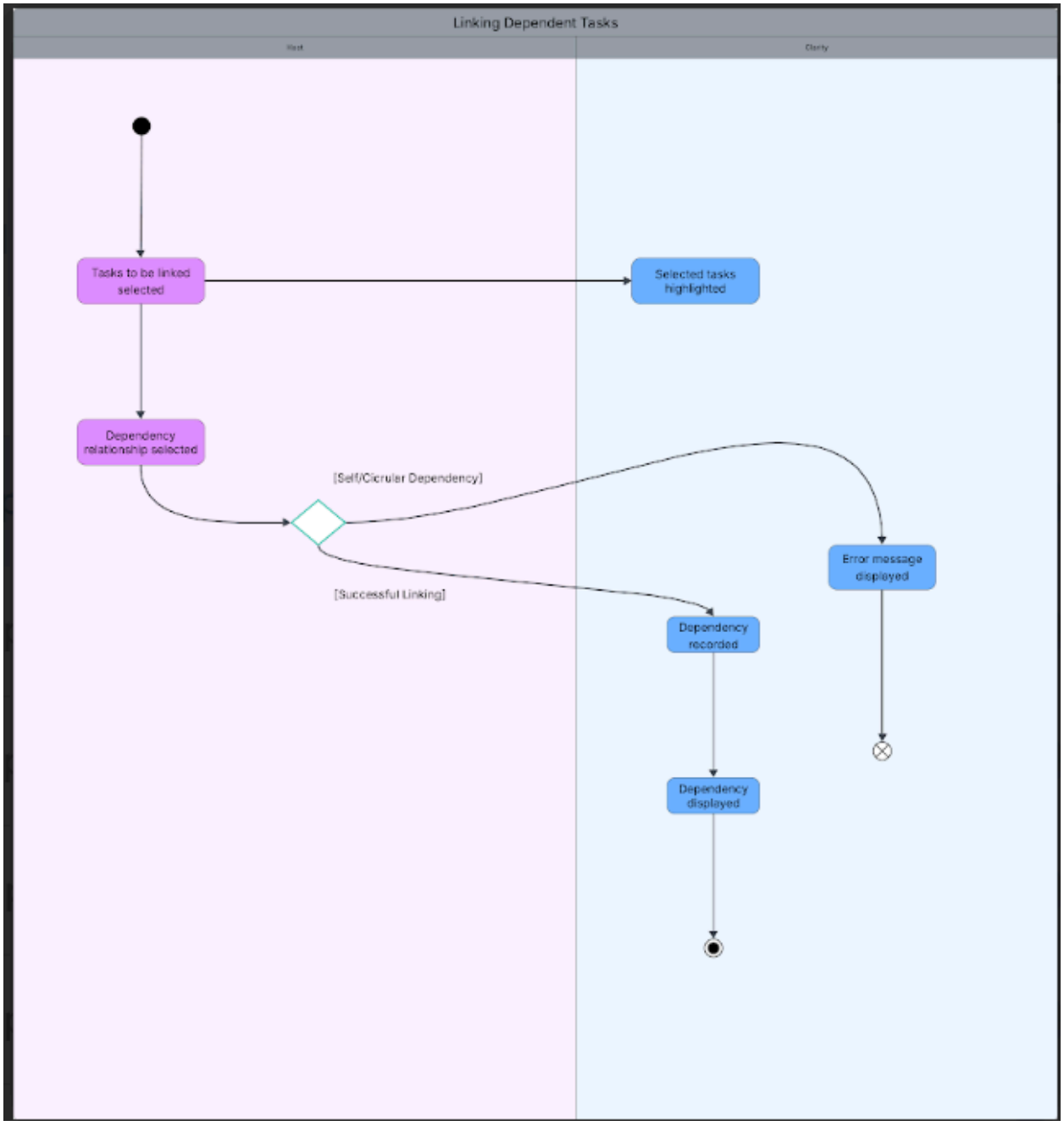
Sequence diagram 21



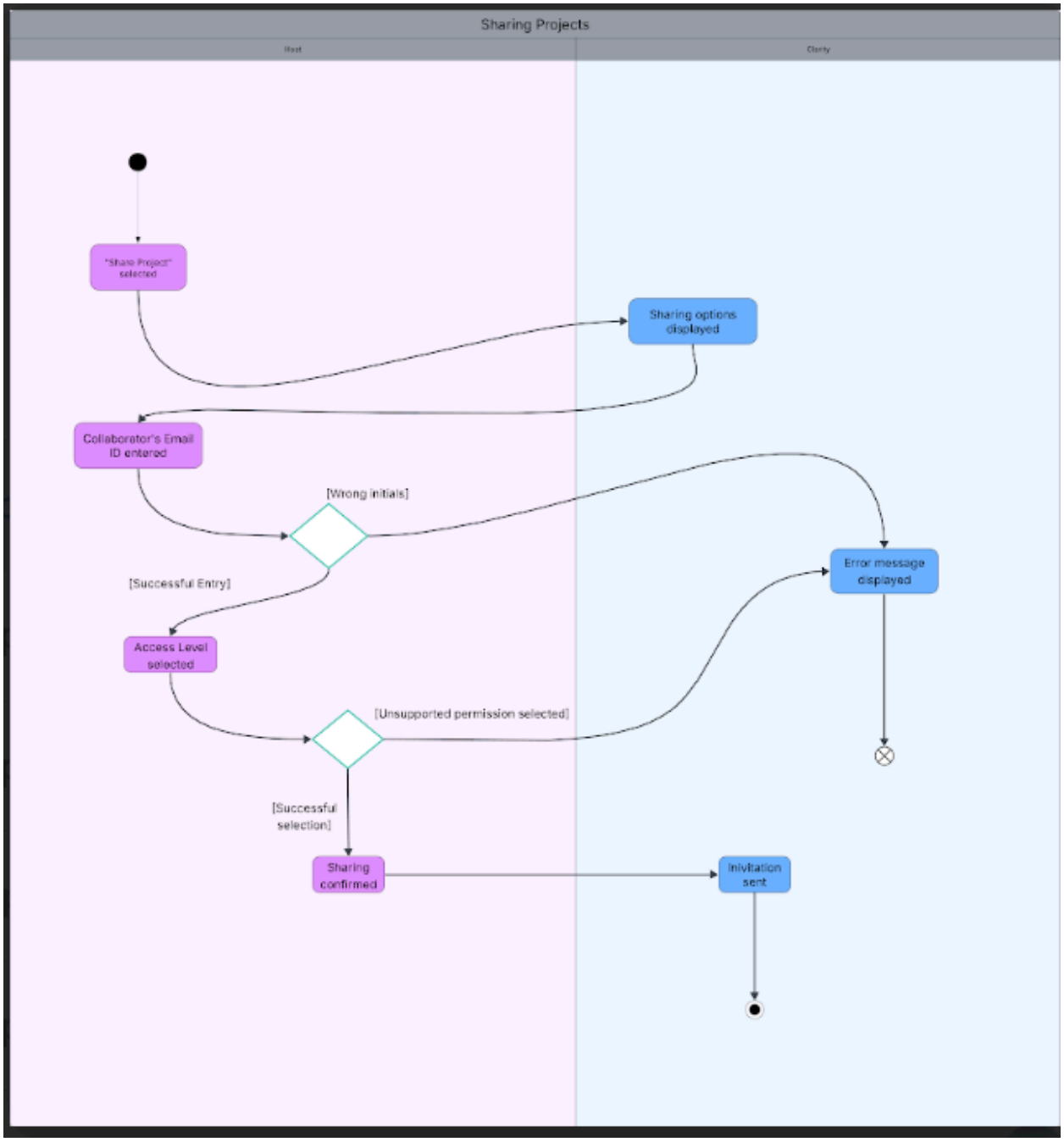
Activity diagram 1



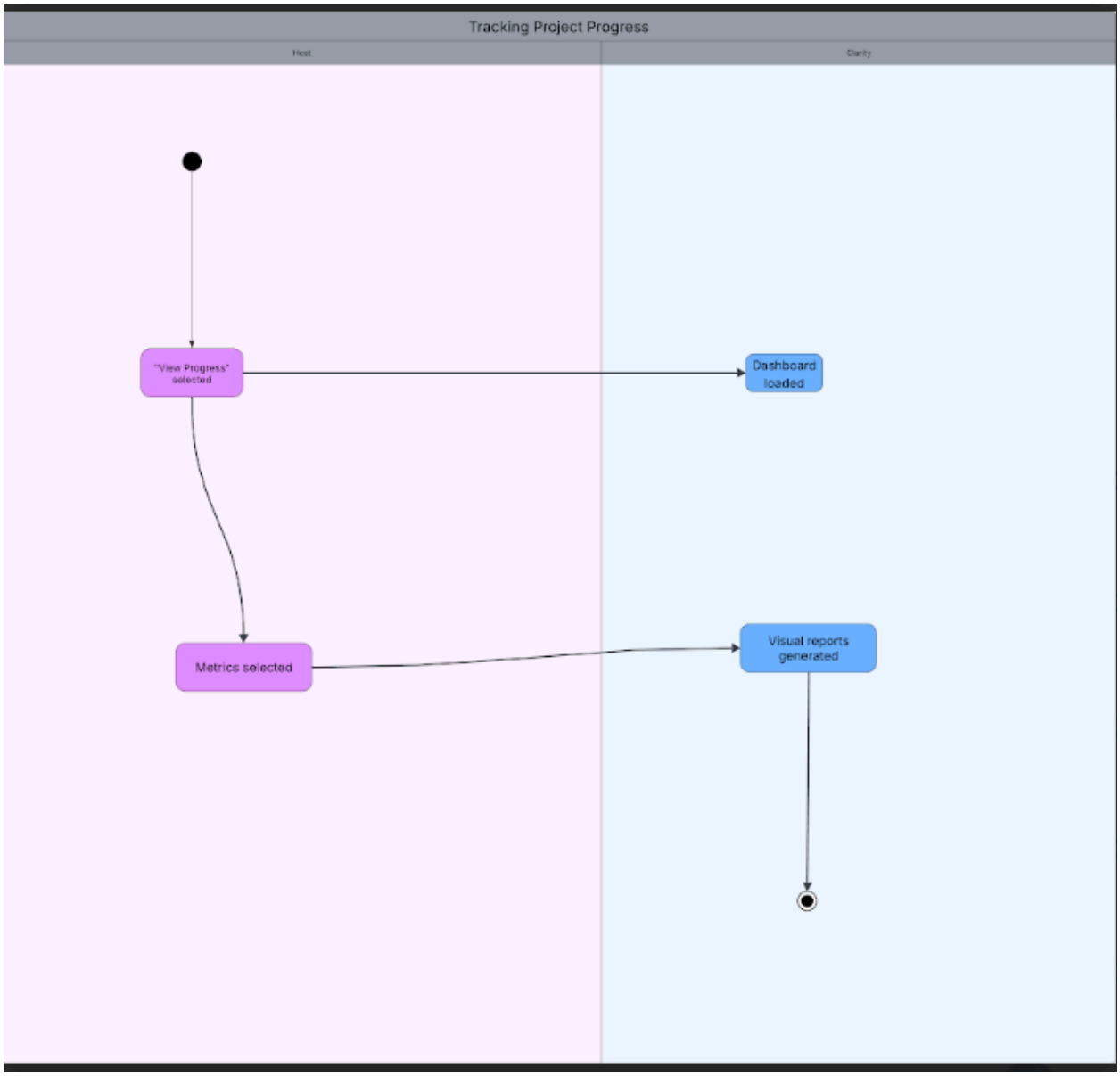
Activity diagram 2



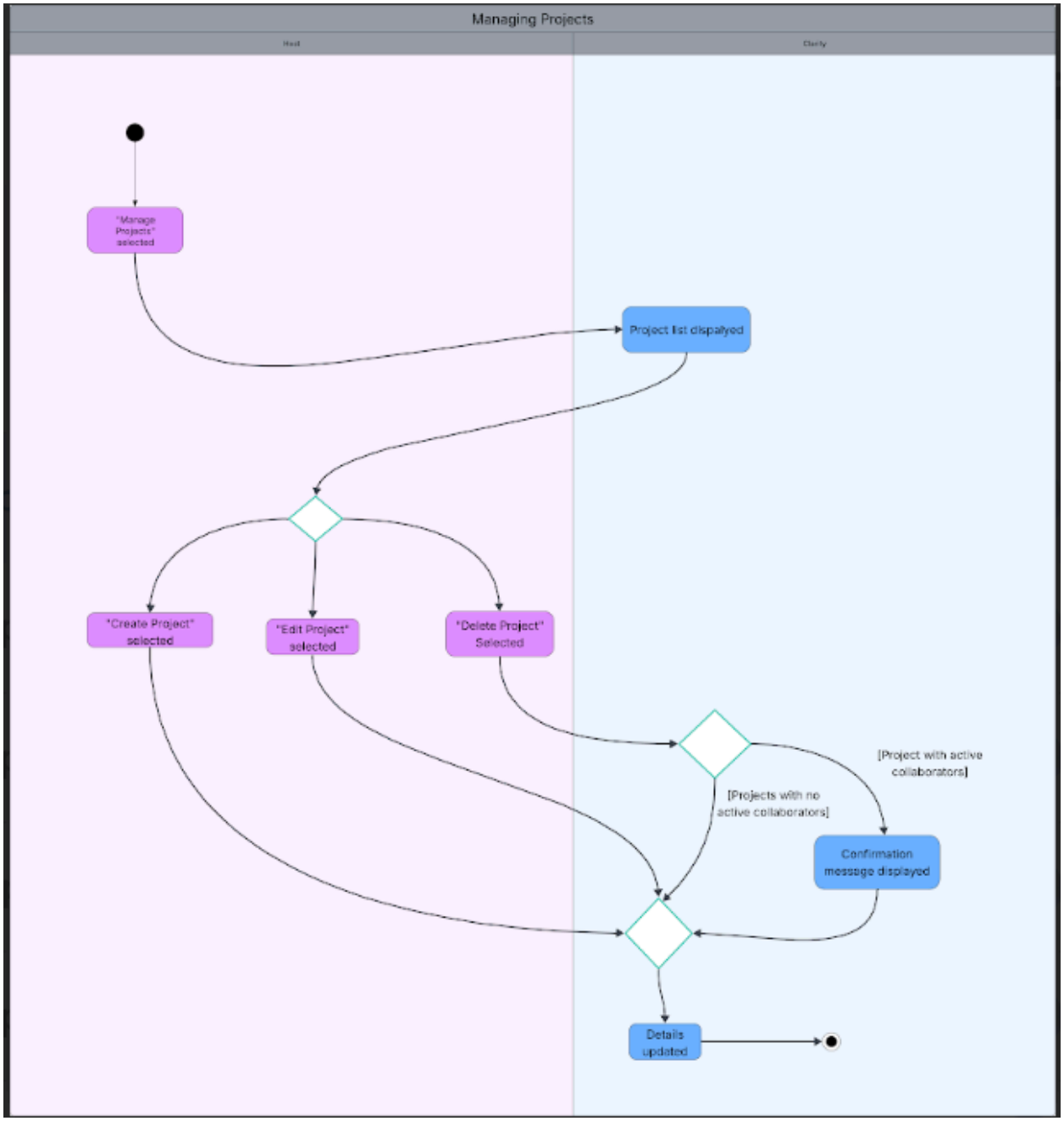
Activity diagram 3



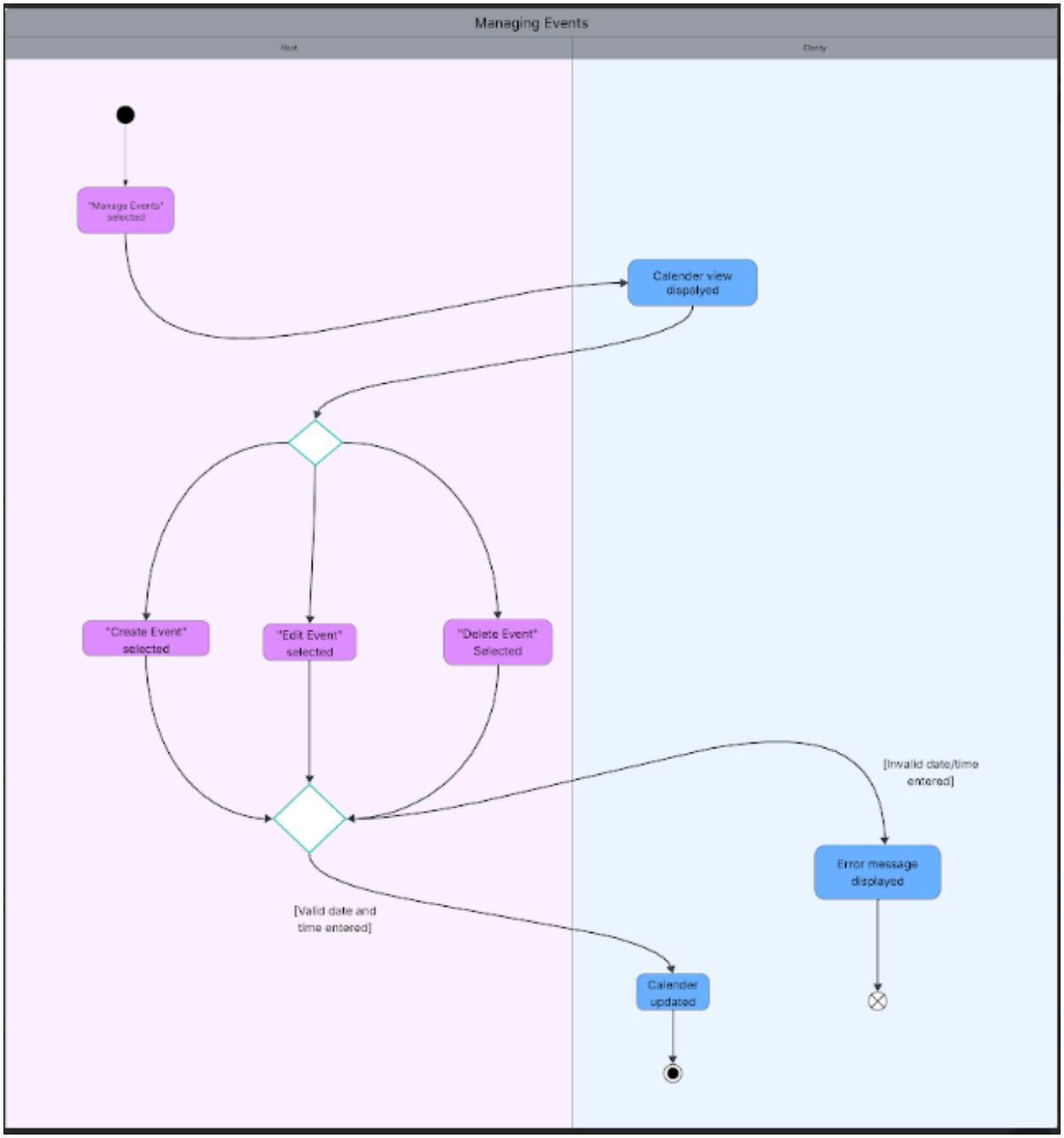
Activity diagram 4



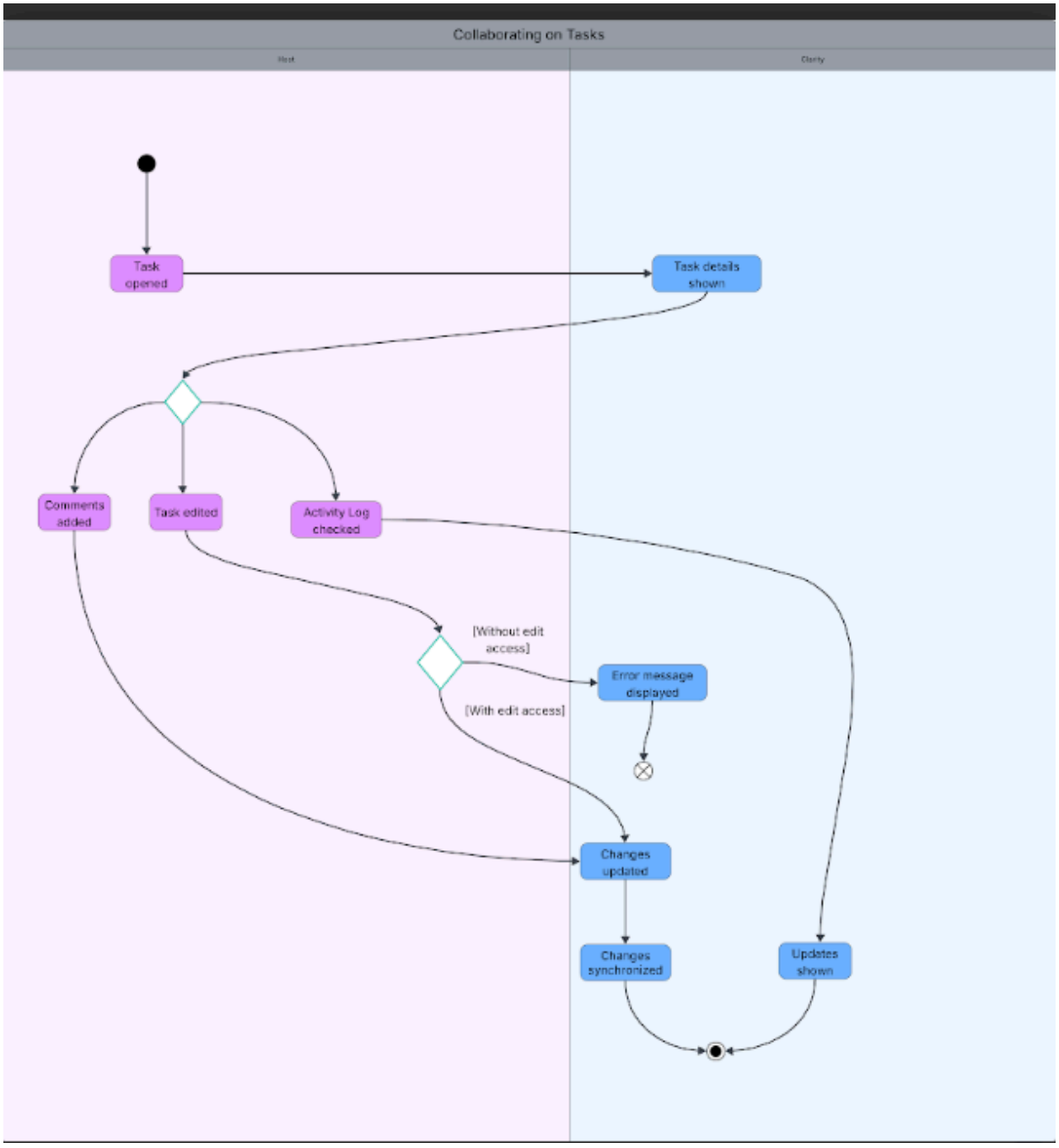
Activity diagram 5



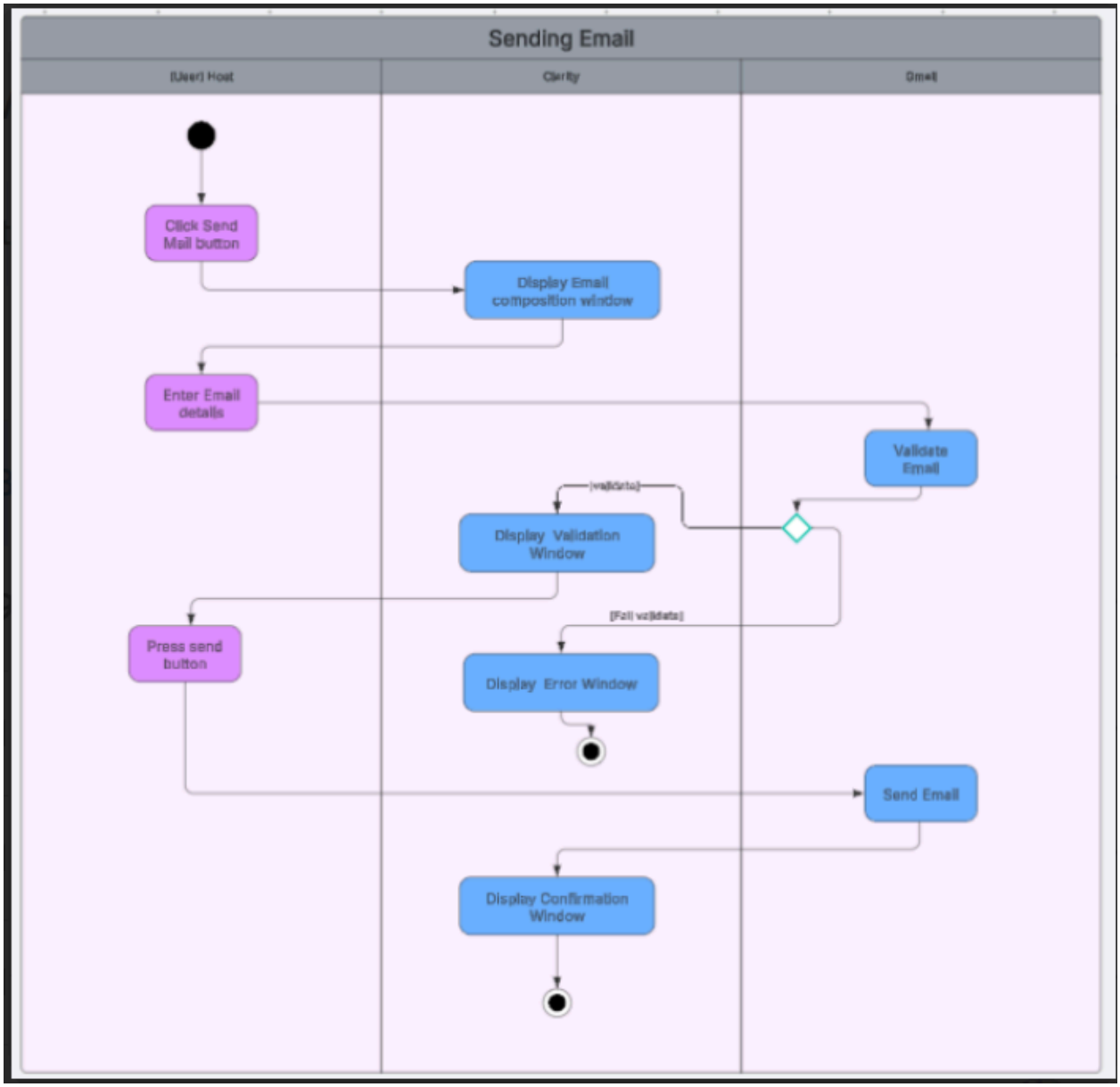
Activity diagram 6



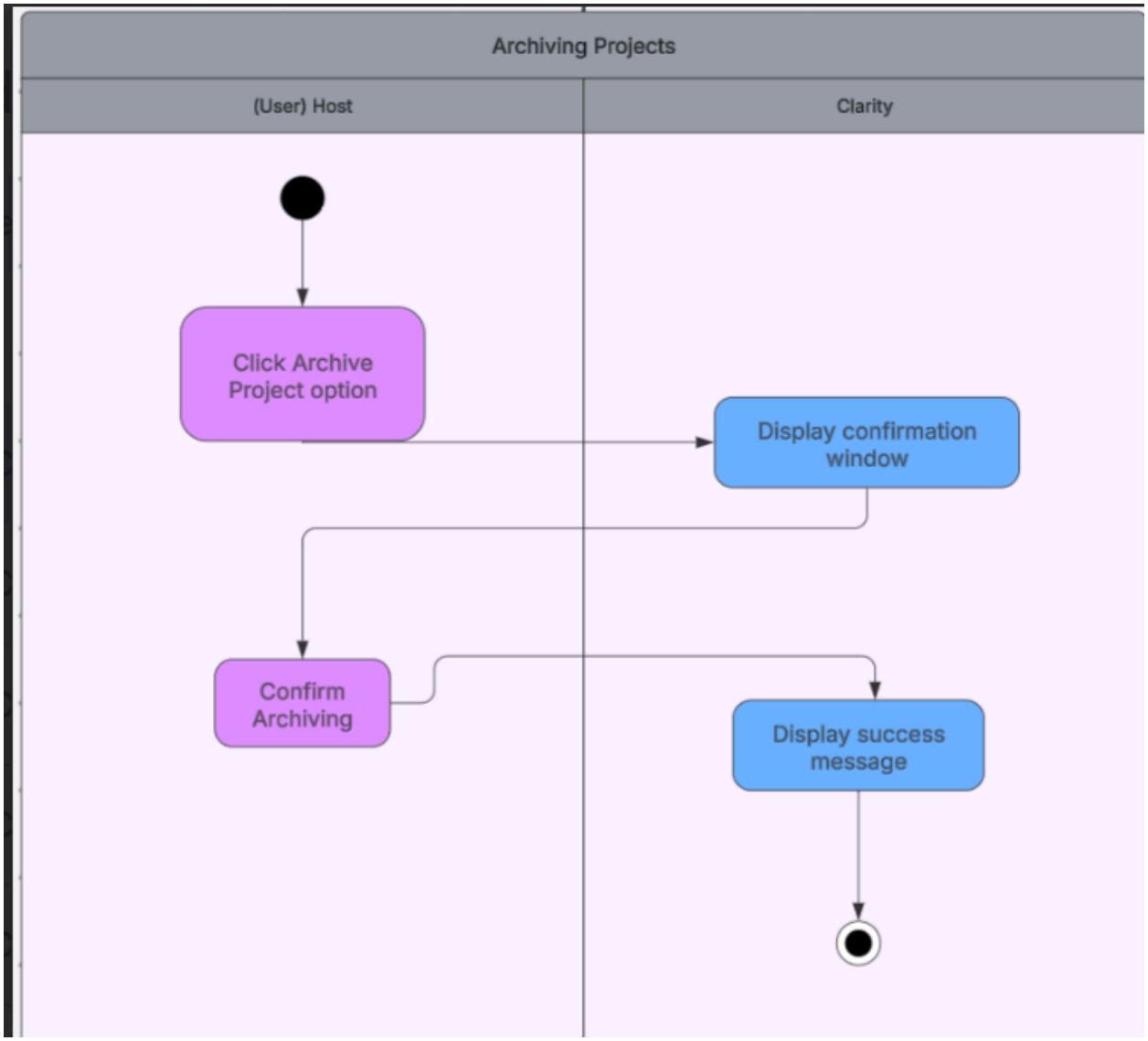
Activity diagram 7



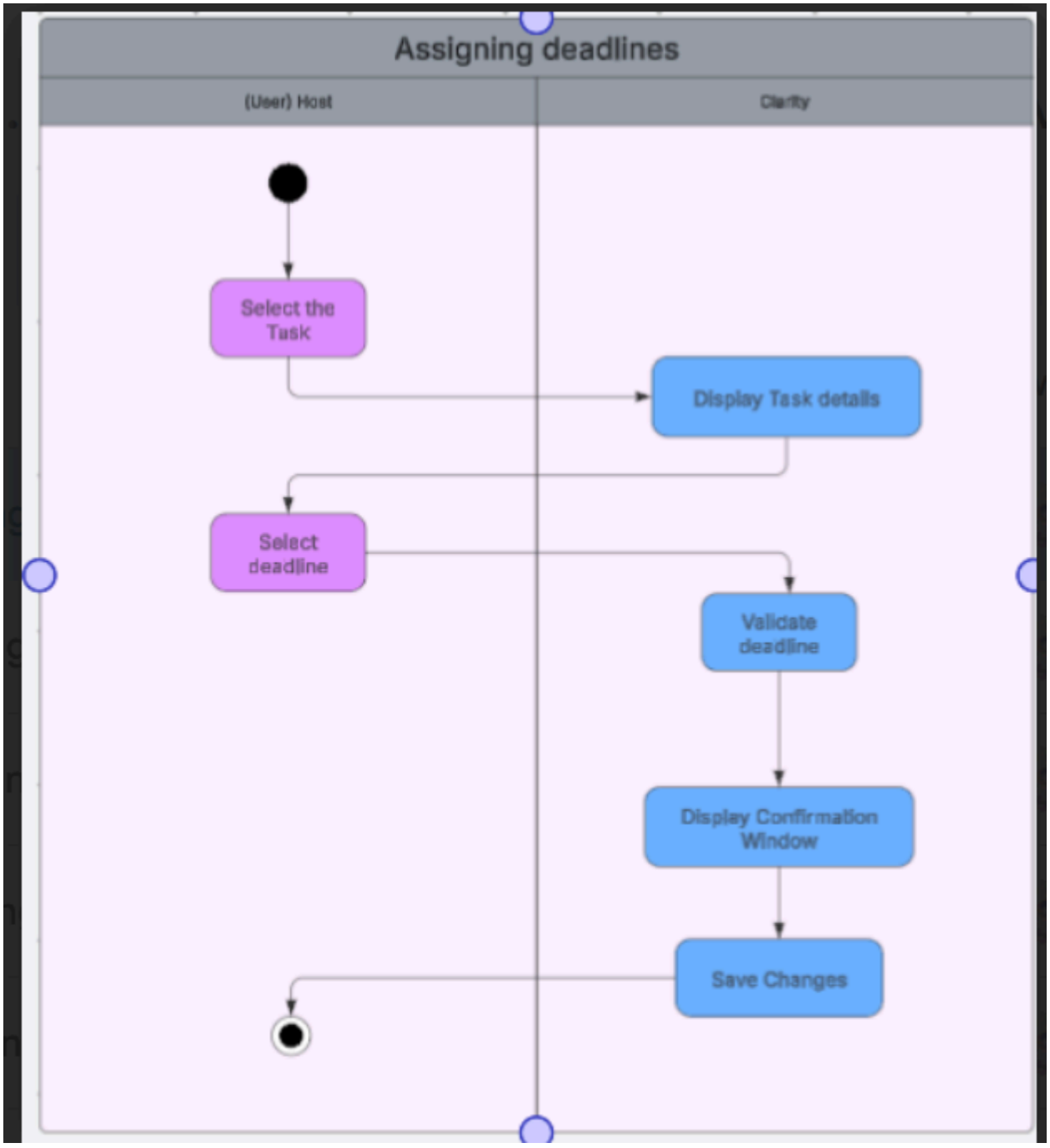
Activity diagram 8



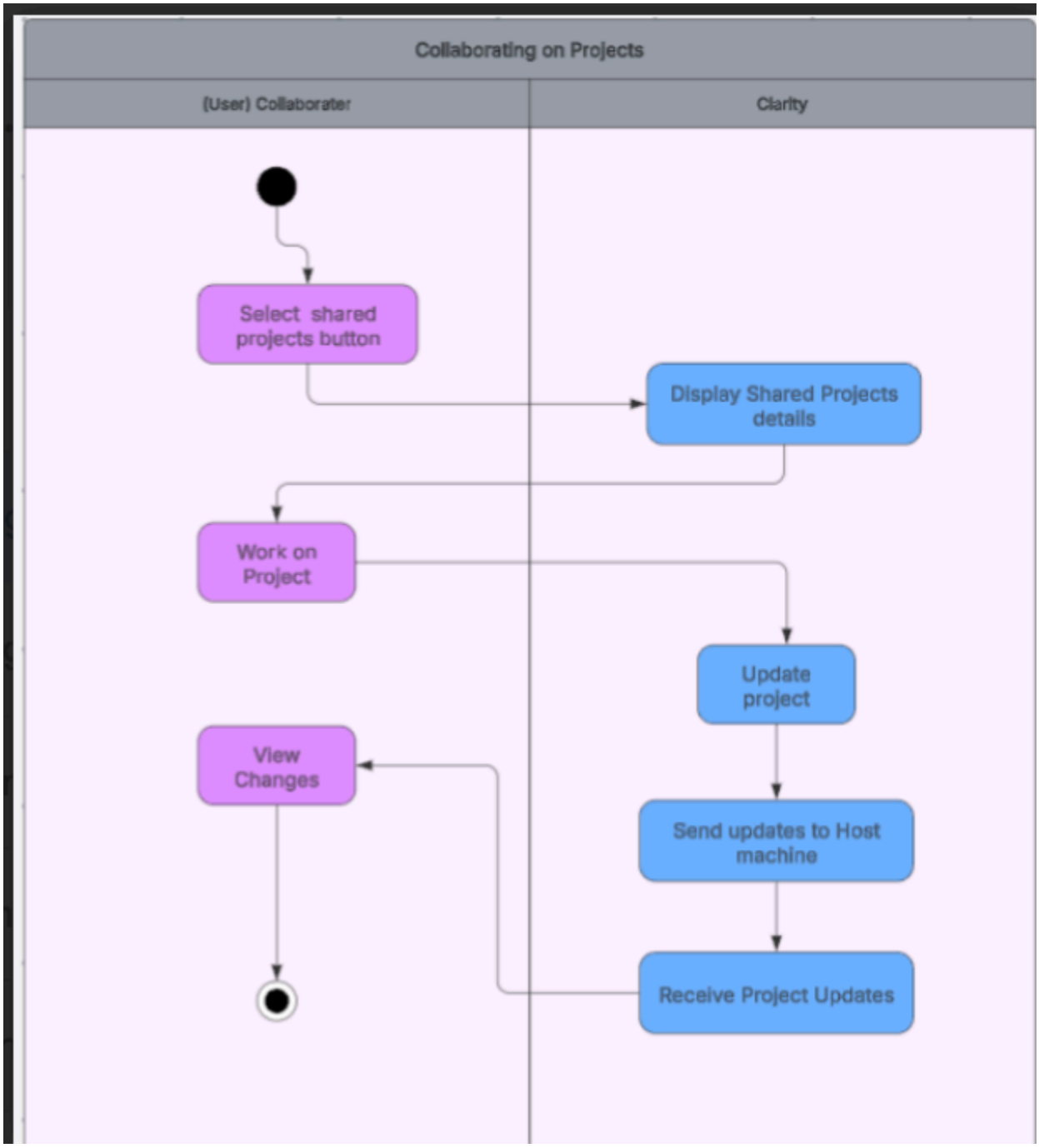
Activity diagram 9



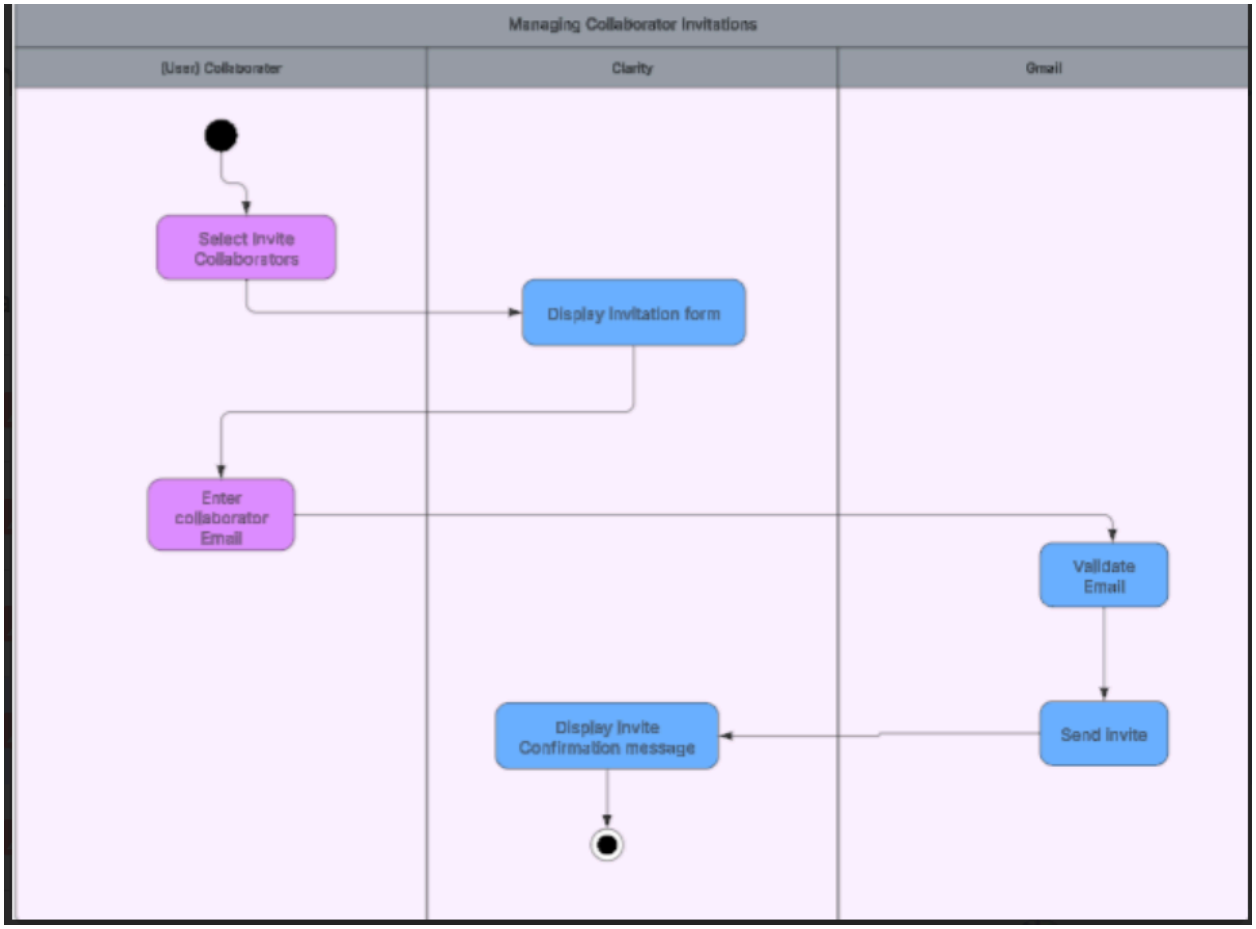
Activity diagram 10



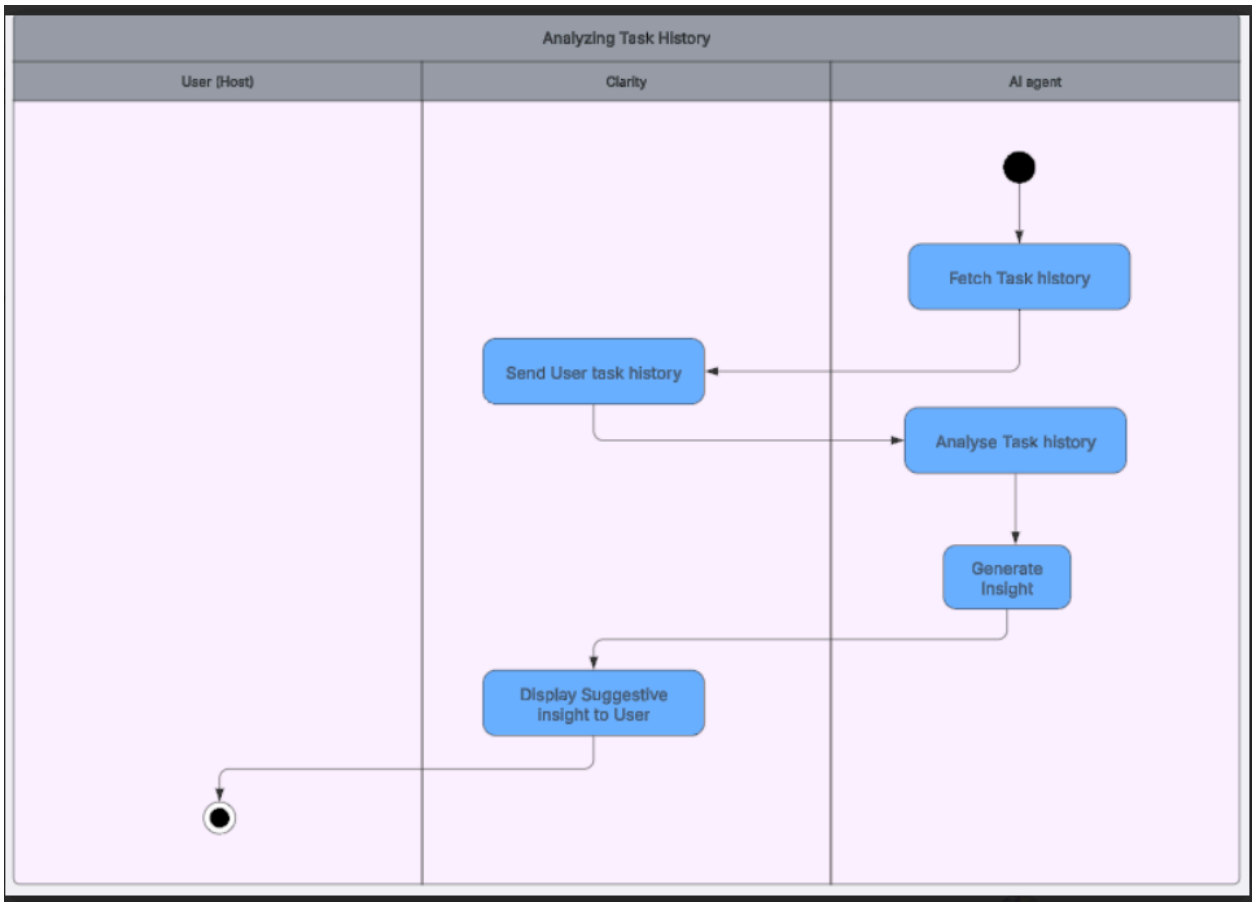
Activity diagram 11



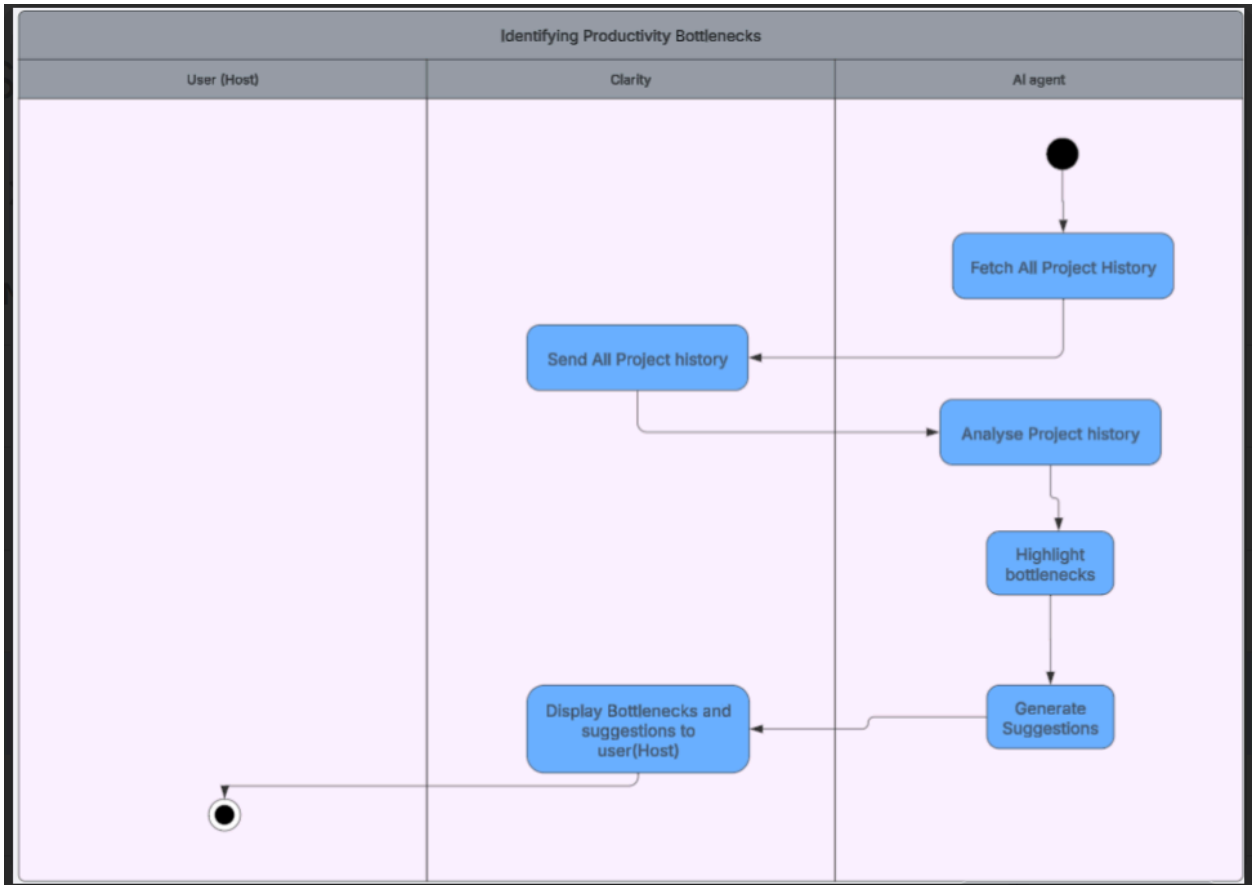
Activity diagram 12



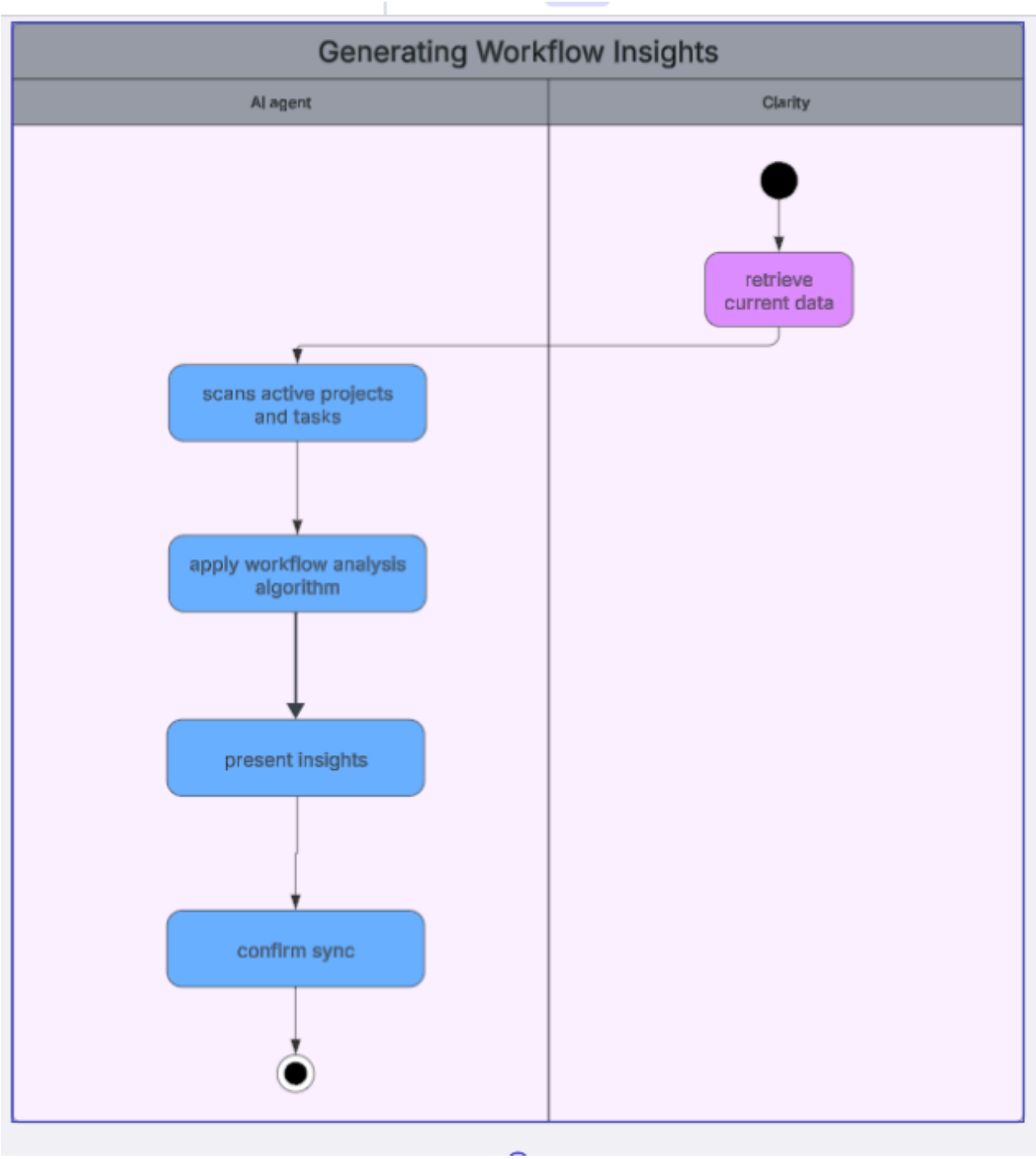
Activity diagram 13



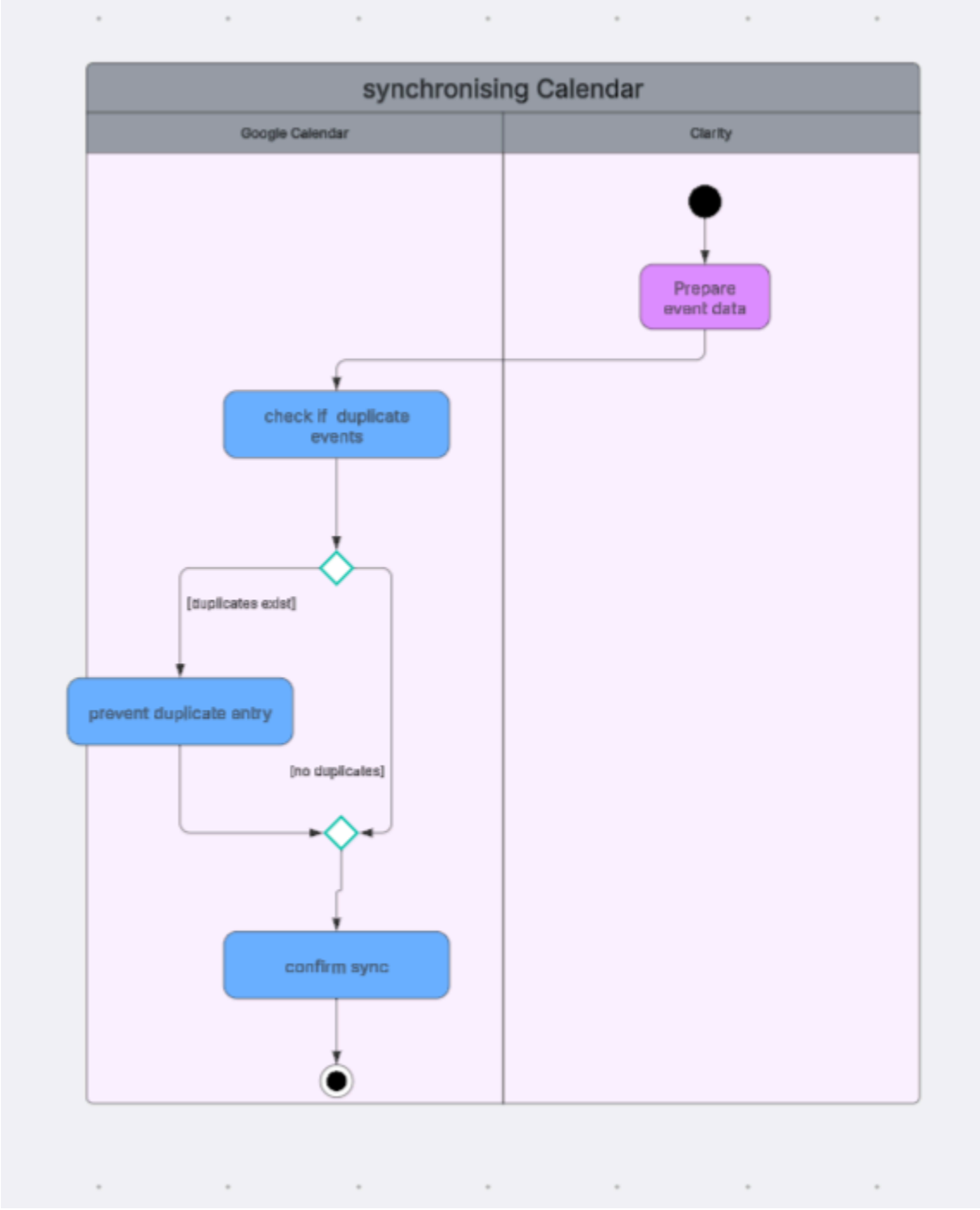
Activity diagram 14



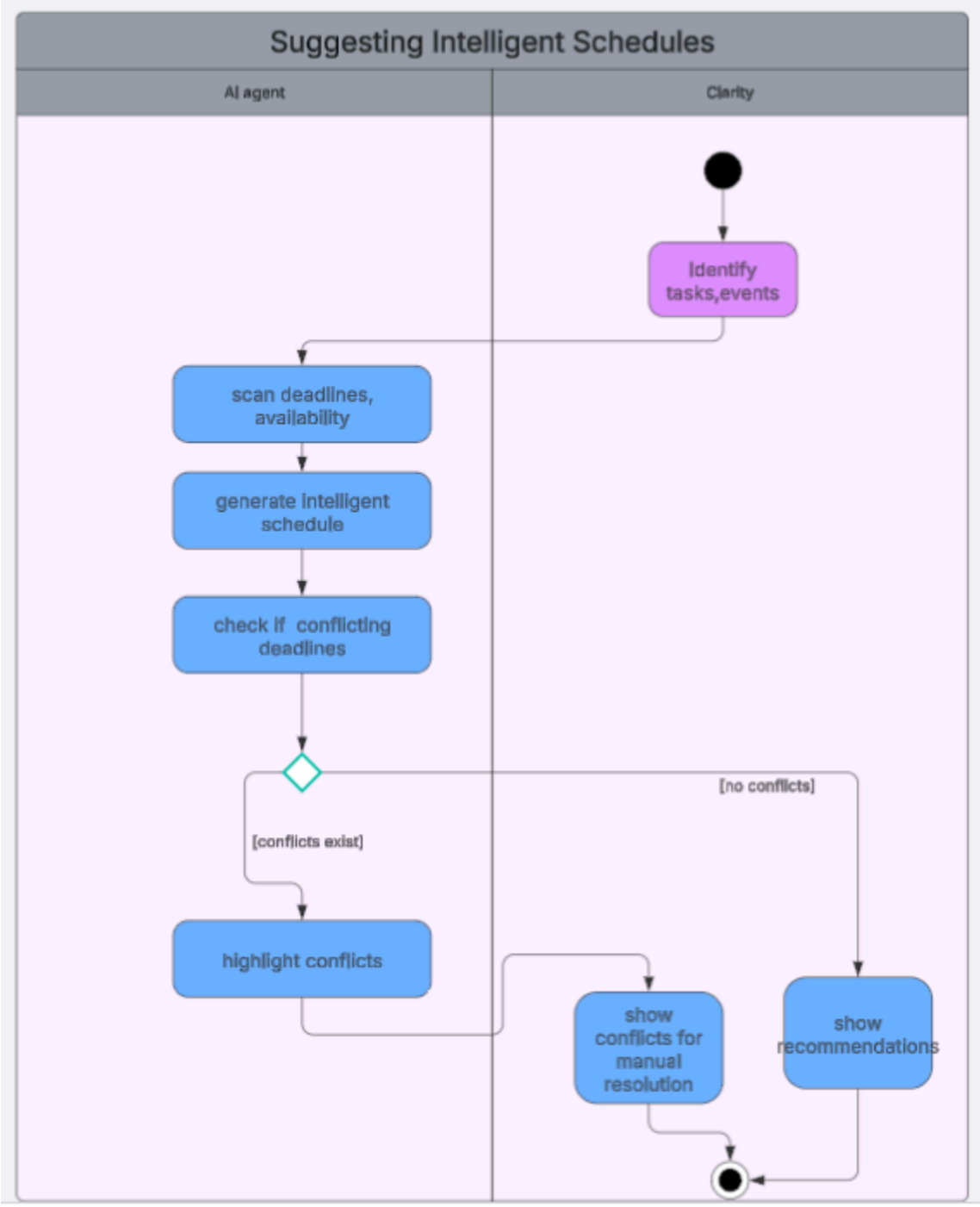
Activity diagram 15



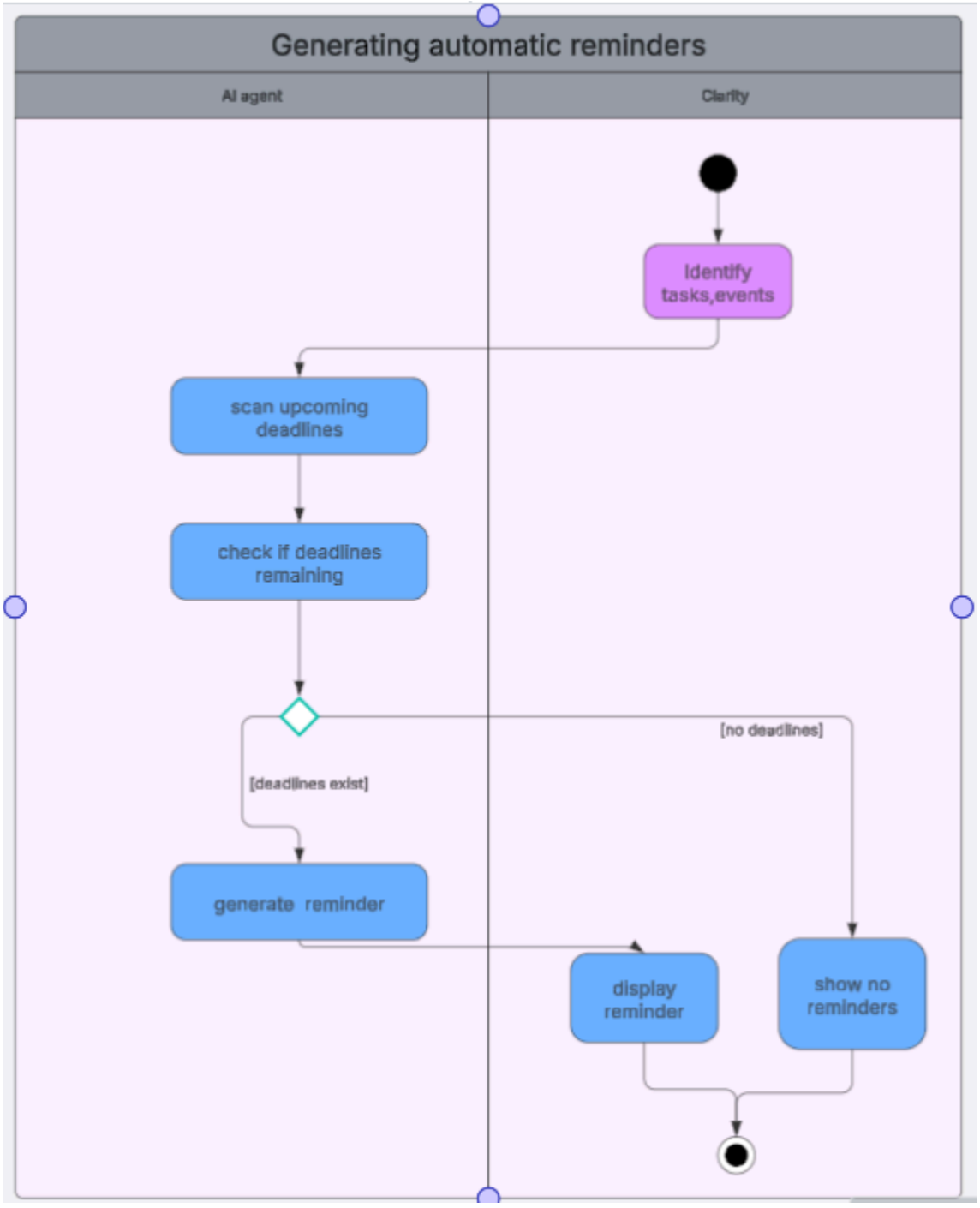
Activity diagram 16



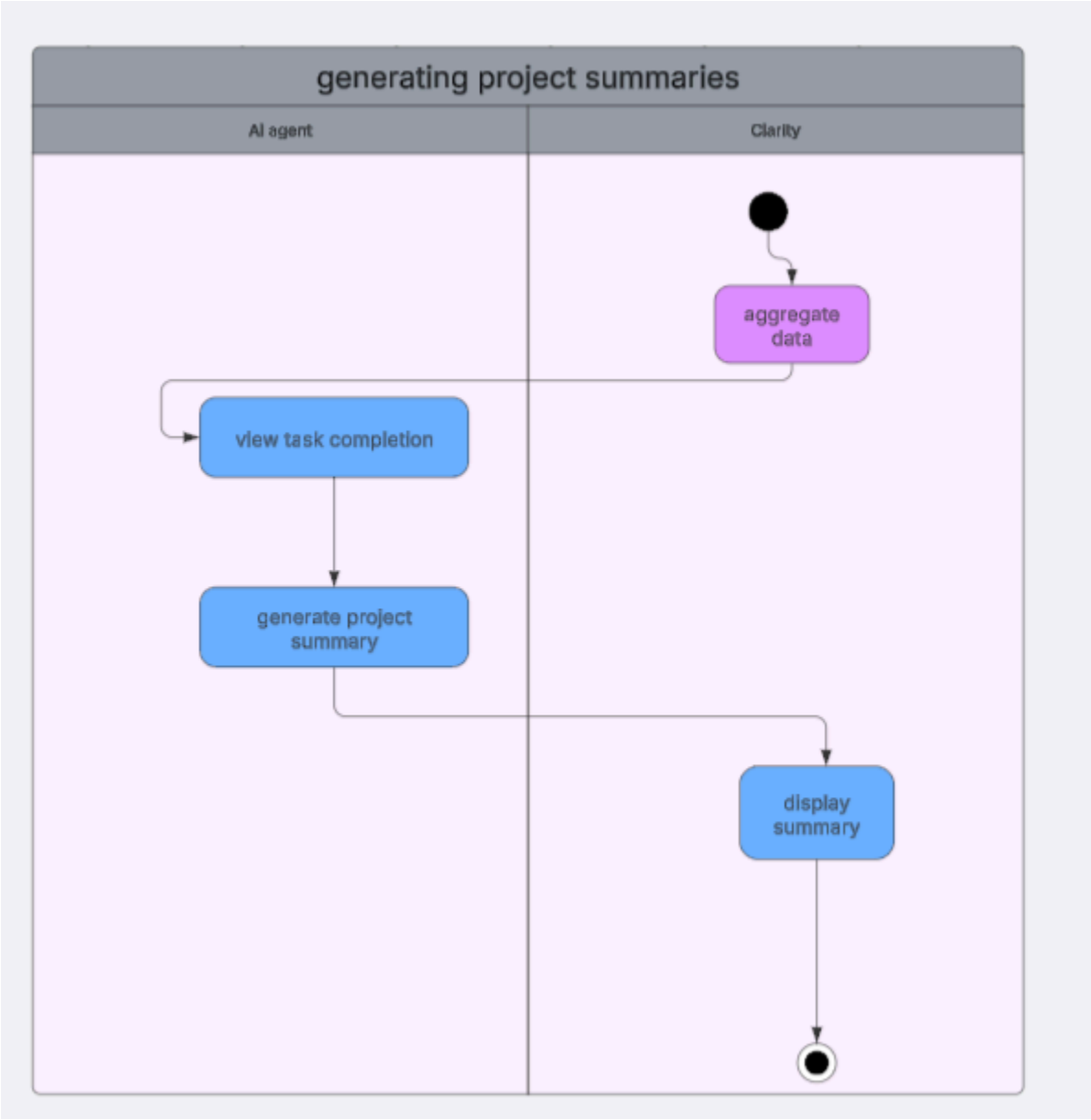
Activity diagram 17



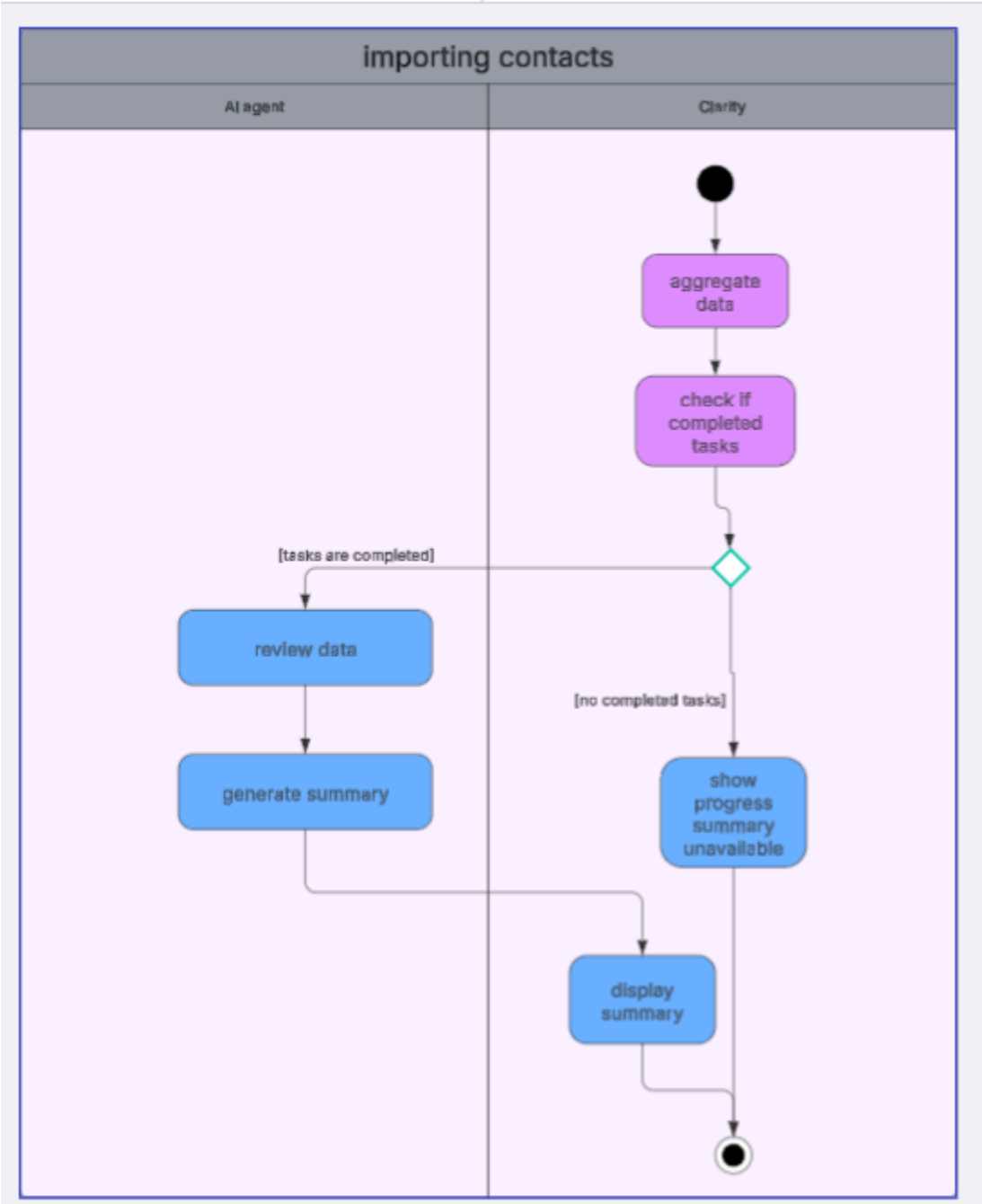
Activity diagram 18



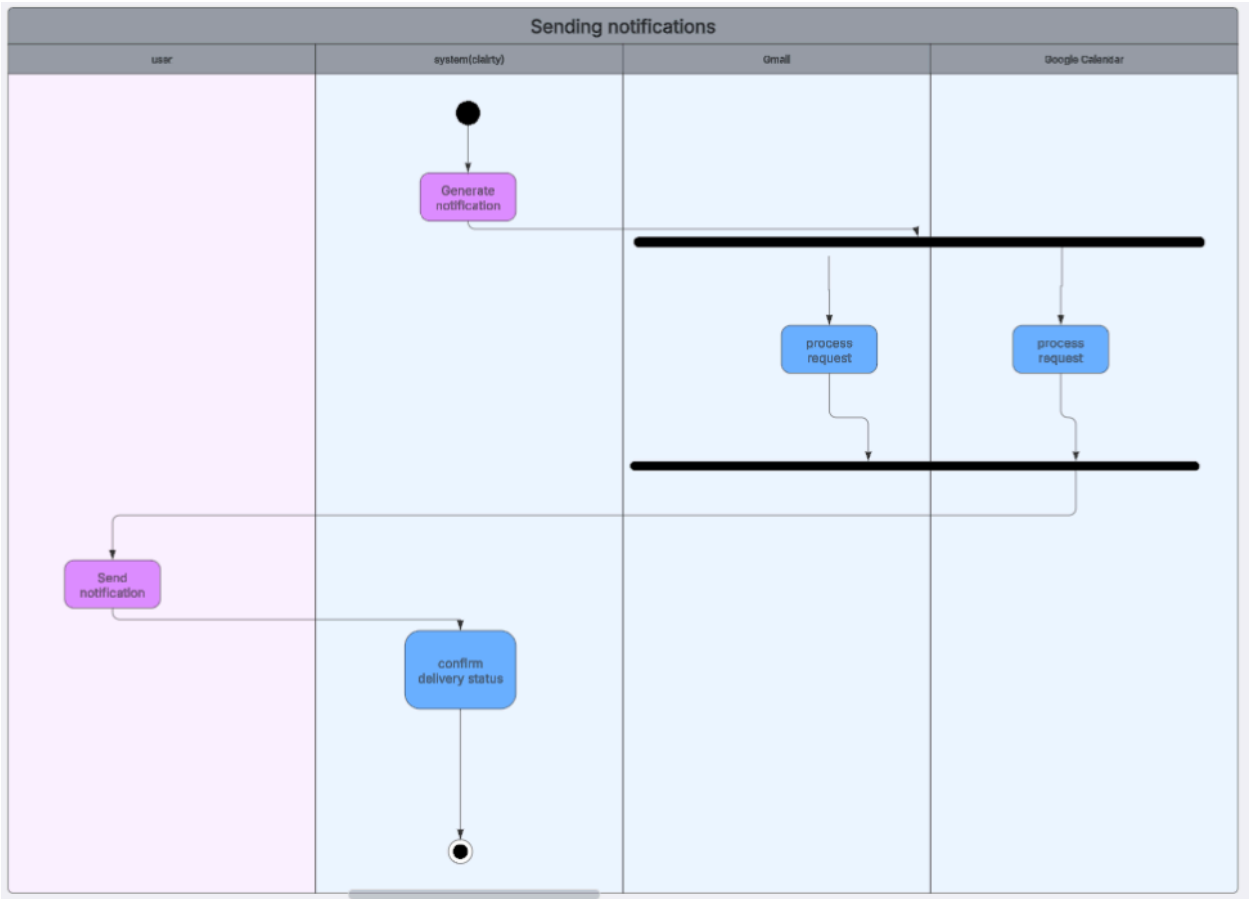
Activity diagram 19



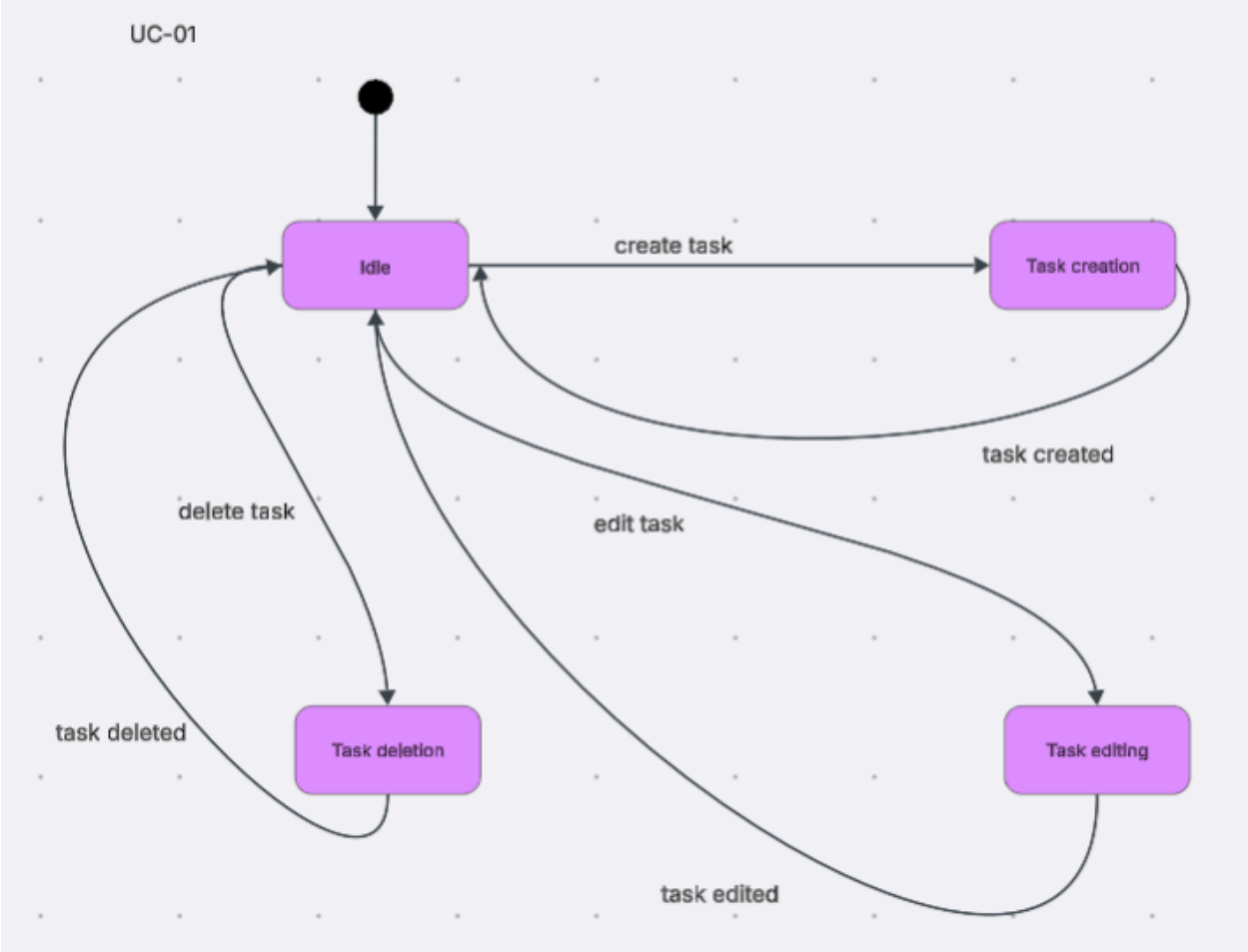
Activity diagram 20



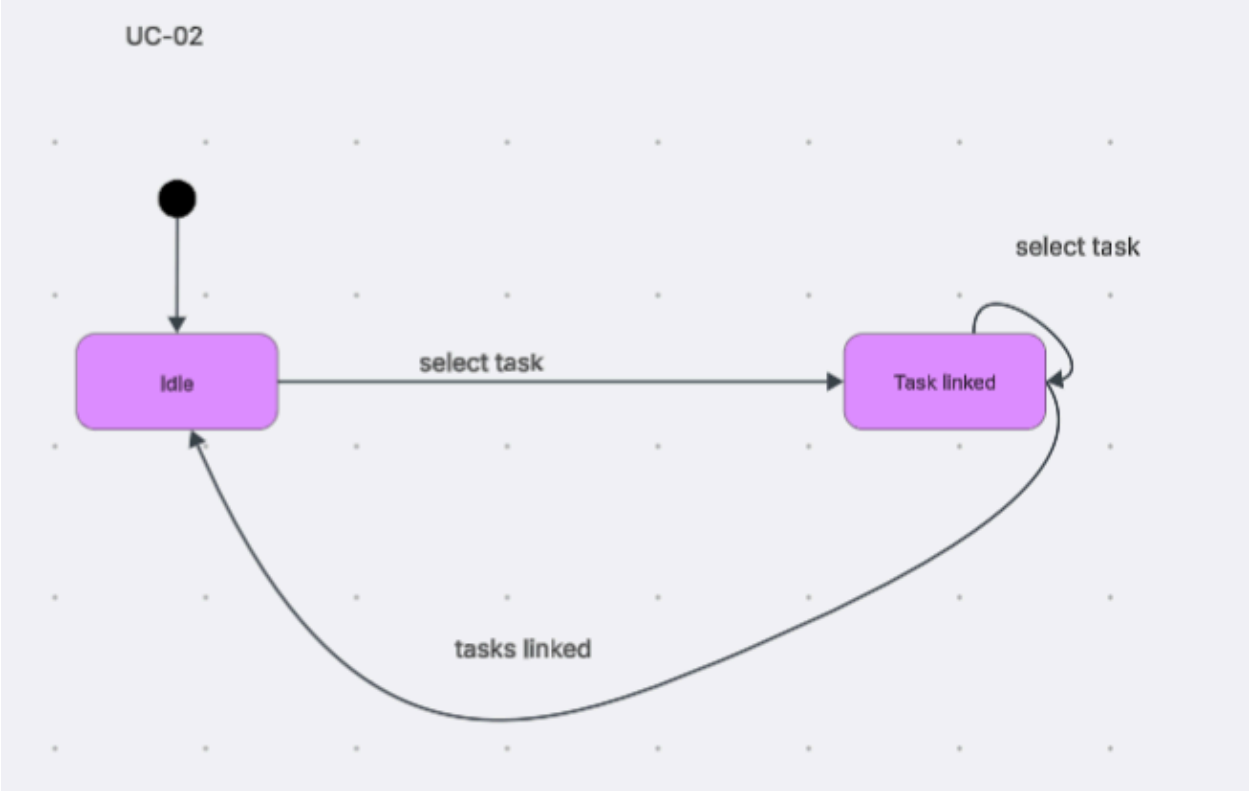
Activity diagram 21



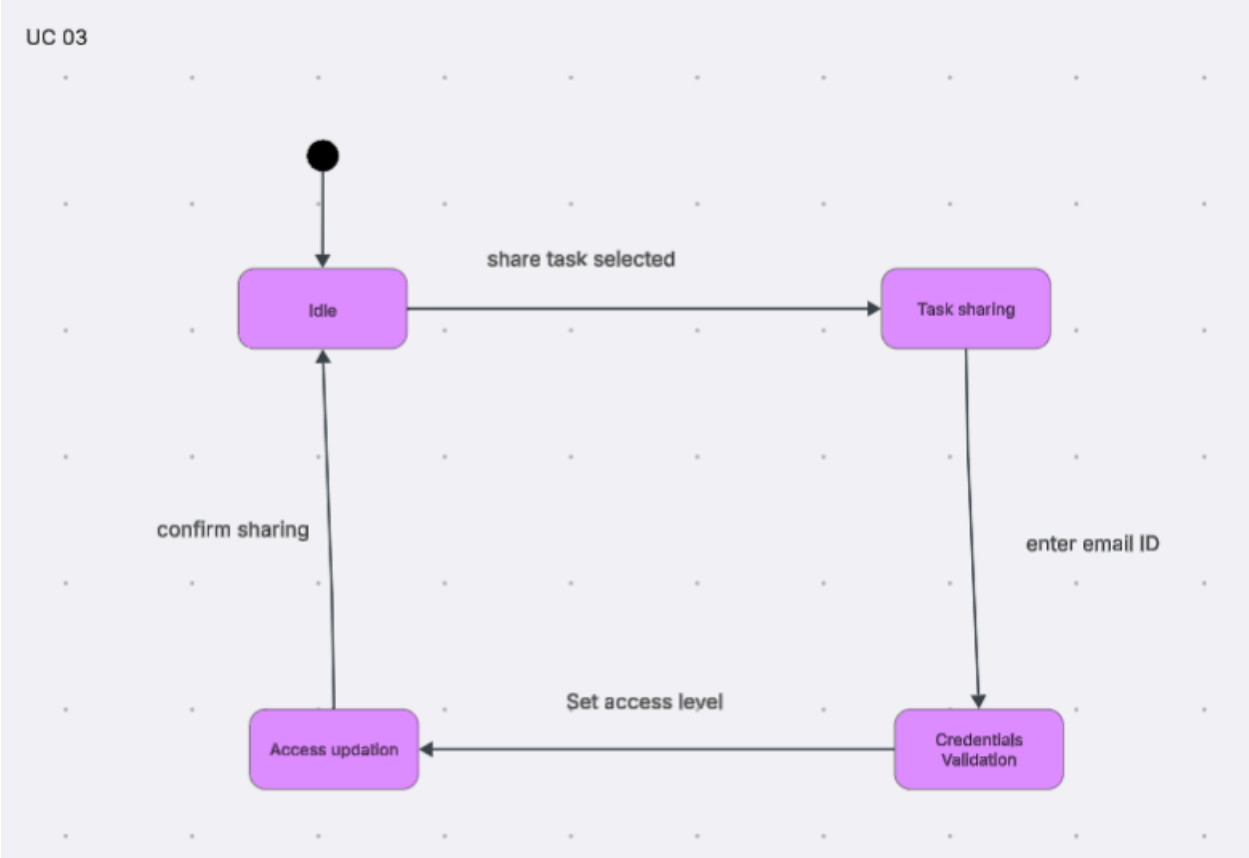
State Diagram 1



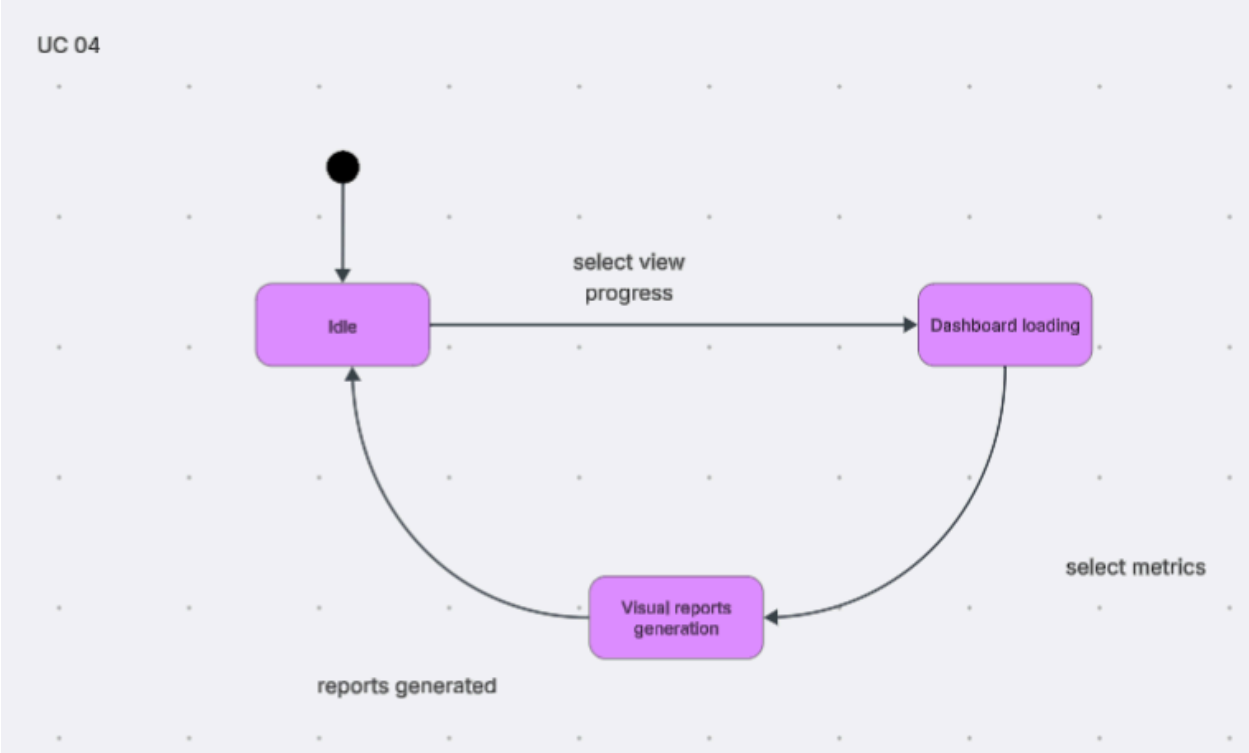
State Diagram 2



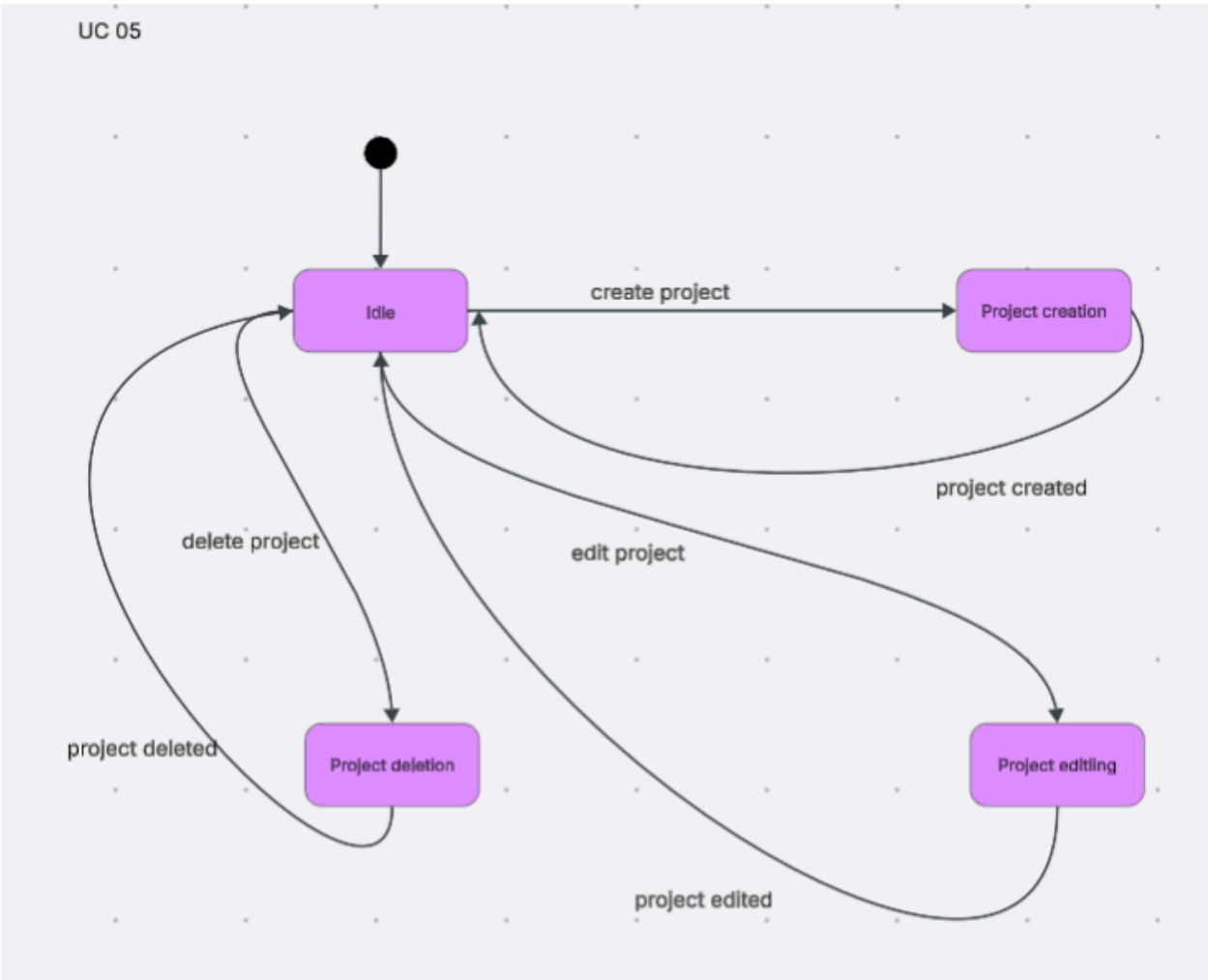
State Diagram 3



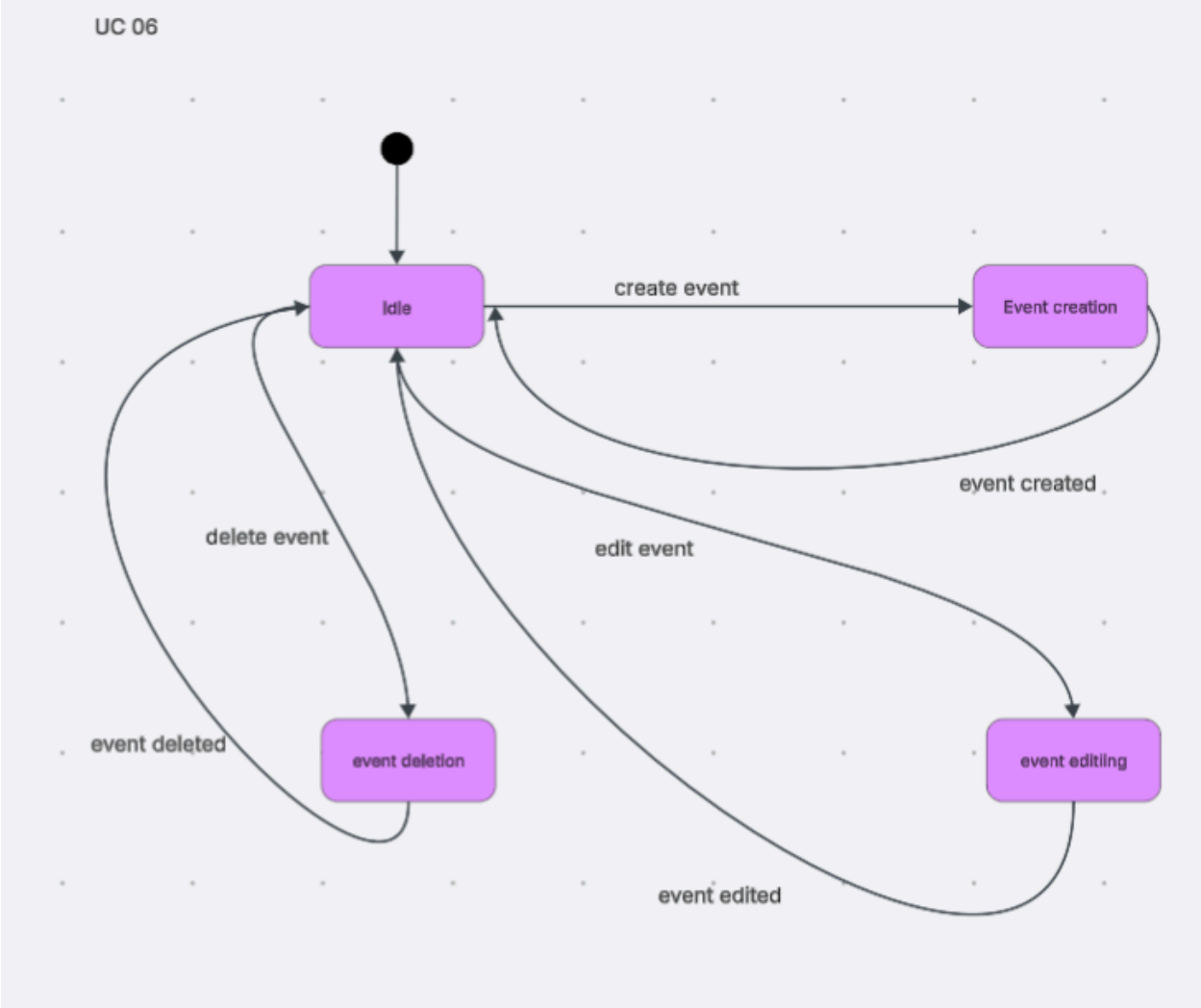
State Diagram 4



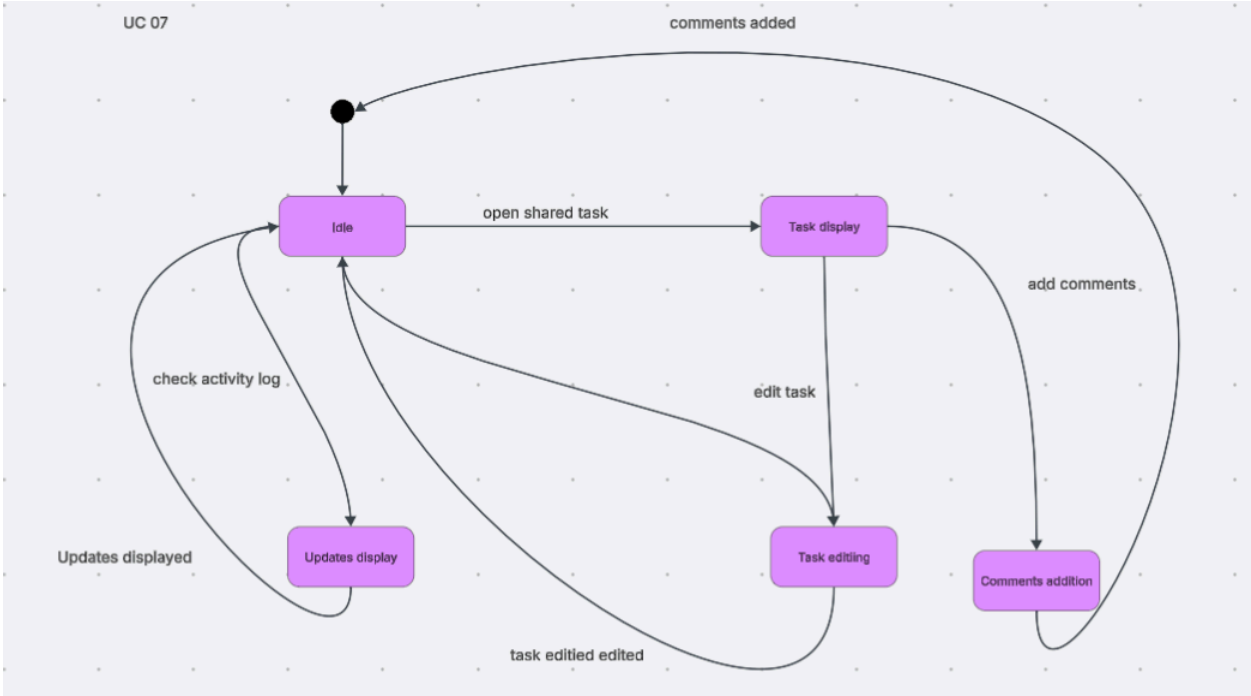
State Diagram 5



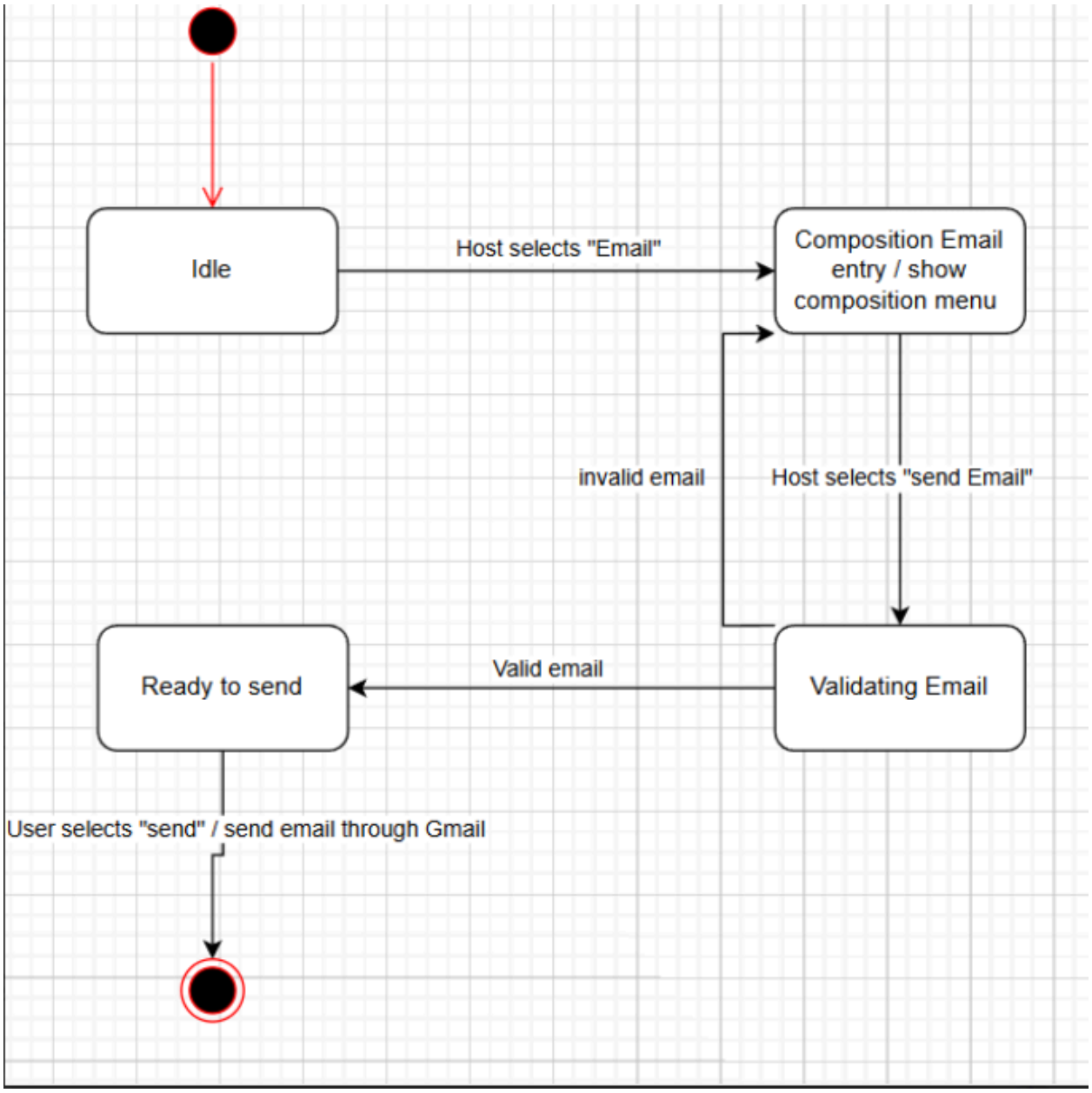
State Diagram 6



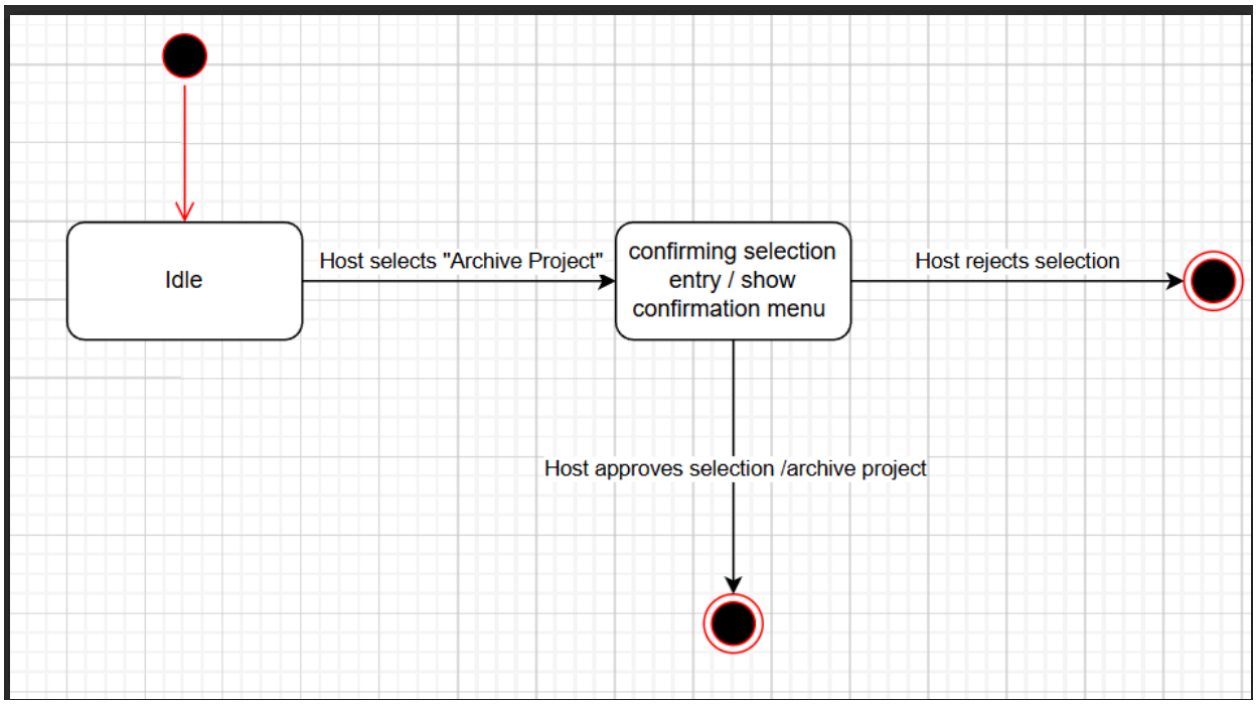
State Diagram 7



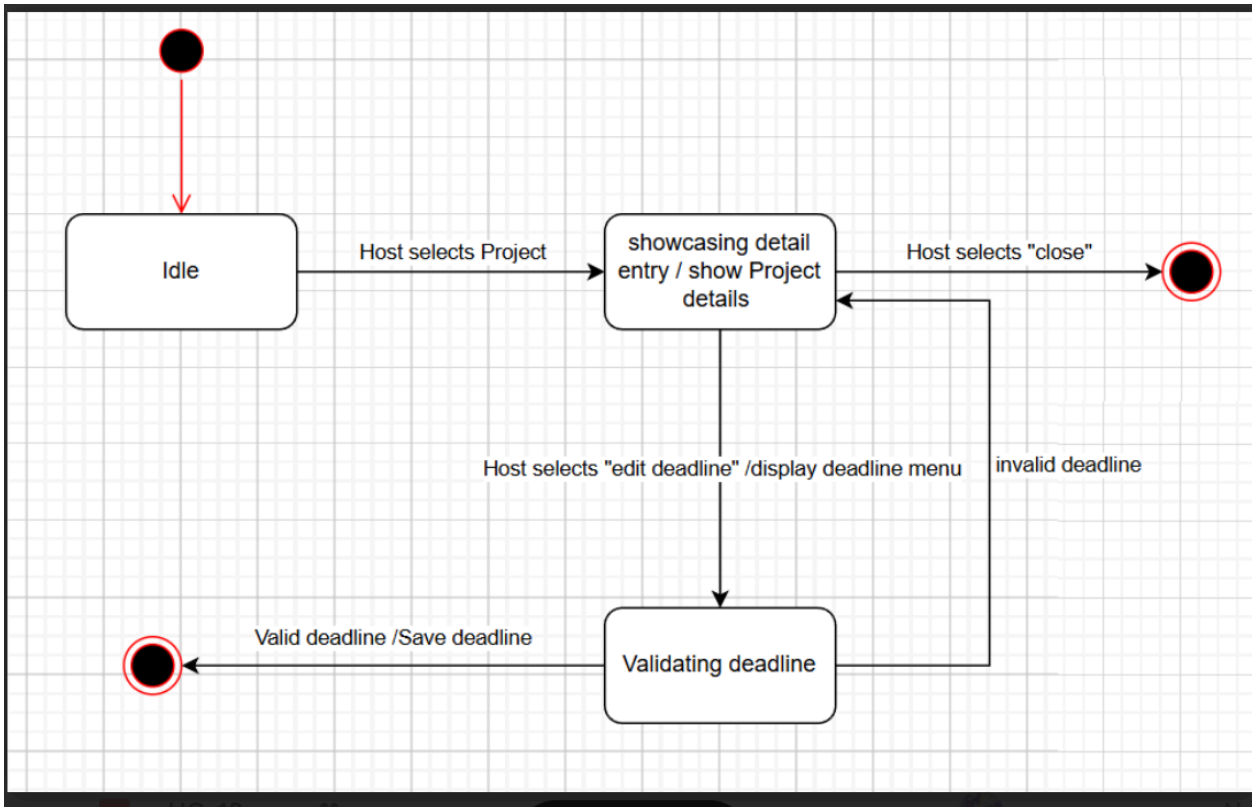
State Diagram 8



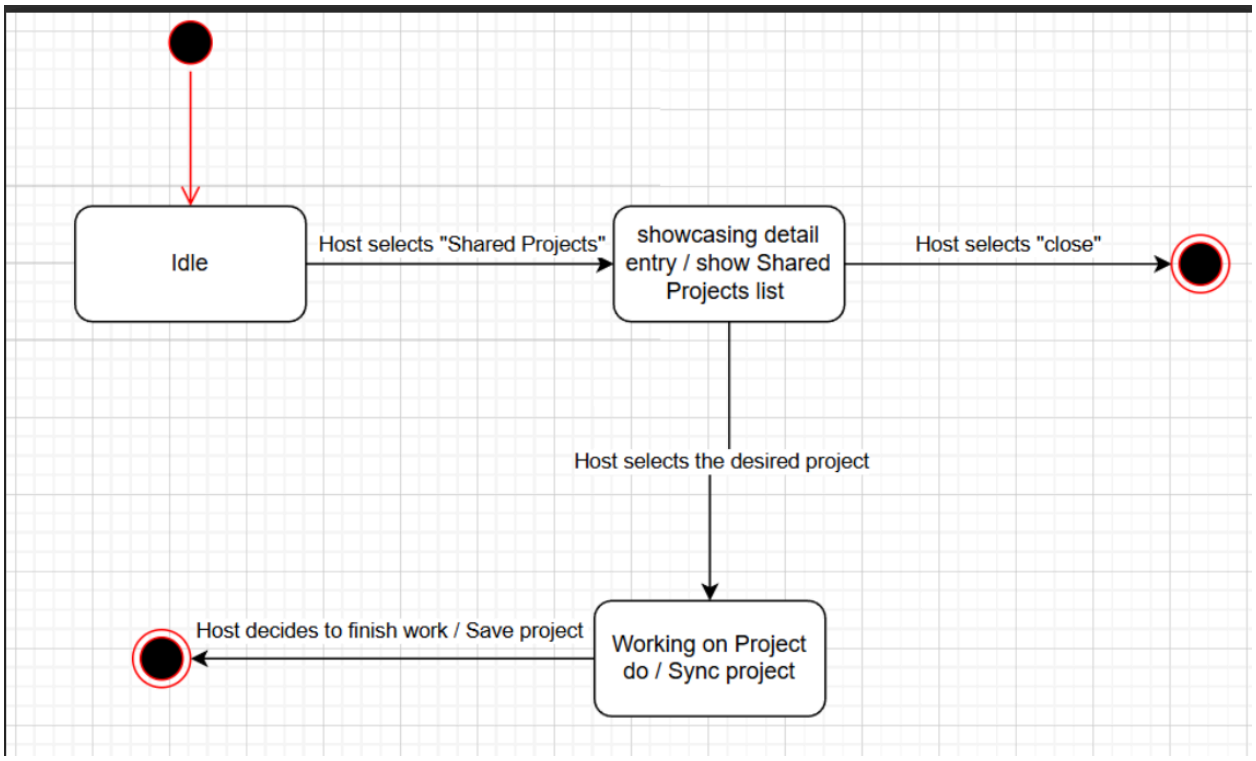
State Diagram 9



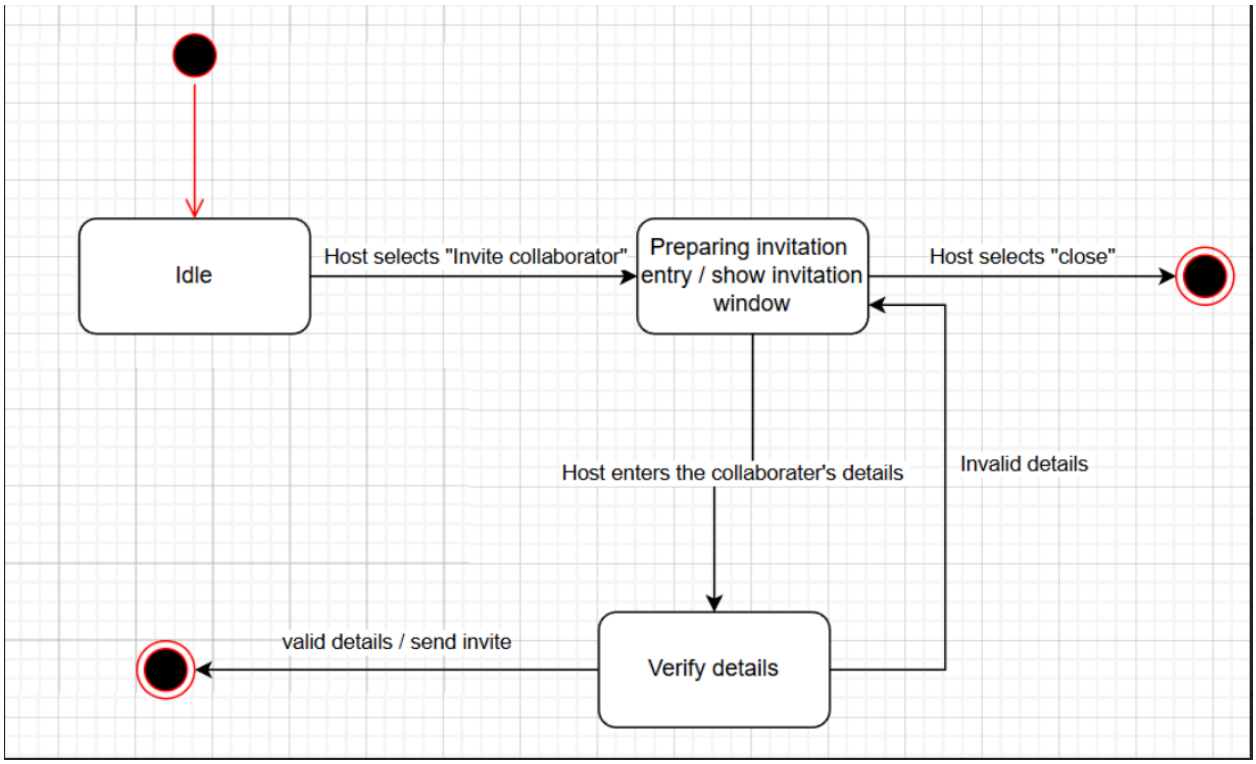
State Diagram 10



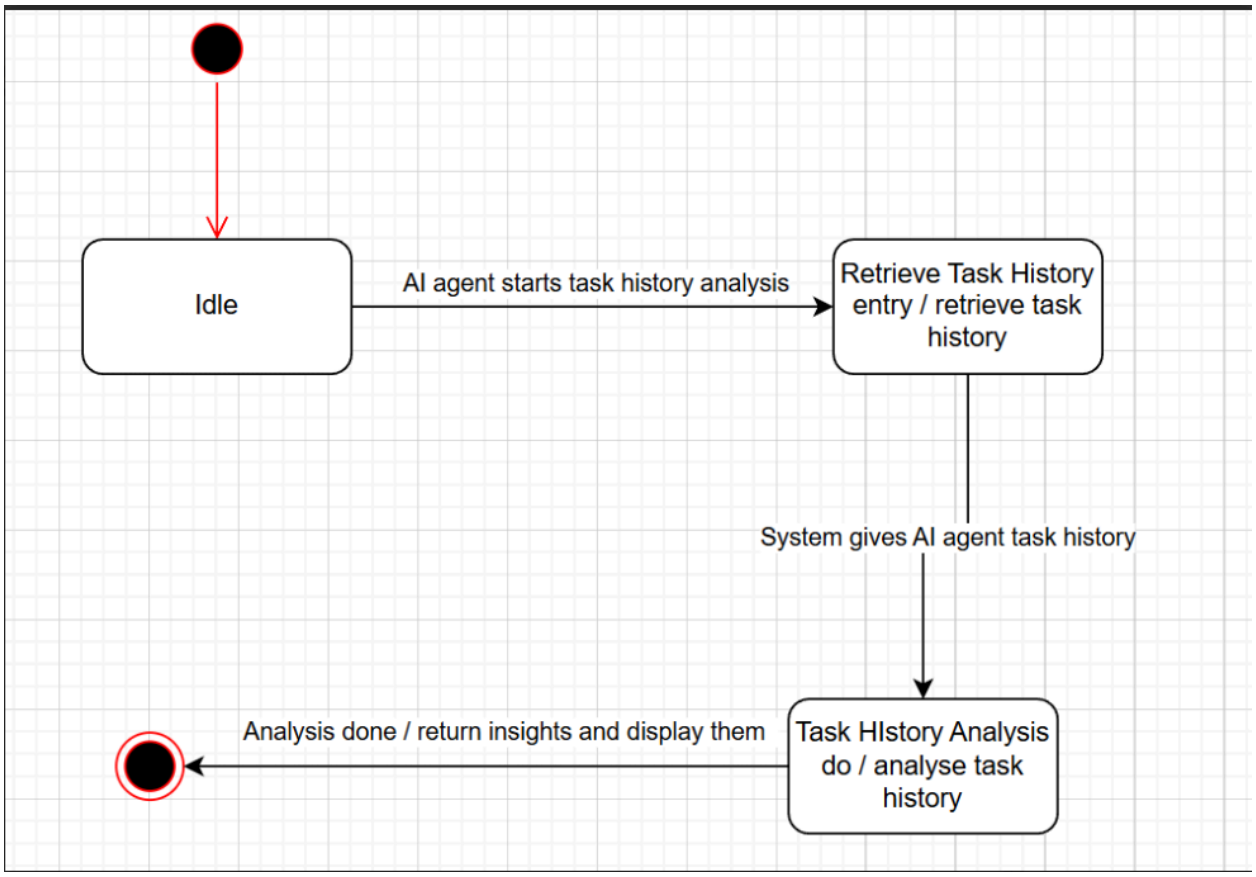
State Diagram 11



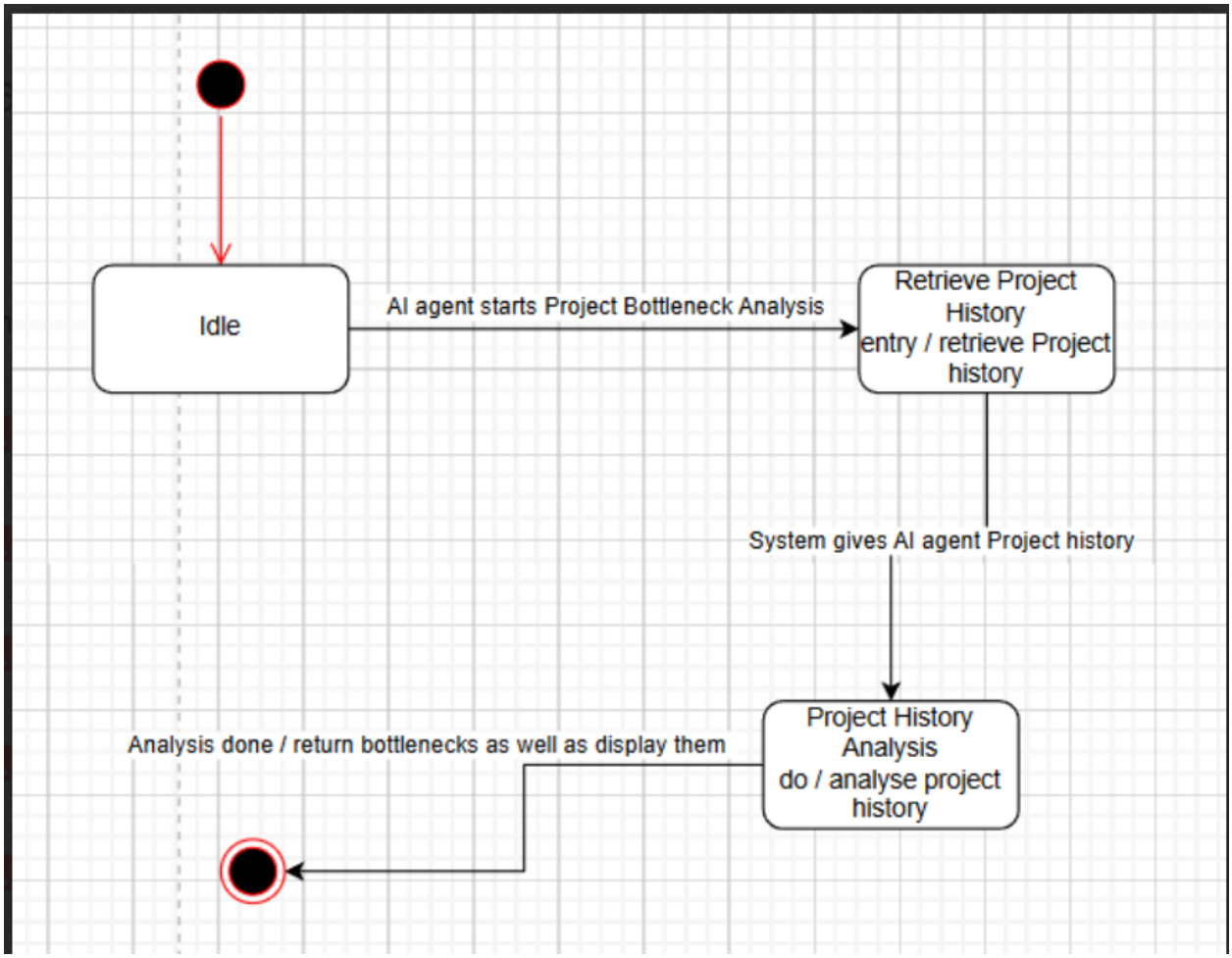
State Diagram 12



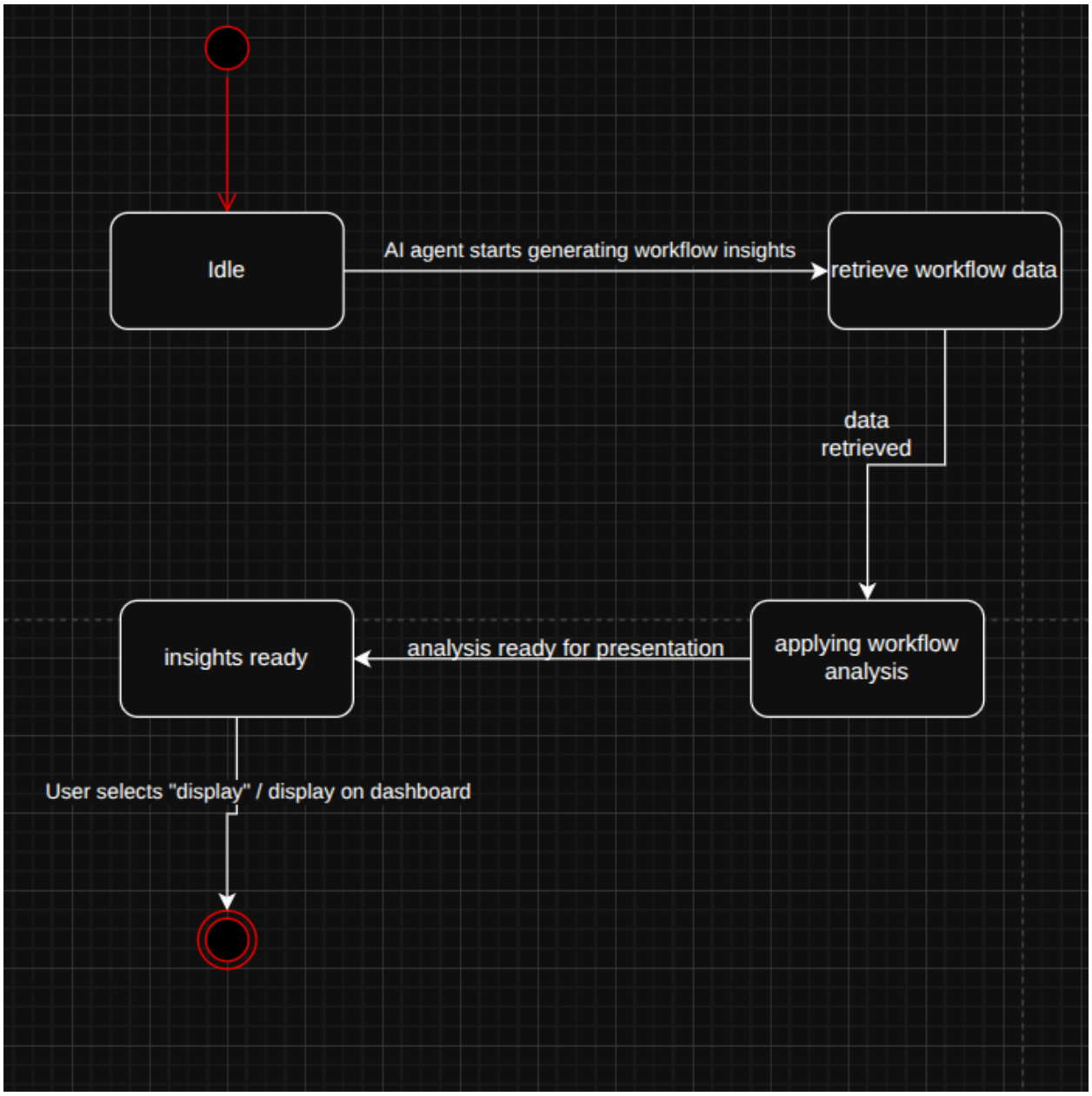
State Diagram 13



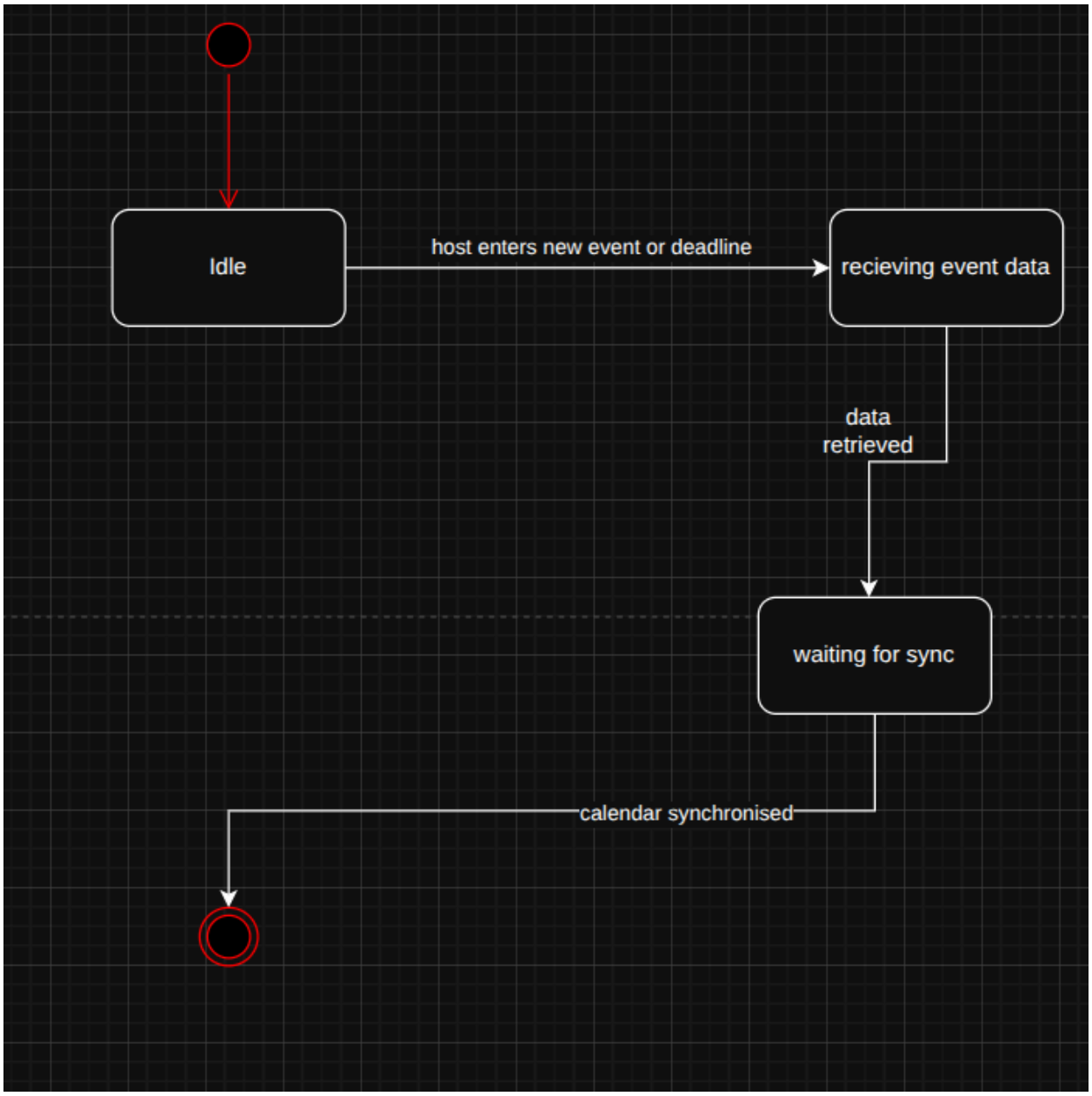
State Diagram 14



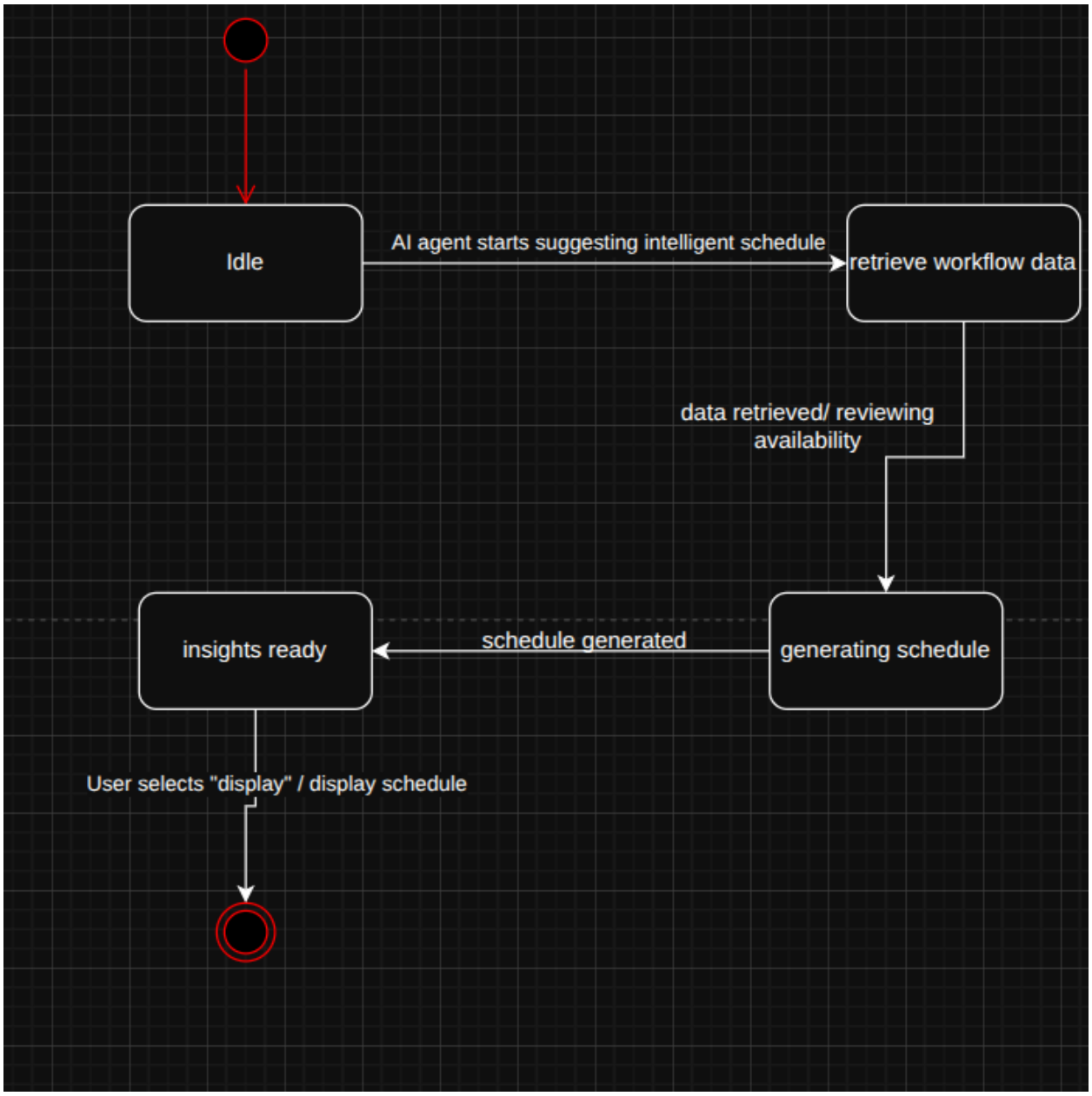
State Diagram 15



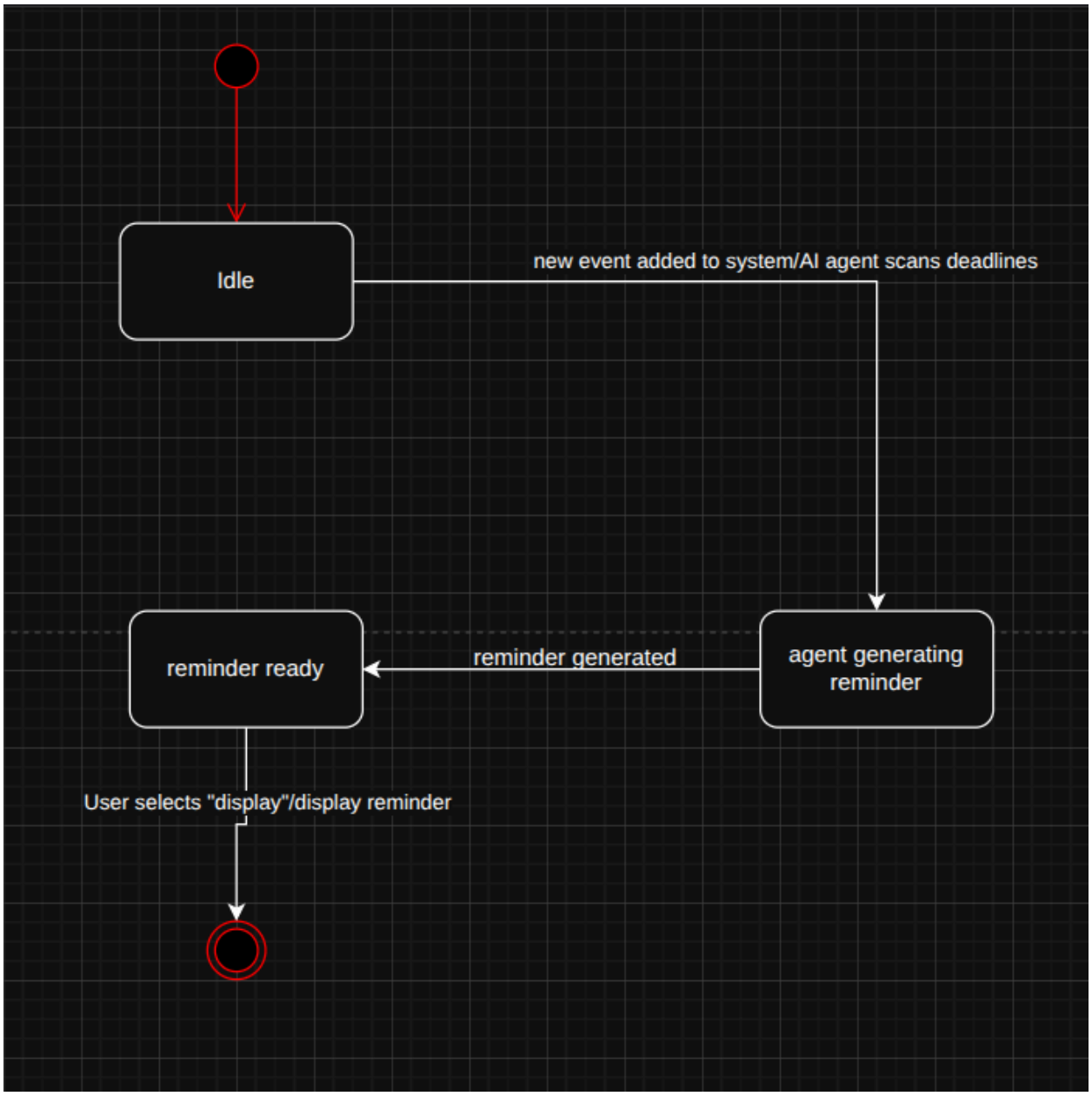
State Diagram 16



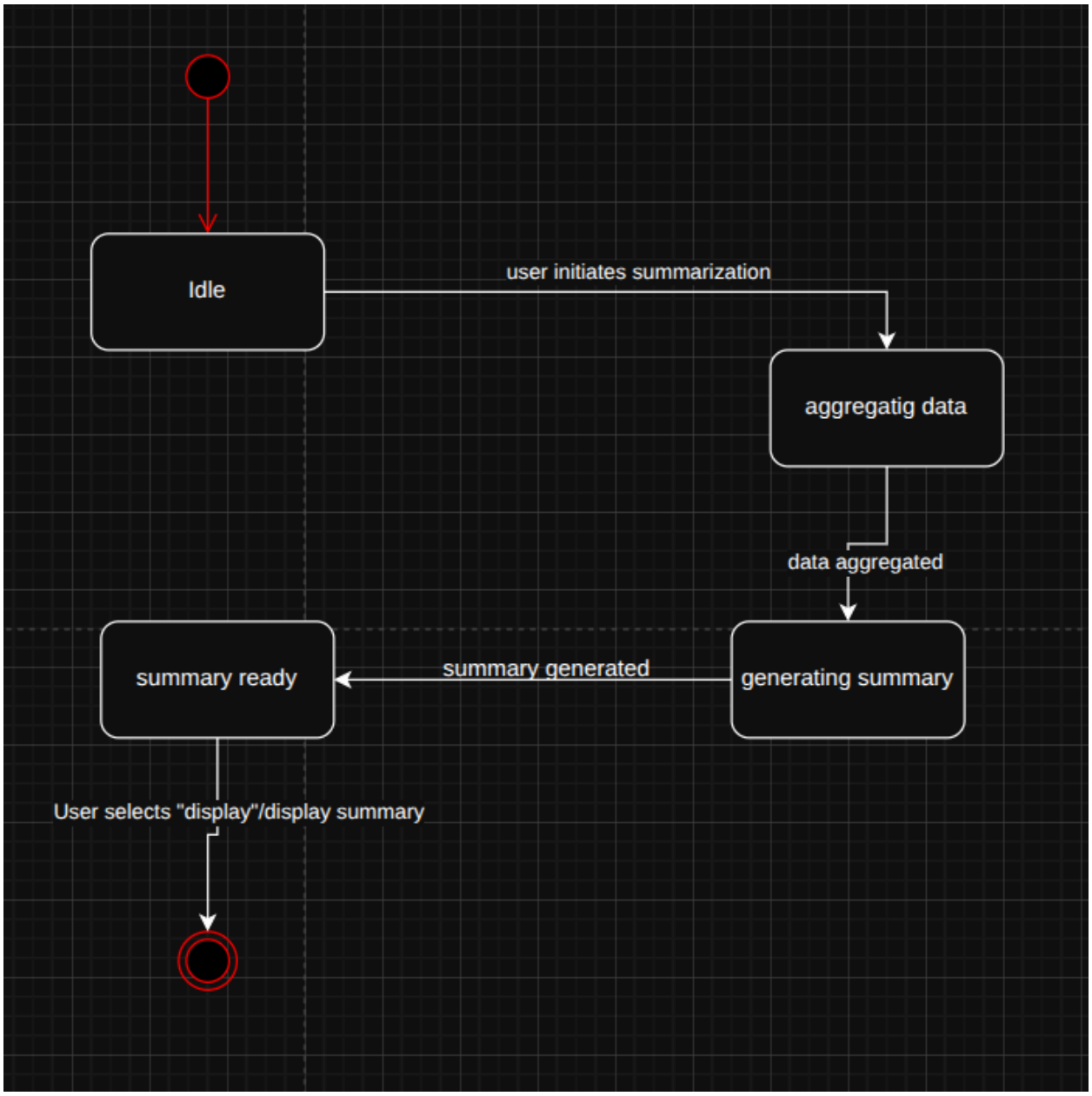
State Diagram 17



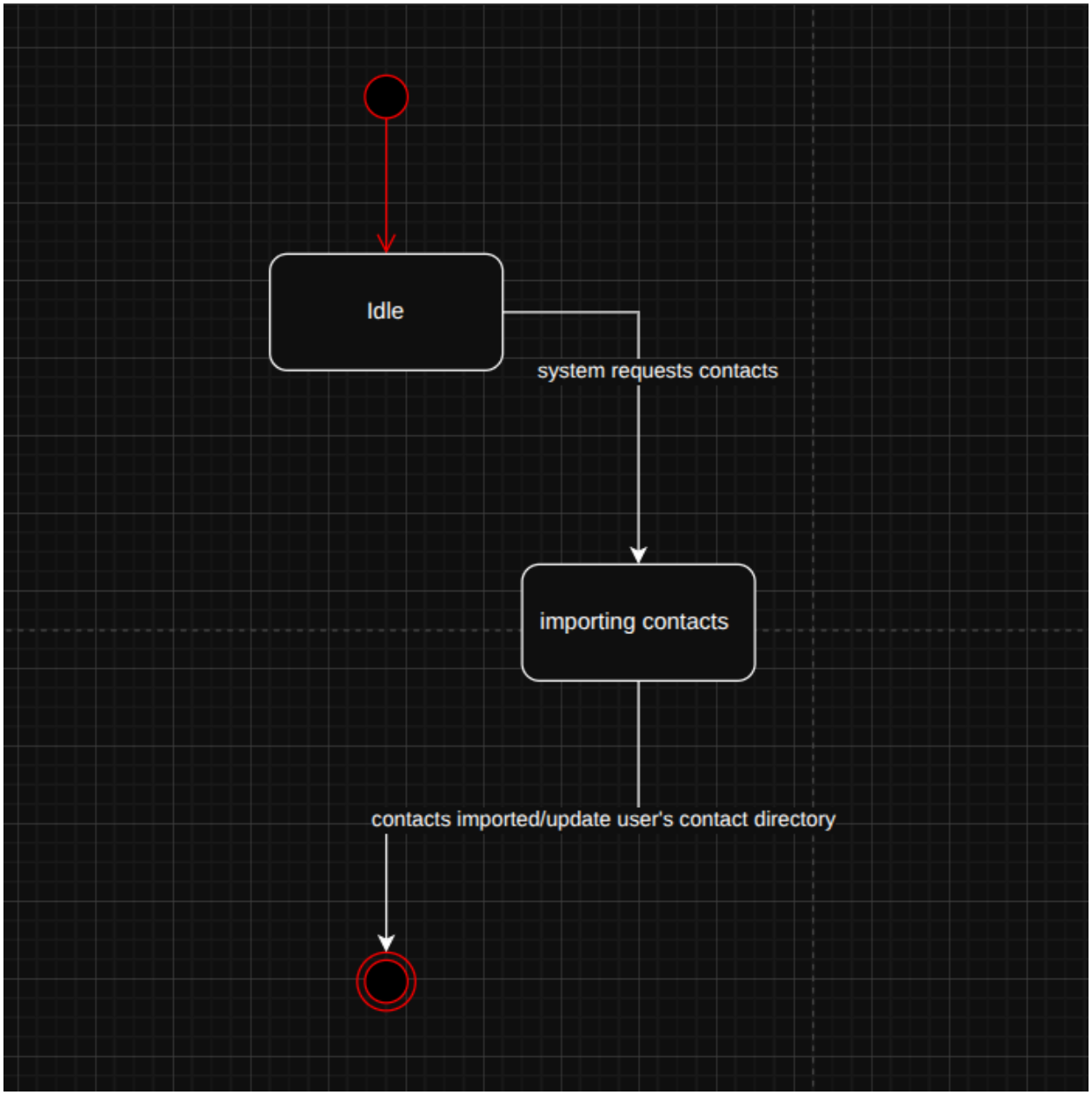
State Diagram 18



State Diagram 19



State Diagram 20



State Diagram 21